

Comment utiliser efficacement le Cross-Validation (CV) pour les Modèles de l'Apprentissage Automatique

Auteur : Abdel YEZZA, Ph.D

Date : Fév. 2026

Réf. utilisée du même auteur : Référence Mathématique des Modèles et Métriques ML

Repository GITHUB : https://github.com/ayezza/ML_CV/

NOTE : des erreurs involontaires peuvent glisser dans ce document, il appartient au lecteur de bien vérifier l'exactitude du contenu avant de procéder à toute expérimentation ou utilisation dans des projets d'entreprises – toute utilisation dans un cadre professionnel uniquement doit référencer l'auteur.

SOMMAIRE

Résumé	3
A. Rappel du principe de CV (Cross-Validation)	4
A1. Validation croisée.....	5
A2. Optimisation des hyperparamètres	6
A2.1 Recherche par grille (Grid Search) : Exploration exhaustive d'une grille de paramètres.....	6
A2.2 Recherche aléatoire (Randomized Search) : Échantillonnage aléatoire d'un nombre fixe (n_{iter}) de combinaisons.....	7
B. 1 ^{er} cas concret de l'application de CV	8
B1. Le dataset ENB_data.csv.....	8
Les variables descriptives et les cibles	8
Utilisation en ligne de commande ou en mode debug.....	13
Configuration globale	14
Configuration des hyperparamètres pour le CV	15
Le pipeline des traitements.....	16
Métriques de Classification	17
Métriques de Régression.....	17
Learning Curves (Courbes d'apprentissage).....	17
Visualisations de l'analyse de validation croisée.....	17
B2. Résultats obtenus pour le dataset ENB_data.csv.....	18
Résultats de la classification.....	19
Résultats de la régression.....	22
B3. Les courbes d'apprentissage par modèle.....	25

Pour les modèles de classification.....	25
Pour les modèles de Régression.....	30
B4. Comparaison des durées de compilation	37
Cas : modèles de classification (CV = 3, 5, 10)	37
Cas : modèles de régression (CV = 3, 5, 10)	38
B5. Les heatmaps des scores.....	39
Cas : modèles de classification (CV = 3, 5, 10)	39
Cas : modèles de régression (CV = 3, 5, 10)	39
C. 2 ^{ème} cas : dataset bike_sharing_hour.csv.....	40
C1. Les variables descriptives et les cibles.....	40
C2. Options de configuration	41
C3. Principaux résultats	42
Résultats de la classification.....	42
Résultats de la régression.....	46
D. 3 ^{ème} cas : dataset weather_data_cleaned.csv	49
D1. Les variables descriptives et les cibles	49
D2. Options de configuration.....	50
D3. Principaux résultats	52
Résultats de la classification.....	52
Exemples des courbes d'apprentissage et matrices de confusion	53
Résultats de la régression.....	55
Exemples des courbes d'apprentissage et matrices de confusion	55
E. Comment quantifier et expliquer l'écart Accuracy vs F1 Score pour une classification binaire	58
E1. Analyse.....	58
E2. Exemple numérique :	60
Annexes.....	64
Annexe 1 : Critères de sélection des modèles	64
Pour la classification.....	64
Pour la régression.....	70

Résumé

Ce guide fournit des informations complètes sur l'utilisation de la méthode de répartition des data en plis (FOLDS) par itération et entraînement/validation, appelée validation croisée (**CV=Cross-Validation**). Elles sont basées sur de nombreux tests réalisés sur 20 modèles d'apprentissage automatique différents (8 modèles de classification et 12 de régression) avec des valeurs de CV égales à 3, 5, 10, 15 et 20 selon la dimension du dataset utilisé :

✅ **Modèles de classification** : RandomForest, Arbre de Décision, SVC, KNN, Régression Logistique, NaiveBayes, Bagging, Stacking

✅ **Modèles de régression** : RandomForest, Arbre de Décision, SVM, KNN, Ridge, Lasso, ElasticNet, GradientBoosting, XGBoost, LightGBM, Bagging, Stacking

➡ **40 expérimentations au total** (20 modèles × 2 valeurs de CV ou plus)

Tout est paramétrable via un unique fichier de configuration.

- Pour accéder au projet GITHUB : [ML_CV](#)
- Pour comprendre les modèles de ML ainsi que les métriques utilisées, veuillez consulter mon article : [Référence mathématique des modèle ML](#)

A. Rappel du principe de CV (Cross-Validation)

Il existe 2 principales manières de procéder dont la complexité d'exécution dépend de plusieurs paramètres, comme cela est expliqué ci-après.

La méthode Search CV (Cross-Validation) a pour objectif d'optimiser le choix des hyperparamètres utilisés par un modèle d'apprentissage automatique, en poursuivant deux finalités essentielles et complémentaires :

1. **Identifier la combinaison optimale d'hyperparamètres** permettant au modèle d'atteindre ses meilleures performances. Le score utilisé pour évaluer cette performance dépend de son type: l'**Accuracy** (généralement) et/ou **F1-score** (pour un jeu de données déséquilibré relativement aux variables descriptives) est privilégié pour la classification, tandis que le score R^2 est généralement retenu pour la régression. Vous pouvez consulter mon article dans ce [lien](#) pour en savoir plus.
2. **Minimiser la marge d'erreur globale** entre les valeurs réelles et les valeurs estimées par le modèle (dans le cas d'une régression) ou les classes réelles vs les classes prédites par le modèle (dans le cas d'une classification).
 - a) **Pour la régression** : Trois principaux indicateurs d'erreur sont couramment utilisés : **MSE** (Mean Squared Error), **RMSE** (Root Mean Squared Error) et **MAE** (Mean Absolute Error, aussi appelée distance de Manhattan), voire en plus le **MAPE** (Mean Absolute Percentage Error). Le choix de l'indicateur dépend notamment de la distribution et de l'échelle des variables du jeu de données. Voir en annexe : [Erreurs quantitatives](#).
 - b) **Pour la classification** : Le plus souvent, on utilise l'AUC et la fonction de perte Log Loss (Fonction de perte d'entropie croisée, voir la référence : [Référence Mathématique des Modèles et Métriques ML](#)) qui principalement utilisée pour pénaliser le niveau de confiance des fausses prédictions. Voir en annexe : [AUC](#) et [AUC1](#).

En quelques mots, la validation croisée est une technique utilisée lors de l'**ajustement des hyperparamètres** afin d'estimer la capacité d'un modèle à généraliser sur des données jamais vues. **ATTENTION !** La valeur de CV influence la **fiabilité de la sélection des hyperparamètres**, mais en aucun cas n'impacte directement les performances finales du modèle.

Il existe principalement deux approches pour la recherche des hyperparamètres optimaux, chacune présentant une complexité d'exécution qui varie selon plusieurs paramètres, comme détaillé dans la suite de ce guide.

La section suivante illustre le principe de manière visuelle plus claire et plus accessible à tout le monde.

A1. Validation croisée

A. K-FOLD : Découpe les données en K sous-ensembles, entraîne sur K-1 et valide sur le dernier.

Principe :

- Découper le dataset en K feuilles distinctes (FOLDS) en fonction de **CV=K $\rightarrow$ K FOLDS distincts**
- Soit $P = \{\text{ensemble des combinaisons possibles des paramètres du modèle}\}$ de cardinalité $= N$. Pour chaque $p = [p_1, p_2, \dots, p_N] \in P$
 - Pour $k = 1$ jusqu'à K :
 - Entraîner le modèle avec les paramètres P sur les $K-1$ folds
 - Valider le modèle sur le FOLD k
 - Retenir : $Score_k = \text{score obtenu sur le fold } k$
 - Calculer le score moyen obtenu qui correspond à p : $Score_p = \left(\frac{1}{K}\right) \sum_{k=1}^K Score_k$
- Le meilleur score retenu avec les paramètres associés : $Score_{best}, p_{best} = \arg \max_p (Score_p)$. On retient alors le modèle avec la combinaison des paramètres : $p_{best} = [p_{1,best}, p_{2,best}, \dots, p_{N,best}]$ ayant le meilleur score correspondant : $Score_{best}$.

Exemple : CV = 5 (5-Fold)

Etape	Ensembles d'entraînement/validation	Explication
Pour chaque $p \in P$:		
1	FOLD 1 → FOLD 2 FOLD 3 FOLD 4 FOLD 5	1. Entraîner le modèle sur FOLD 2, 3, 4, 5 2. Valider le modèle sur le FOLD 1 3. Calculer le score obtenu : $Score_1$
2	FOLD 1 FOLD 2 → FOLD 3 FOLD 4 FOLD 5	1. Entraîner le modèle sur FOLD 1, 3, 4, 5 2. Valider le modèle sur le FOLD 2 3. Calculer le score obtenu : $Score_2$
3	FOLD 1 FOLD 2 FOLD 3 → FOLD 4 FOLD 5	1. Entraîner le modèle sur FOLD 1, 2, 4, 5 2. Valider le modèle sur le FOLD 3 3. Calculer le score obtenu : $Score_3$
4	FOLD 1 FOLD 2 FOLD 3 FOLD 4 → FOLD 5	1. Entraîner le modèle sur FOLD 1, 2, 3, 5 2. Valider le modèle sur le FOLD 4 3. Calculer le score obtenu : $Score_4$
5	FOLD 1 FOLD 2 FOLD 3 FOLD 4 FOLD 5 →	1. Entraîner le modèle sur FOLD 1, 2, 3, 4 2. Valider le modèle sur le FOLD 5 3. Calculer le score obtenu : $Score_5$
6	Calculer le score moyen : $CV_{Score} = \frac{(Score_1 + Score_2 + Score_3 + Score_4 + Score_5)}{5}$	
7	Calculer le meilleur score et le meilleur paramètre correspondant : $Score_{best}, p_{best} = \arg \max_p (Score_p)$ Utiliser le modèle avec le paramètre : $p_{best} = [p_{1,best}, p_{2,best} \dots, p_{N,best}]$ ayant le score : $Score_{best}$	

Contrairement au cycle CV (K-FOLDS) qui fini par visiter tout le dataset dans des cycles distincts, le modèle retenu n'aura pas cette chance ! Dans la pratique, on effectue cette évaluation pour plusieurs valeurs de CV (K) 3, 5, 10, 15, 20...La décision de garder une combinaison particulière des paramètres utilisés pour le modèle final retenu, dépend du choix de CV ayant donné le meilleur score en phase d'évaluation pour chaque valeur de CV. En résumé :

- Pour CV dans $[3, 5, 10, \dots]$:
 - Effectuer l'entraînement/la validation pour obtenir :
 $Score_{CV,best}, p_{CV,best} = \arg \max_p (Score_{CV,p})$
 - Entraîner/Tester le modèle correspondant aux paramètres $p_{CV,best}$
 - Evaluer sa performance (Accuracy, F1_Score, R2...)
- Garder le modèle ayant la meilleure performance obtenue dans 1. avec les paramètres utilisés

- B. **Stratified K-Fold** : Préserve la distribution des classes dans chaque feuille (fold).
- C. **Leave-One-Out (LOOCV)** : $K = n$. Chaque échantillon est utilisé comme validation une fois.

A2. Optimisation des hyperparamètres

A2.1 Recherche par grille (Grid Search) : Exploration exhaustive d'une grille de paramètres.

Pour chaque combinaison $(\theta^1, \theta^2, \dots, \theta_k)$:

$$Score_{\theta} = CV_{score}(model(\theta^1, \dots, \theta_k))$$

$$Best_{\theta} = \arg \max_{\theta} (Score_{\theta})$$

Complexité: $O(|\theta_1| \times |\theta_2| \times \dots \times |\theta_k| \times CV \times n)$; où CV est généralement = 3, 5, 10, 15... et n=dimension du jeu de données

Cette approche convient parfaitement dans le cas de jeu de données de dimension raisonnable (grille ~100 combinaisons possible). Dans le cas de dataset de dimension trop importante, il convient d'opter pour :

A2.2 Recherche aléatoire (Randomized Search) : Échantillonnage aléatoire d'un nombre fixe (n_iter) de combinaisons.

Pour $i = 1$ jusqu'à n_iter :
 $\theta \sim$ échantillon des distributions
 $Score_i = CV_{score}(model(\theta))$

$Best_\theta = \arg \max_i (Score_i)$

Complexité: $O(n_iter \times CV \times n)$

Avantages: temps d'exécution fixe, souvent le score est proche de l'optimal de $\sim 1-2\%$

Pour comparer la complexité des deux (Grid Search vs Random Search) si on utilise le même CV, il suffit de comparer :

$(|\theta_1| \times |\theta_2| \times \dots \times |\theta_k|) \sim n_iter$

Exemple 1: Pour un modèle avec un # de combinaisons possibles des hyperparamètres = 12

Random Search : Avec $n_iter=20$ et $CV=5$, on a besoin de $5 \times 20 = 100$ fits (ajustement des paramètres) du modèle

Grid Search : Avec $CV=5$, on a besoin de $5 \times 12 = 60$ fits (40 fits de moins que le Random Search)

Exemple 2: Pour un modèle avec un # de combinaisons possibles des hyperparamètres 10 fois plus que le précédent = 120

Random Search : Avec $n_iter=20$ et $CV=5$, on a besoin de $5 \times 20 = 100$ fits (ajustements) du modèle

Grid Search : Avec $CV=5$, on a besoin de $5 \times 120 = 600$ fits (6 fois plus que le Random Search !)

Random Search illustré (CV=5):

FOLD 1	Entraînement des data sur 80% (4/5 du total), validation sur le reste 20% (1/5)
FOLD 2	Pour 20 combinaisons aléatoires (par échantillonnage):
FOLD 3	1. Répéter 2,3 et 4 pour chaque combinaison des hyperparamètres :
FOLD 4	2. Entraîner sur 4 FOLDS (80%), Valider sur 1 FOLD (20%)
FOLD 5	3. Faire la même chose sur les 5 FOLDS
FOLD 4	4. Calculer la Moyenne des 5 scores obtenus des 5 validations (5 FOLDS)
FOLD 5	5. Garder le meilleur (valeur MAX) des 20 scores obtenus (à partir de 4.)

Avantages

1. Beaucoup plus rapide (20 à 30 fois plus rapide pour les grandes grilles)
2. Temps de calcul fixe (contrôlé par le nombre d'itération fixe n_iter)
3. Bonne exploration de l'espace des paramètres
4. Trouve généralement des hyperparamètres à **1~2 %** de l'optimal

Inconvénients

1. Ne garantit pas toujours de trouver la solution optimale absolue
2. Non déterministe (résultats différents à chaque exécution, sauf si **random_state** est défini)
3. Peut ne pas trouver l'optimum, notamment si n_iter est trop faible (la découverte des données pas assez suffisante)

B. 1^{er} cas concret de l'application de CV

Bien que le projet indiqué en début de ce guide puisse s'appliquer à tout jeu de données répondant aux conditions préalables, on va prendre un jeu de données classique très connu sur lequel une panoplie de projets existe dans le domaine public.

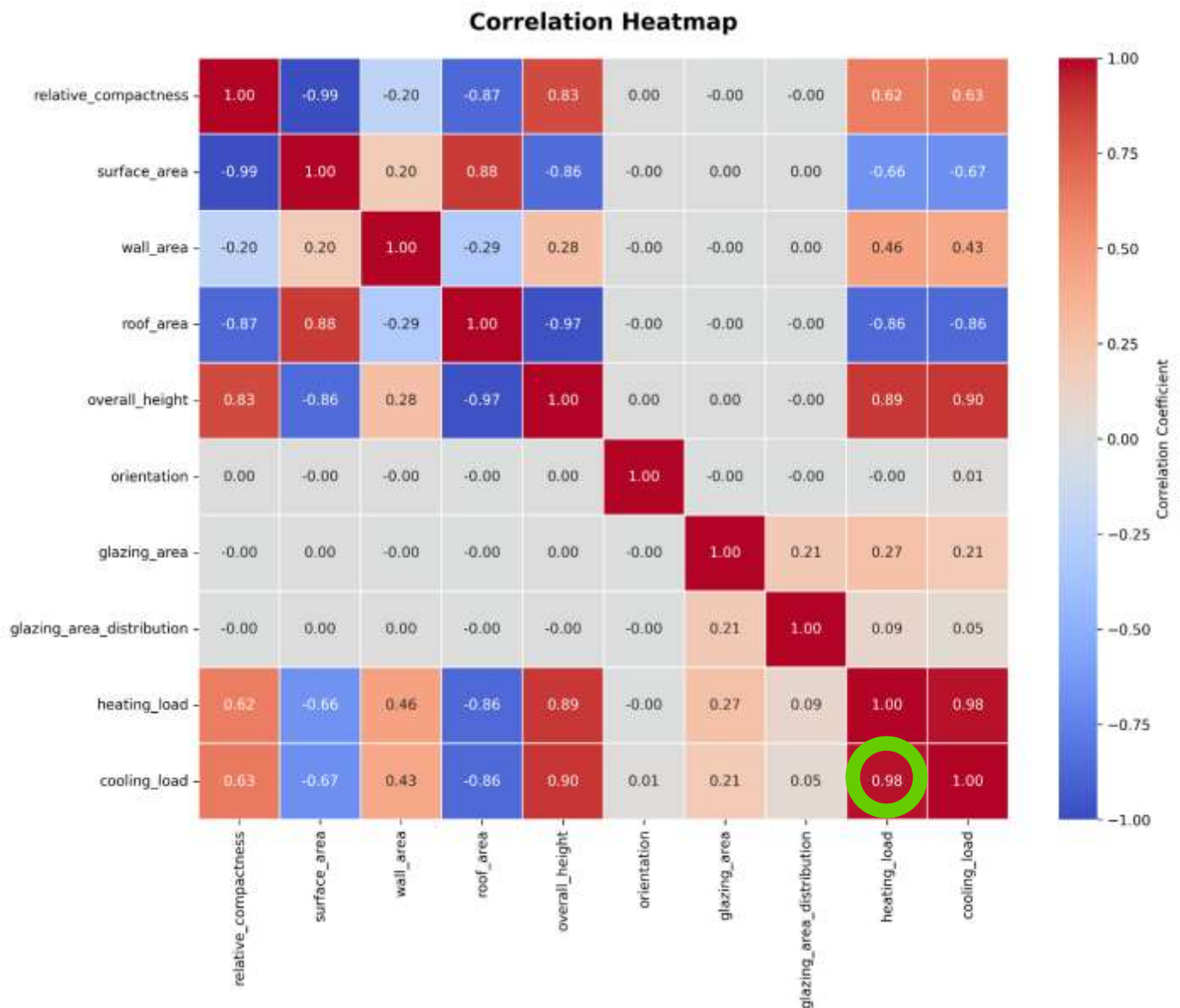
B1. Le dataset ENB_data.csv

- **Dataset CSV** : ENB_data.csv
- **URL 1**: <https://www.kaggle.com/datasets/elikplim/eergy-efficiency-dataset>
- **URL Espace GITHUB de ce guide** :
https://github.com/ayezza/ML_CV/blob/main/data/ENB_data.csv
- **Dimension du dataset** : 768 x 10 → 7 680 valeurs

Les variables descriptives et les cibles

Variable	Description	Descriptive OU cible (Target)
relative_compactness	Compacité relative	Descriptive
surface_area	Surface au sol	Descriptive
wall_area	Surface des murs	Descriptive
roof_area	Surface du toit	Descriptive
overall_height	Hauteur totale	Descriptive
orientation	Orientation (catégorielle)	Descriptive
glazing_area	Surface vitrée	Descriptive
glazing_area_distribution	Répartition de la surface vitrée (catégorielle)	Descriptive
heating_load	Besoin de chauffage	Target
cooling_load	Besoin de refroidissement	Target

Une 1^{ère} analyse des corrélations entre l'ensemble des variables exprimée par le graphe de type carte thermique (heatmap), montre l'existence d'une corrélation presque parfaite avec un coefficient de **0.98** entre les variables cibles **heating_load** et **cooling_load** :



Ce constat, nous permet de combiner les deux variables cibles en une seule par le biais d'une agrégation qui permet de déterminer la classe de chaque variable pour les modèles de classification et les valeurs de chacune des variables pour les modèles de régression.

B2. Agrégation

Rappelons qu'une agrégation du point de vue purement mathématique est tout simplement une fonction réelle à 2 ou plusieurs variables :

$$f: R^n \rightarrow R, y = f(x_1, x_2, \dots, x_n)$$

L'agrégation peut être appliquée aussi bien aux variables descriptives que cibles en fonction de l'impact sur le modèle et les performances obtenues. Dans notre cas, on va plutôt s'en servir pour les deux variables cibles ($x_1 = \text{heating_load}$, $x_2 = \text{cooling_load}$) en y appliquant plusieurs agrégations pour en choisir celle qui fournit les meilleures performances, mais aussi les prédictions individuelles, autrement dit, trouver x_1, x_2 à partir de la valeur agrégée.

Il existe déjà une multitude de fonctions d'agrégation disponibles en Python, comme vous pouvez définir la vôtre (voir **conf.py** du projet ou [ci-dessous](#)). Une partie seulement est décrite ci-dessous ; je vous laisse voir leurs implémentations dans le projet y compris les agrégations personnalisées. De préférence, optez pour une agrégation inversible en la combinant avec une autre équation dans le cas de 2 variables (le plus simple).

Si on pose : $h = \text{heating_load}$, $c = \text{cooling_load}$, $n = \text{dimension du dataset}$

Agrégation	Formule	Coût	Avantage
sum	$h + c$	O(n)	Simple, interprétable et surtout réversible en connaissant la relation entre les 2 variables h, c
mean	$(h + c) / 2$	O(n)	Moyennes des énergies
euclidean	$\sqrt{h^2 + c^2}$	O(n)	L2 norm
manhattan	$ h + c $	O(n)	L1 norm
chebyshev	$\max(h , c)$	O(n)	L_∞ norm, charge la plus haute des deux
minkowski	$\sqrt[3]{h^3 + c^3}$	O(n)	L3 norm
seuclidean	$\sqrt{\left(\frac{h^2}{\text{VAR}(h)} + \frac{c^2}{\text{VAR}(c)}\right)}$	O(n)	Normalisée à l'échelle
...			

Dans notre cas, on peut prendre une agrégation simple, la somme des deux variables cibles :

$$f: R^2 : (\text{heating_load} + \text{cooling_load}) \rightarrow \text{charges_sum} = \text{heating_load} + \text{cooling_load}$$

De sorte que la variable **charges_sum** devient la variable cible pour notre modèle.

Le projet propose plus d'une dizaine de fonction d'agrégation à deux variables dont une partie est inversible. A noter que dans le cas d'un modèle de classification, il n'est pas exigé que l'agrégation doit être inversible, si on cible uniquement la classe prédite par le modèle sur la base de la valeur agrégée !

Exemple de fonction d'agrégation 'euclidean' :

Posons : $x_1 = \text{heating_load} = h, x_2 = \text{cooling_load} = c$

Il s'agit de résoudre le système d'équation :

$$\begin{cases} \sqrt{h^2 + c^2} = p \text{ (} p \text{ étant la valeur prédite)} \\ c = ah + b \end{cases}$$

La 2^{ème} équation provient de la corrélation forte existante entre les variable h et c (0.98, voir le heatmap qui le montre clairement) :

$$c \approx 0.920 \times h + 4.064 \text{ (} r \approx 0.98 \text{)}$$

Autrement dit, : $a = 0.920, b = 4.064$ et il s'agit de résoudre le système d'équations :

$$\begin{cases} \sqrt{h^2 + c^2} = p \text{ (} p \text{ étant la valeur prédite)} \\ c = 0.92h + 4.064 \end{cases}$$

Cela donne :

$$\begin{cases} (1 + a^2)h^2 + 2abh + (b^2 - p^2) = 0 \\ c = ah + b \end{cases}$$

Le discriminant de la 1^{ère} équation de 2nd degré en h donne :

$$\Delta = 4[p^2(a^2 - 1) - b^2]$$

Par conséquent,

$$\Delta > 0 \Leftrightarrow p > \frac{b}{\sqrt{1 + a^2}}$$

Comme cette dernière inégalité est toujours vraie, alors les 2 solutions possibles sont :

$$\begin{cases} h_1 = \frac{-ab + \sqrt{p^2(1 + a^2) - b^2}}{1 + a^2} \\ h_2 = \frac{-ab - \sqrt{p^2(1 + a^2) - b^2}}{1 + a^2} \end{cases}$$

La solution h_2 est écartée du fait qu'elle est toujours négative. Par conséquent, l'unique solution du système d'équations acceptée est :

$$\begin{cases} h_1 = \frac{-ab + \sqrt{p^2(1 + a^2) - b^2}}{1 + a^2} \\ c_1 = ah_1 + b = \frac{b - a\sqrt{p^2(1 + a^2) - b^2}}{1 + a^2} \end{cases}$$

Il reste de s'assurer toutefois que h_1 est toujours positive, ce qui est équivalent à $p > b$, autrement dit, toutes les prédictions doivent être supérieures à $b = 4.064$. Or en examinant le dataset, la valeur minimale de p peut être calculée facilement pour obtenir la valeur $p_{\min} = 12.48 < b = 4.064$. Cette valeur est fournie systématiquement par le projet référencé en début de cet article lors de la détermination du nombre de classes en appliquant les méthodes **Silhouette**, **Calinski-Harabasz** et **Davies-Bouldin**. Autrement, une méthode simple et directe est la suivante :

```
import pandas as pd

if __name__ == "__main__":
    # Create a sample DataFrame
    df = pd.read_csv("ENB_data.csv")
    df['euclidean_agg'] = ((df['heating_load'] ** 2 + df['cooling_load'] ** 2) ** 0.5)
    # get the min and max of the new column
    min_value = df['euclidean_agg'].min()
    max_value = df['euclidean_agg'].max()
    print(f"Min value of euclidean_agg: {min_value}")
    print(f"Max value of euclidean_agg: {max_value}")
```

Ce qui donne en sortie :

Min value of euclidean_agg: **12.476173291518517**

Max value of euclidean_agg: 63.75082195548541

En conclusion, si on utilise l'agrégation 'euclidean', à partir de n'importe quelle prédiction via un modèle de régression, on peut retrouver les variables cibles agrégées. Cela a un intérêt, car on ne prédit qu'une variable au lieu de 2 (moins de dimensions), et par conséquent, peut être plus performant en temps d'exécution.

A vous de tester les autres fonctions d'agrégation ! Les tests ci-dessous utiliseront l'agrégation 'sum' la plus simple (voir [Configuration globale](#)).

Utilisation en ligne de commande ou en mode debug

Syntaxe avec tous les paramètres (non obligatoires)

```
python cv_experiment.py [--cv CV_VALUES] [--models MODEL_NAMES] [--task TASK_TYPE]
```

Arguments:

```
# --cv      Space-separated CV fold values (default: 3 5)
# --models  Space-separated supported model names (default: all)
# --task    classification, regression, or both (default: both)
```

Exemples:

```
python cv_experiment.py --cv 3 5 10 15 --task classification
python cv_experiment.py --models XGBoost LightGBM --cv 5 10
python cv_experiment.py --models RandomForest --cv 3 5 10 --task both
```

Si vous êtes sous Windows et souhaitez avoir la sortie dans un fichier texte uniquement pour les modèles de classification, vous pouvez lancer par exemple :

```
python -X utf8 cv_experiment.py --cv 3 5 10 --task classification 2>&1 > .\output\output_CLF.txt
```

Pour la régression uniquement :

```
python -X utf8 cv_experiment.py --cv 3 5 10 --task regression 2>&1 > .\output\output_REG.txt
```

Pour les deux à la fois :

```
python -X utf8 cv_experiment.py --cv 3 5 10 2>&1 > .\output\output_CLF_REG.txt
```

Si vous utilisez **VSCode**, vous pouvez effectuer le **debugging** via le **launcher.json** en y indiquant les paramètres de la ligne de commande souhaités :

Exemple pour la classification :

```
{
  "name": "CV Experiment - Classification",
  "type": "debugpy",
  "request": "launch",
  "python": "${workspaceFolder}/venv/Scripts/python.exe",
  "program": "./cv_experiment.py",
  "console": "integratedTerminal",
  "pythonArgs": ["-X", "utf8"],
  "args": ["--cv", "3", "5", "10", "--task", "classification"]
}
```

Ce paramétrage est équivalent à la commande suivante, mais avec l'accessibilité de l'interface de debug en temps réel :

```
python -X utf8 cv_experiment.py --cv 3 5 10 --task classification
```

Configuration globale

L'ensemble des paramètres sont modifiables dans un fichier unique **conf.py** dont les sections sont les suivantes :

Dataset

```
# Path to your dataset
DATA_PATH = DATA_DIR / 'ENB_data.csv'

# CSV delimiter (auto-detect or specify explicitly)
CSV_DELIMITER = 'auto' # Options: 'auto', ',', ';', '\t'

# Columns to aggregate for target variable
# case : Single column: No aggregation needed, used directly as target
AGGREGATION_COLS = ['Temperature (C)']
# Case : Multiple columns: Aggregation function combines them into a single target
AGGREGATION_COLS = ['heating_load', 'cooling_load']

# Aggregation method (ignored for single column)
AGGREGATION_NAME = 'sum' # Options: 'sum', 'mean', 'euclidean', 'manhattan', etc.

# Columns to exclude from features (will not be used for training)
EXCLUDE_COLS = ['id', 'Daily Summary']
```

Data Preprocessing

```
# Outlier removal using IQR method
REMOVE_OUTLIERS = True # Set to False to skip
OUTLIER_IQR_MULTIPLIER = 1.5 # 1.5 = standard, 3.0 = extreme outliers only

# Polynomial features for linear regression models
# Only applied to: Ridge, Lasso, ElasticNet, LinearRegression
# Tree-based models (RF, GBT, XGBoost, LightGBM) use original features
USE_POLYNOMIAL_FEATURES = False
POLYNOMIAL_DEGREE = 2 # 2 = quadratic, 3 = cubic (slower)
```

Model Settings

```
RANDOM_STATE = 42 # Reproducibility seed
TEST_SIZE = 0.2 # Train/test split ratio
CV_FOLDS = 10 # Default cross-validation folds
N_JOBS = -1 # Use all CPU cores, set this value if your CPU resources are limited

# Hyperparameter tuning strategy
SEARCH_TYPE = 'random' # 'grid' (exhaustive, for small/medium dim datasets) or
                        'random' (faster, for high dim datasets)
```

Classification Settings

```
N_CLASSES = 4 # set to 'auto' or to a fixed number of classes
CLUSTERING_METHOD = 'kmeans' # 'kmeans' or 'qcut' for label creation
```

Learning Curves

```
GENERATE_LEARNING_CURVES = True # Set to False to skip (saves time)
```

Categorical Encoding Settings

```
# Columns with unique values <= threshold will be encoded
# Columns with unique values > threshold will be dropped
```

```
CATEGORICAL_ENCODING_THRESHOLD = 10    # Max unique values for encoding

# Encoding method:
# 'Label' (LabelEncoder) or
# 'onehot' (OneHotEncoder) - increases the dataset dimension
CATEGORICAL_ENCODING_METHOD = 'label'
```

Configuration des hyperparamètres pour le CV

Si vous souhaitez modifier le dictionnaire contenant les hyperparamètres à utiliser pour chaque modèle de classification (CLF_) ou de régression (REG_) lors de l'exécution du CV (en mode grid : [GridSearchCV](#) ou en mode random : [RandomizedSearchCV](#)) sans modifier le core, vous pouvez le faire dans `hyperparameters.py` :

```
# uncomment and modify to customize CV hyperparameters
CLF_PARAM_GRIDS = {
    'RandomForest': {
        'n_estimators': [50, 100, 200],
        'max_depth': [None, 5, 10],
    },
    ...
}

REG_PARAM_GRIDS = {
    'Lasso': {
        'alpha': [0.001, 0.01, 0.1, 1.0, 10.0],
    },
    ...
}
```

Dans le cas où vous utilisiez le mode 'random' pour la recherche CV (`SEARCH_TYPE = 'random'`), vous avez la possibilité d'ajuster le nombre d'itérations pour chaque modèle afin de caper le # d'itérations total (car cela dépend du nombre total des combinaisons des hyperparamètres par modèle) :

```
CLF_N_ITER = {
    'RandomForest': 30,
    'DecisionTree': 25,
    'SVC': 20,
    'KNN': 20,
    'LogisticRegression': 25,
    'NaiveBayes': 8,
    'Bagging': 15,
    'Stacking': 2,
}

REG_N_ITER = {
    'RandomForest': 30,
    'DecisionTree': 25,
    'SVM': 20,
    'KNN': 20,
```

```

'LinearRegression': 2,
'Ridge': 15,
'Lasso': 10,
'ElasticNet': 15,
'GradientBoosting': 30,
'XGBoost': 25,
'LightGBM': 25,
'Bagging': 15,
'Stacking': 2,
}

```

Le pipeline des traitements

1. **Load du Dataset** - Auto-detect CSV delimiter (comma, semicolon, tab)
↓
2. **Ecarter les Outliers** - IQR method (configurable multiplier)
↓
3. **Créer les variables cibles** - Aggregation + classification labels (qcut/kmeans)
↓
4. **Split features/cible** - Separate features (X) from targets (y)
↓
Handle dates, categoricals, excluded columns
5. **Train/Test Split** - Separate splits for classification and regression
↓
6. **Feature Scaling** - StandardScaler (fit on train only)
↓
7. **Polynomial (opt.)** - PolynomialFeatures for linear models only if set
↓
8. **Model Training** - Hyperparameter tuning (Grid/Random search)
↓
Per-model, per-CV-value learning curves
9. **Evaluation** - Metrics collection, visualizations, reports
↓
10. **Summary** - Learning curves summary table, CV analysis

*les étapes 2. Et 7. Sont optionnelles

En détails :

Métriques de Classification

- Accuracy, Precision, Recall, F1-Score (Macro & Micro)
- AUC-ROC (micro/macro)
- Matrices de confusion
- Courbes ROC par classe

Métriques de Régression

- R2 Score
- RMSE
- MAE
- MAPE

Learning Curves (Courbes d'apprentissage)

- Par modèle, par valeur de CV
- Diagnostique de l'Overfitting/l'Underfitting (sur-apprentissage/sous-apprentissage) par modèle
- Synthèse comparant tous les modèles avec CV, le gap et diagnostics

Visualisations de l'analyse de validation croisée

- Graphes pour comparer par modèle les performances par CV
- Durée des traitements vs valeur de CV
- Heatmaps des métriques de performances
- Par modèle, les meilleurs paramètres ayant produit le meilleur score

B2. Résultats obtenus pour le dataset ENB_data.csv

Bien que la cible de ce dataset requiert plutôt une modélisation de régression, on va l'utiliser aussi pour la classification dans un but académique. En outre, afin de disposer plus rapidement des résultats, on opte pour la recherche *random* au lieu de *grid*.

Liste de toutes les variables :

```
['relative_compactness', 'surface_area', 'wall_area', 'roof_area', 'overall_height',
'orientation', 'glazing_area', 'glazing_area_distribution', 'heating_load', 'cooling_load']
```

Liste des variables cibles :

```
['heating_load', 'cooling_load']
```

Variable cible agrégée et nombre de classes trouvé :

En fixant le paramètre AGGREGATION_NAME = 'sum',
la variable cible agrégée = 'heating_load' + 'cooling_load'

Détermination du # de classes :

En fixant le nombre de classes de N_CLASSES = 'auto', l'utilisation des 4 méthodes sur une échelle de 10 au maximum pour déterminer le nombre de classes, fournit :

k	Silhouette	Calinski-H	Davies-B	Inertia

2	0.7165	3410.43	0.3827	281.72
3	0.6387	3881.90	0.5588	137.81
4	0.5967	3766.36	0.5918	97.28
5	0.5091	4294.11	0.6701	65.33
6	0.4999	4307.62	0.6989	52.49
7	0.4690	4324.41	0.6930	43.78
8	0.4695	4473.60	0.6919	36.39
9	0.5030	4594.75	0.6818	31.08
10	0.4889	4837.73	0.6825	26.29

La méthode Silhouette souvent utilisée, détermine le # de classes ayant la valeur la plus grande. Donc N_CLASSES = 2.

Clusters trouvés :

L'utilisation de l'algorithme KMEANS pour déterminer les 2 clusters des 2 classes, fournit :

```
Class 0: 398 samples
Range: [16.95, 46.84]
Class 1: 370 samples
Range: [48.15, 89.95]
```

Parfaitement équilibré en termes d'effectifs 51% vs 49%, mais sans trop d'écart en termes de l'étendu des intervalles des valeurs 41% vs 59%.

Ligne de commande :

```
python -X utf8 cv_experiment.py --cv 3 5 2>&1 >.\output\ output_KMEANS_RANDOM_REG_CLF_ENB_data.txt
```

Résultats de la classification

Model	CV_Folds	Tuning_Time_sec	Best_CV_Score	Test_Accuracy	Test_F1
RandomForest	3	2.736993	0.998359	0.993506	0.993500
	5	1.353302	0.998355	0.993506	0.993500
	10	2.540750	0.998354	0.993506	0.993500
DecisionTree	3	0.097998	0.998359	0.993506	0.993500
	5	0.071720	0.998355	0.993506	0.993500
	10	0.125808	0.998354	0.993506	0.993500
SVC	3	0.079792	0.996729	0.993506	0.993500
	5	0.127197	0.996725	0.993506	0.993500
	10	0.220844	0.996707	0.993506	0.993500
KNN	3	0.165425	0.988588	0.993506	0.993500
	5	0.085722	0.991838	0.993506	0.993500
	10	0.177057	0.991833	0.993506	0.993500
LogisticRegression	3	0.250487	0.996729	0.993506	0.993500
	5	0.484294	0.998355	0.993506	0.993500
	10	1.104319	0.996712	0.993506	0.993500
NaiveBayes	3	0.060446	0.980440	0.987013	0.986978
	5	0.048306	0.980428	0.987013	0.986978
	10	0.072839	0.980444	0.987013	0.986978
Bagging	3	0.958246	0.998359	0.993506	0.993500
	5	1.490452	0.998355	0.993506	0.993500
	10	2.653427	0.998354	0.993506	0.993500
Stacking	3	1.132461	0.995100	0.993506	0.993500
	5	1.426672	0.996725	0.993506	0.993500
	10	1.902592	0.996737	0.993506	0.993500

Meilleure valeur de CV: 3 folds

Test Accuracy: 0.99351

Durée du Tuning (valeur relative, dépend du host d'exécution et du nombre total des combinaisons des hyperparamètres): 2.74s

NOTE importante :

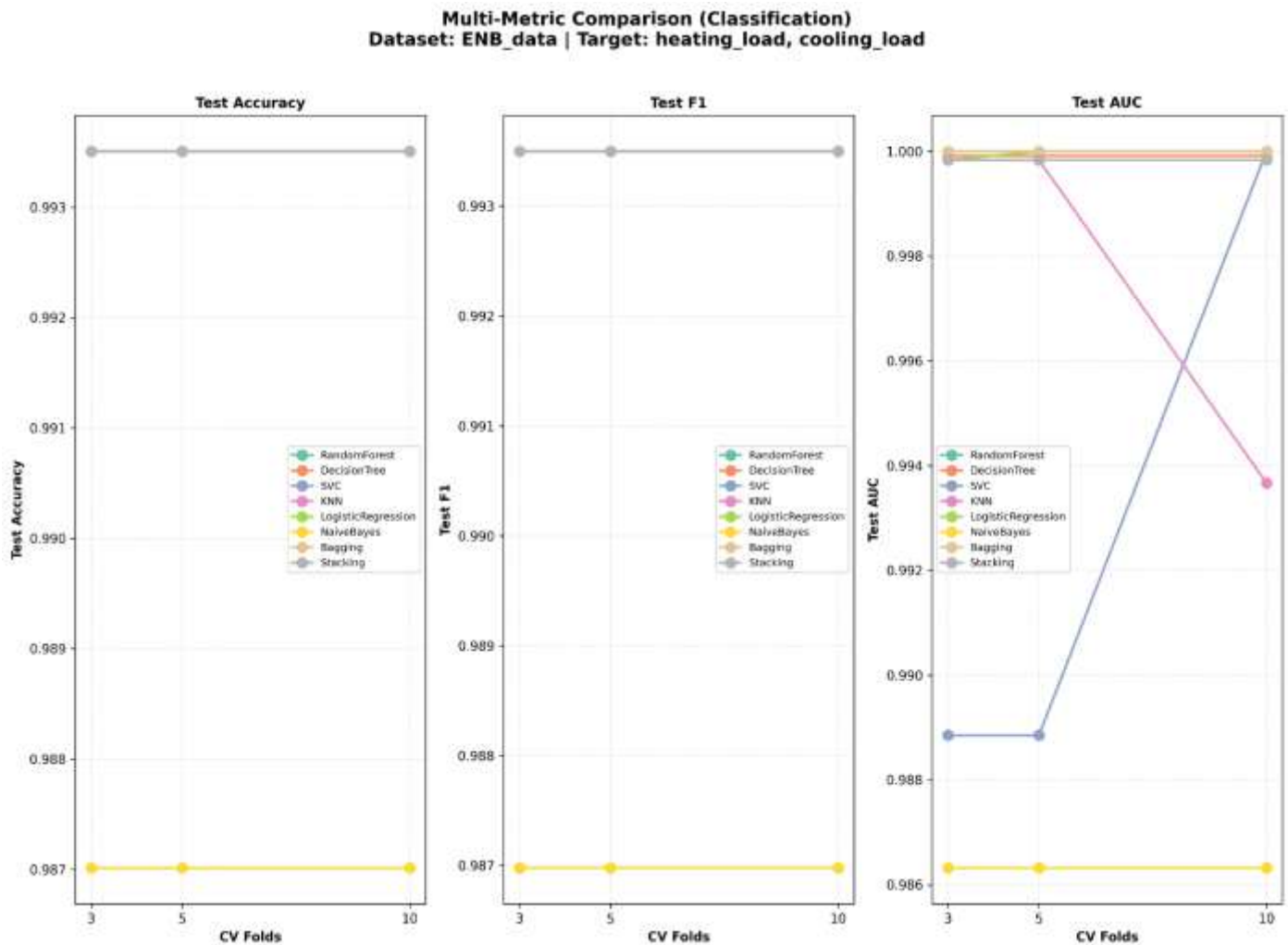
Sur la base de ce qui différencie l'**Accuracy** de **F1_Score** ce sont les **TN**. Dans le cas où l'écart entre **Test_Accuracy** et **Test_F1** est trop important, cela signifie que :

1. Le modèle est fortement biaisé en faveur plutôt de la classe majoritaire
2. Il excelle en Accuracy grâce à la prédiction correcte de la classe négative (**TN**)
3. Au contraire, il possède une faible performance pour la classe positive (faible **Précision** et/ou **Rappel**)

Cela n'est pas le cas de notre modèle qui est parfaitement équilibré, voire trop ! Le gap entre l'**Accuracy** de **F1_Score** est quasiment nul pour toutes les compilations.

Pour une interprétation plus mathématique, voir : [E. Comment quantifier et expliquer l'écart Accuracy vs F1 Score pour une classification binaire.](#)

Avec CV = 3, 5, 10, les métriques **Accuracy**, **F1_SCORE** et **AUC** évoluent selon les 3 graphes suivants :



Pour une valeur de CV donnée :

1. Plus l'**Accuracy/F1_Score** sont proches de 1, c'est meilleur
2. Plus l'**AUC** est proche de 1, c'est mieux pour des niveaux **Accuracy/F1_Score** assez proches les uns des autres

A l'exception du modèle **NaiveBayes**, tous les modèles possèdent presque la même **Accuracy/F1_Score**.

On note qu'une valeur importante de CV n'assure pas nécessairement l'obtention de meilleurs résultats. En clair, CV = 3 suffit et à titre d'exemple, le modèle **Bagging** fournit les meilleurs résultats pour l'ensemble des critères. La synthèse se présente comme suit :

Modèle	CV_Folds	Best_Params
RandomForest	3	{'n_estimators': 100, 'min_samples_split': 12, 'min_samples_leaf': 1, 'max_features': 'log2', 'max_depth': None}
	5	{'n_estimators': 100, 'min_samples_split': 12, 'min_samples_leaf': 1, 'max_features': 'log2', 'max_depth': None}
	10	{'n_estimators': 100, 'min_samples_split': 12, 'min_samples_leaf': 1, 'max_features': 'log2', 'max_depth': None}
DecisionTree	3	{'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt', 'max_depth': 5, 'criterion': 'entropy'}
	5	{'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt', 'max_depth': 5, 'criterion': 'entropy'}
	10	{'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt', 'max_depth': 5, 'criterion': 'entropy'}
SVC	3	{'kernel': 'rbf', 'gamma': 'scale', 'class_weight': 'balanced', 'C': 100}
	5	{'kernel': 'rbf', 'gamma': 'scale', 'class_weight': 'balanced', 'C': 100}
	10	{'kernel': 'rbf', 'gamma': 0.1, 'class_weight': None, 'C': 10}
KNN	3	{'weights': 'distance', 'n_neighbors': 9, 'metric': 'manhattan'}
	5	{'weights': 'distance', 'n_neighbors': 9, 'metric': 'manhattan'}
	10	{'weights': 'uniform', 'n_neighbors': 3, 'metric': 'euclidean'}
LogisticRegression	3	{'solver': 'lbfgs', 'penalty': None, 'max_iter': 5000, 'class_weight': None}
	5	{'solver': 'liblinear', 'penalty': 'l1', 'max_iter': 5000, 'class_weight': None, 'C': 10}
	10	{'solver': 'saga', 'penalty': 'elasticnet', 'max_iter': 5000, 'l1_ratio': 0.7, 'class_weight': 'balanced', 'C': 100}
NaiveBayes	3	{'var_smoothing': 1e-11, 'priors': None}
	5	{'var_smoothing': 1e-11, 'priors': None}
	10	{'var_smoothing': 1e-11, 'priors': None}
Bagging	3	{'n_estimators': 100, 'max_samples': 1.0, 'max_features': 0.6}
	5	{'n_estimators': 100, 'max_samples': 1.0, 'max_features': 0.6}
	10	{'n_estimators': 100, 'max_samples': 1.0, 'max_features': 0.6}
Stacking	3	{'cv': 3}
	5	{'cv': 5}
	10	{'cv': 3}

La synthèse de l'ensemble des modèles traités se trouve dans le dossier : **output\reports** au format EXCEL, ex., **cv_experiment_all_models_kmeans.xlsx**.

Résultats de la régression

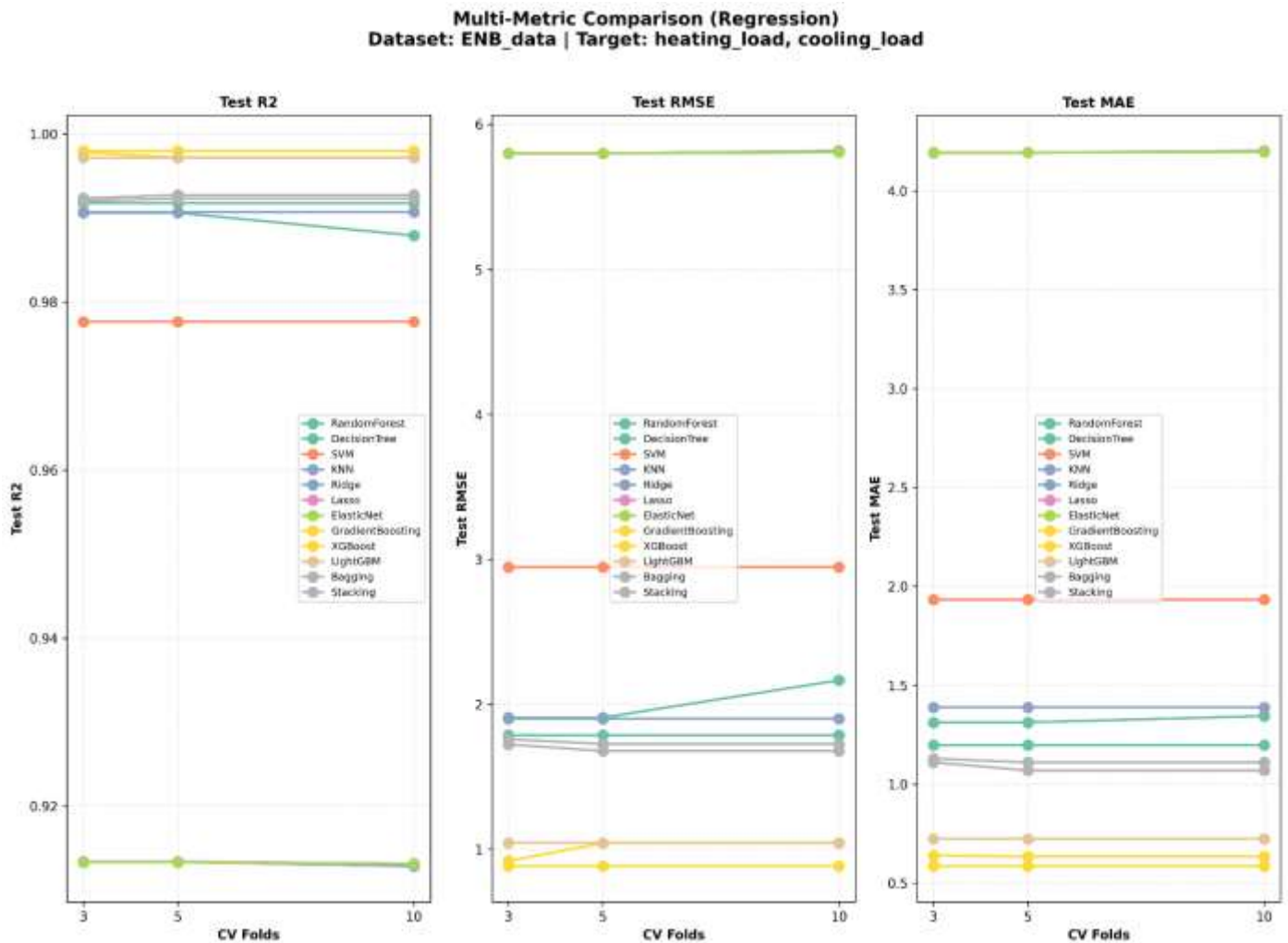
Model	CV_Folds	Tuning_Time_sec	Best_CV_Score	Test_R2	Test_RMSE
RandomForest	3	0.723943	0.991165	0.991777	1.786977
	5	1.152024	0.991551	0.991777	1.786977
	10	2.140132	0.991818	0.991777	1.786977
DecisionTree	3	0.071713	0.988402	0.990622	1.908332
	5	0.110323	0.989128	0.990622	1.908332
	10	0.149401	0.988818	0.987922	2.165750
SVM	3	0.346546	0.949317	0.977664	2.945176
	5	0.631534	0.959393	0.977664	2.945176
	10	1.137473	0.966325	0.977664	2.945176
KNN	3	0.071435	0.975327	0.990700	1.900420
	5	0.097203	0.980965	0.990700	1.900420
	10	0.151746	0.982934	0.990700	1.900420
Ridge	3	0.101103	0.909534	0.913364	5.800391
	5	0.090261	0.908761	0.913364	5.800391
	10	0.128299	0.909046	0.912851	5.817546
Lasso	3	0.039761	0.909531	0.913393	5.799449
	5	0.053559	0.908755	0.913394	5.799414
	10	0.077818	0.909059	0.913146	5.807688
ElasticNet	3	0.087367	0.909533	0.913312	5.802138
	5	0.125499	0.908763	0.913312	5.802138
	10	0.216329	0.909085	0.913124	5.808431
GradientBoosting	3	0.664514	0.996282	0.997825	0.919125
	5	1.130923	0.997423	0.997199	1.042999
	10	1.994915	0.997856	0.997199	1.042999
XGBoost	3	3.696587	0.996549	0.997992	0.883167
	5	0.489694	0.997435	0.997992	0.883167
	10	0.790384	0.997716	0.997992	0.883167
LightGBM	3	6.178935	0.995943	0.997191	1.044529
	5	4.136097	0.996768	0.997191	1.044529
	10	11.697350	0.996928	0.997191	1.044529
Bagging	3	1.212543	0.991424	0.992035	1.758687
	5	1.987613	0.991821	0.992326	1.726321
	10	4.136444	0.991812	0.992326	1.726321
Stacking	3	1.169720	0.991102	0.992349	1.723721
	5	1.532032	0.991673	0.992750	1.677945
	10	2.528369	0.991650	0.992750	1.677945

Meilleure valeur de CV: 3 folds

Test R2: 0.99799

Durée du Tuning (valeur relative, dépend du host d'exécution et du nombre total des combinaisons des hyperparamètres): 3.70s

Avec CV = 3, 5, 10, les métriques **R2**, **RMSE** et **MAE** évoluent selon les 3 graphes suivants :



Pour une valeur de CV donnée :

3. Plus **R2** est proche de 1, c'est meilleur
4. Plus **RMSE/MAE** est petit, c'est mieux pour des niveaux **R2** assez proches les uns des autres

Avec ces 2 critères, les modèles XGBOOST et LightGBM se positionnent en tête de liste.

On note qu'une valeur importante de CV n'assure pas nécessairement l'obtention de meilleurs résultats, comme pour le modèle **RandomForest** avec CV = 10. En clair, CV = 3 suffit pour déterminer les meilleurs modèles et les hyperparamètres associés dont voici la synthèse :

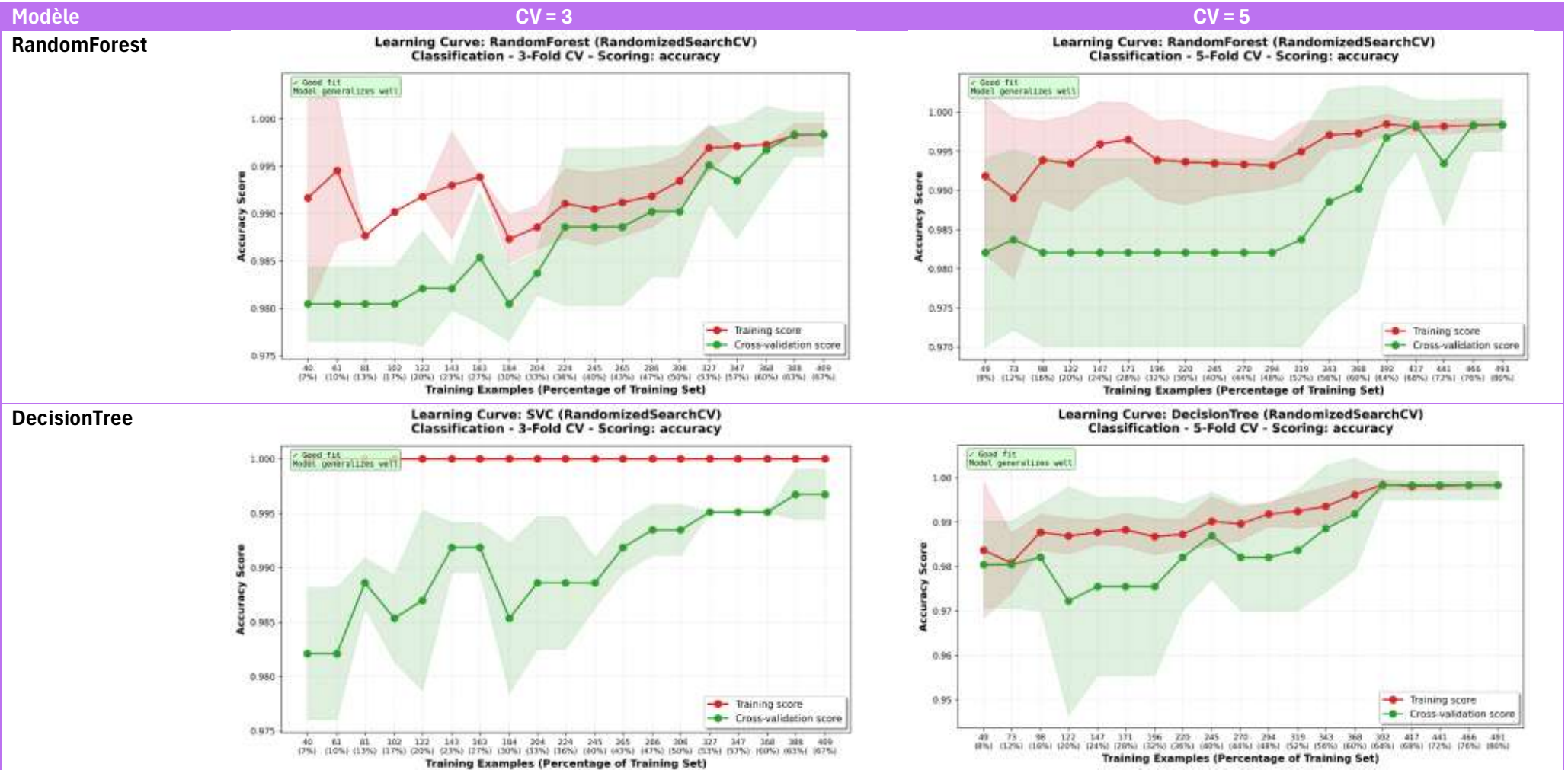
Modèle	CV_Folds	Best_Params
RandomForest	3	{'n_estimators': 50, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_depth': 7}
	5	{'n_estimators': 50, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_depth': 7}
	10	{'n_estimators': 50, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_depth': 7}
DecisionTree	3	{'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': None, 'max_depth': 6, 'criterion': 'squared_error'}
	5	{'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': None, 'max_depth': 6, 'criterion': 'squared_error'}
	10	{'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': None, 'max_depth': 7, 'criterion': 'absolute_error'}
SVM	3	{'kernel': 'rbf', 'gamma': 'scale', 'epsilon': 0.2, 'C': 100}
	5	{'kernel': 'rbf', 'gamma': 'scale', 'epsilon': 0.2, 'C': 100}
	10	{'kernel': 'rbf', 'gamma': 'scale', 'epsilon': 0.2, 'C': 100}
KNN	3	{'weights': 'distance', 'n_neighbors': 3, 'metric': 'manhattan'}
	5	{'weights': 'distance', 'n_neighbors': 3, 'metric': 'manhattan'}
	10	{'weights': 'distance', 'n_neighbors': 3, 'metric': 'manhattan'}
Ridge	3	{'solver': 'cholesky', 'max_iter': 10000, 'alpha': 0.1}
	5	{'solver': 'lsqr', 'max_iter': 10000, 'alpha': 0.1}
	10	{'solver': 'lsqr', 'max_iter': 10000, 'alpha': 1.0}
Lasso	3	{'tol': 0.0001, 'selection': 'cyclic', 'max_iter': 10000, 'alpha': 0.001}
	5	{'tol': 0.0001, 'selection': 'random', 'max_iter': 10000, 'alpha': 0.001}
	10	{'tol': 0.0001, 'selection': 'random', 'max_iter': 10000, 'alpha': 0.01}
ElasticNet	3	{'tol': 0.0001, 'selection': 'cyclic', 'max_iter': 10000, 'l1_ratio': 0.7, 'alpha': 0.001}
	5	{'tol': 0.0001, 'selection': 'cyclic', 'max_iter': 10000, 'l1_ratio': 0.7, 'alpha': 0.001}
	10	{'tol': 0.0001, 'selection': 'cyclic', 'max_iter': 10000, 'l1_ratio': 0.1, 'alpha': 0.001}
GradientBoosting	3	{'n_estimators': 100, 'min_samples_split': 10, 'max_depth': 5, 'learning_rate': 0.2}
	5	{'n_estimators': 250, 'min_samples_split': 2, 'max_depth': 5, 'learning_rate': 0.5}
	10	{'n_estimators': 250, 'min_samples_split': 2, 'max_depth': 5, 'learning_rate': 0.5}
XGBoost	3	{'n_estimators': 250, 'min_child_weight': 5, 'max_depth': 5, 'learning_rate': 0.2}
	5	{'n_estimators': 250, 'min_child_weight': 5, 'max_depth': 5, 'learning_rate': 0.2}
	10	{'n_estimators': 250, 'min_child_weight': 5, 'max_depth': 5, 'learning_rate': 0.2}
LightGBM	3	{'num_leaves': 50, 'n_estimators': 250, 'max_depth': 7, 'learning_rate': 0.2}
	5	{'num_leaves': 50, 'n_estimators': 250, 'max_depth': 7, 'learning_rate': 0.2}
	10	{'num_leaves': 50, 'n_estimators': 250, 'max_depth': 7, 'learning_rate': 0.2}
Bagging	3	{'n_estimators': 50, 'max_samples': 0.8, 'max_features': 1.0}
	5	{'n_estimators': 100, 'max_samples': 0.8, 'max_features': 1.0}
	10	{'n_estimators': 100, 'max_samples': 0.8, 'max_features': 1.0}
Stacking	3	{'cv': 3}
	5	{'cv': 5}
	10	{'cv': 5}

B3. Les courbes d'apprentissage par modèle

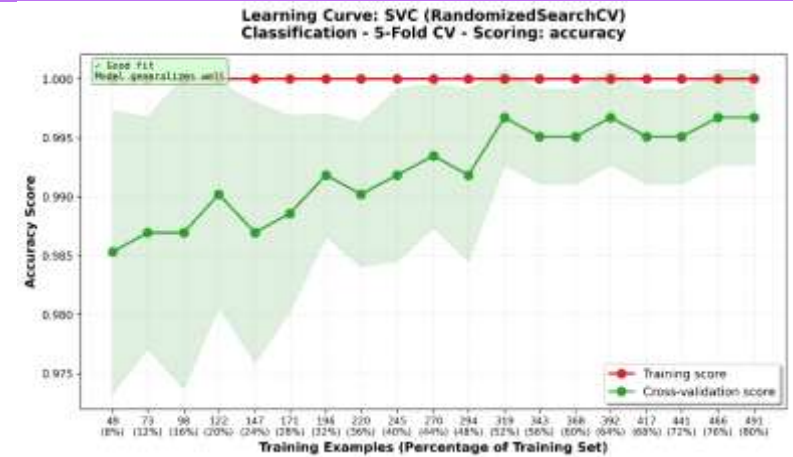
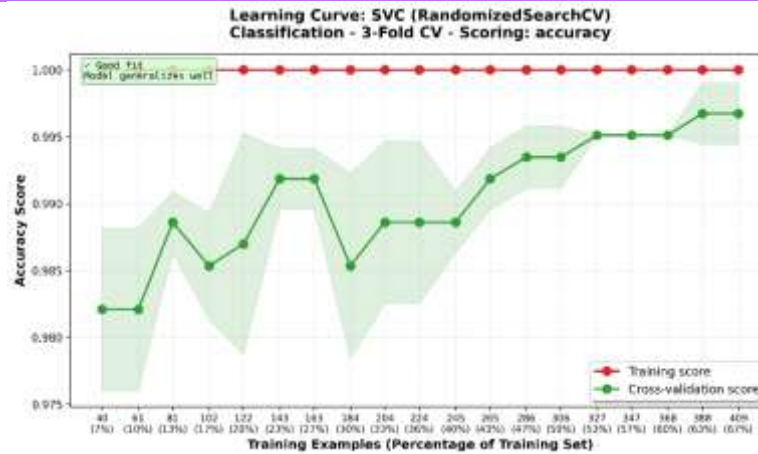
Pour les modèles de classification

Le score utilisé est l'**Accuracy** car le dataset est bien équilibré entre les deux classes comme cela a été constaté plus haut.

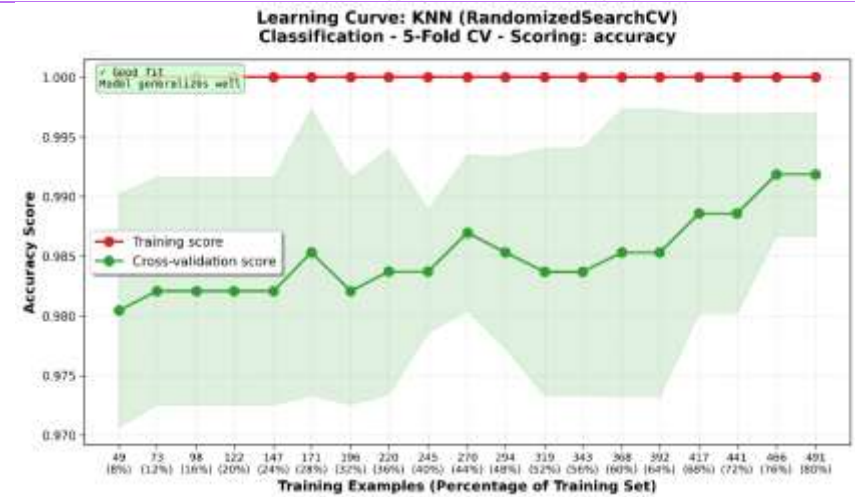
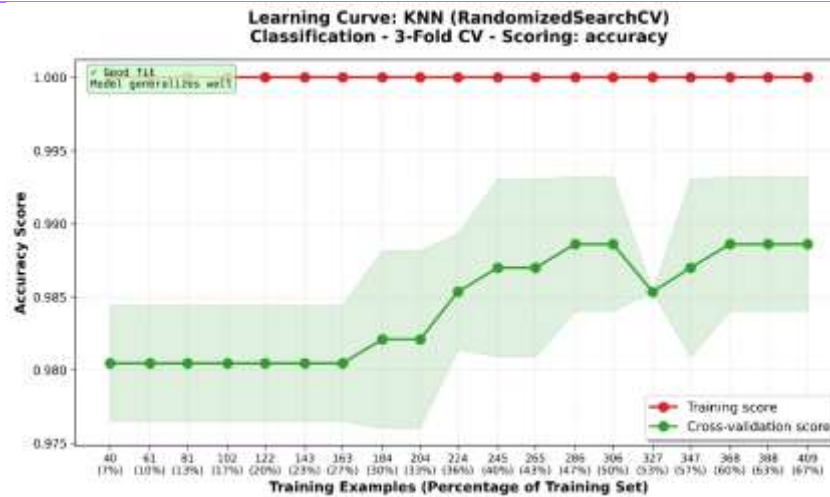
Model	CV	Train_Score	std (Standard Deviation)	CV_Score	std (Standard Deviation)	Gap	Diagnostic
RandomForest	3	0.9984	(±0.0012)	0.9984	(±0.0023)	0.0000	Good fit
	5	0.9984	(±0.0008)	0.9984	(±0.0033)	0.0000	Good fit
DecisionTree	3	0.9984	(±0.0012)	0.9984	(±0.0023)	0.0000	Good fit
	5	0.9984	(±0.0008)	0.9984	(±0.0033)	0.0000	Good fit
SVC	3	1.0000	(±0.0000)	0.9967	(±0.0023)	0.0033	Good fit
	5	1.0000	(±0.0000)	0.9967	(±0.0040)	0.0033	Good fit
KNN	3	1.0000	(±0.0000)	0.9886	(±0.0046)	0.0114	Good fit
	5	1.0000	(±0.0000)	0.9919	(±0.0051)	0.0081	Good fit
LogisticRegression	3	1.0000	(±0.0000)	0.9967	(±0.0023)	0.0033	Good fit
	5	0.9988	(±0.0010)	0.9984	(±0.0033)	0.0004	Good fit
NaiveBayes	3	0.9804	(±0.0020)	0.9805	(±0.0040)	-0.0000	Good fit
	5	0.9804	(±0.0024)	0.9804	(±0.0098)	0.0000	Good fit
Bagging	3	1.0000	(±0.0000)	0.9984	(±0.0023)	0.0016	Good fit
	5	1.0000	(±0.0000)	0.9984	(±0.0033)	0.0016	Good fit
Stacking	3	1.0000	(±0.0000)	0.9951	(±0.0000)	0.0049	Good fit
	5	1.0000	(±0.0000)	0.9967	(±0.0040)	0.0033	Good fit



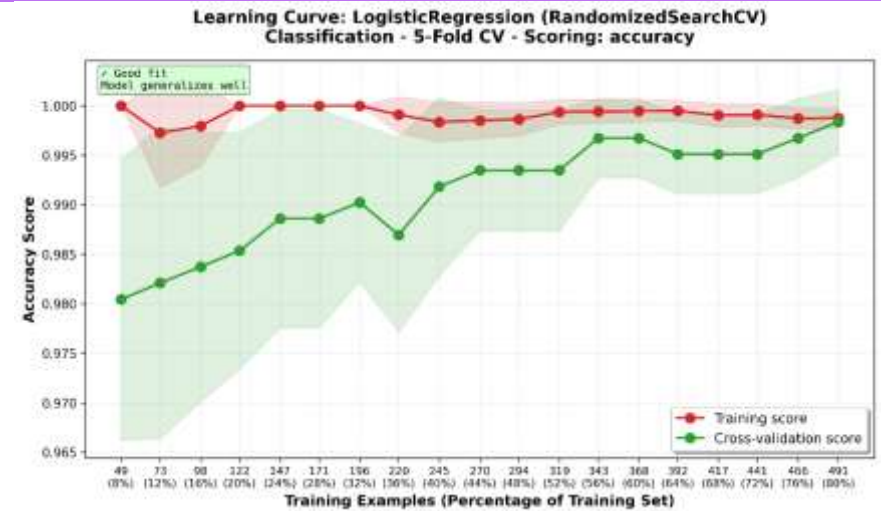
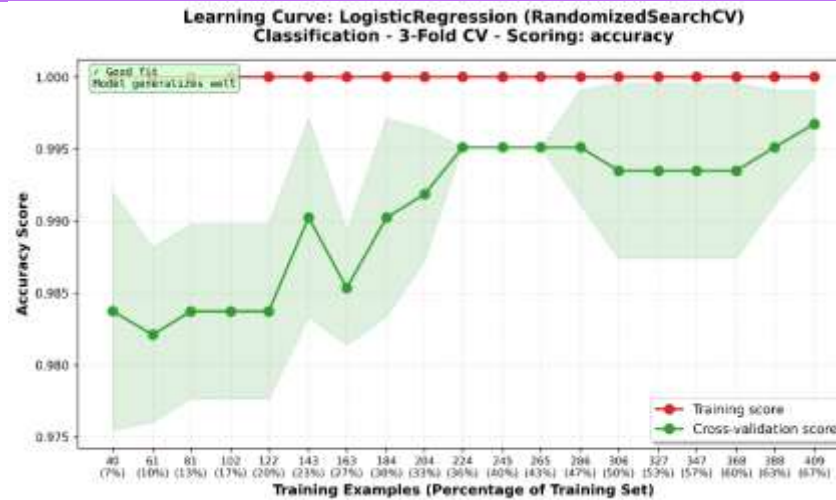
SVC



KNN

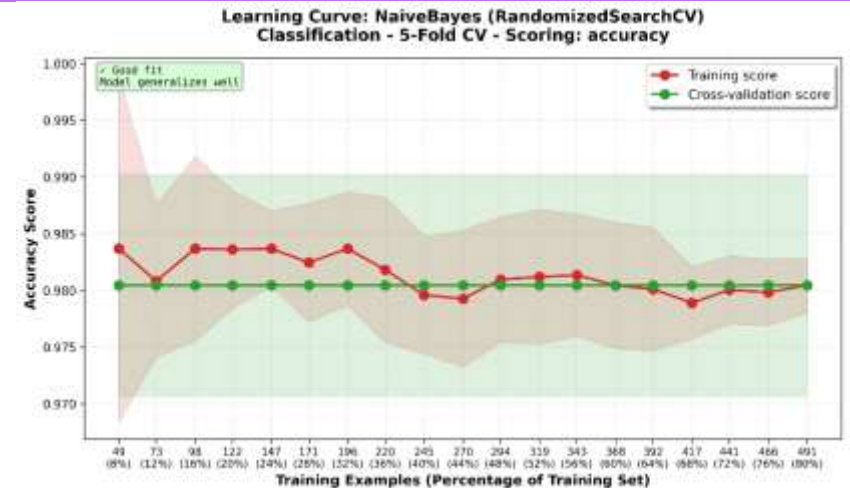
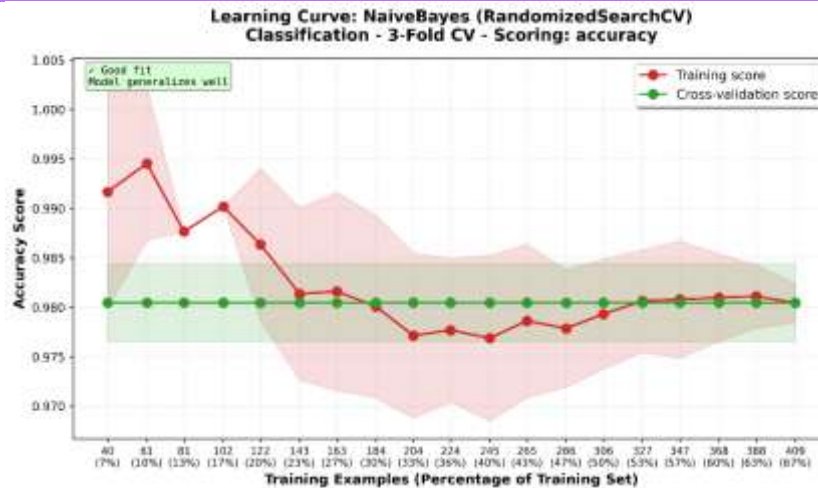


LogisticRegression

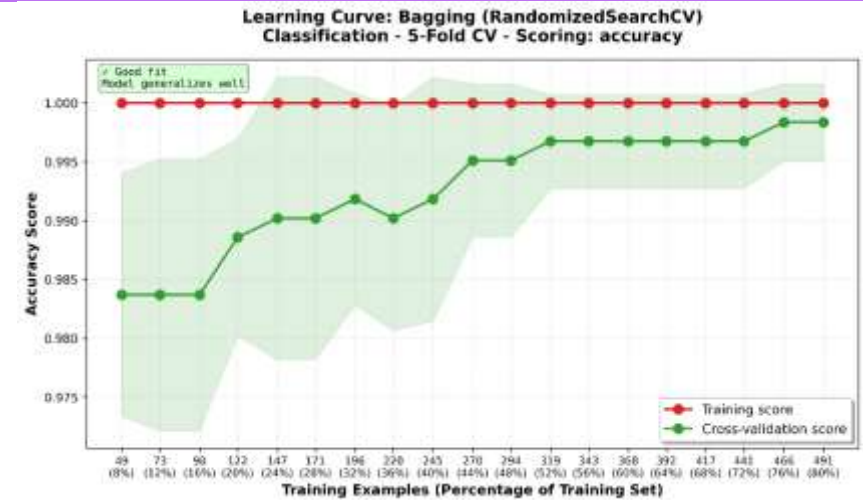
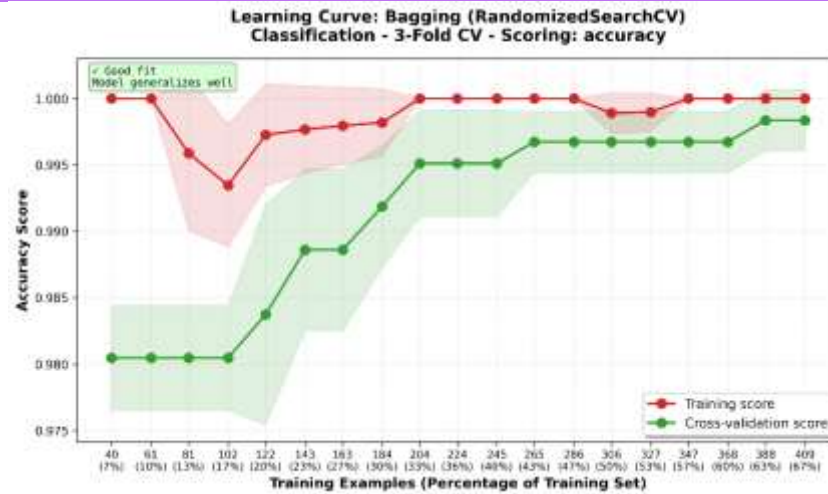


NaiveBayes

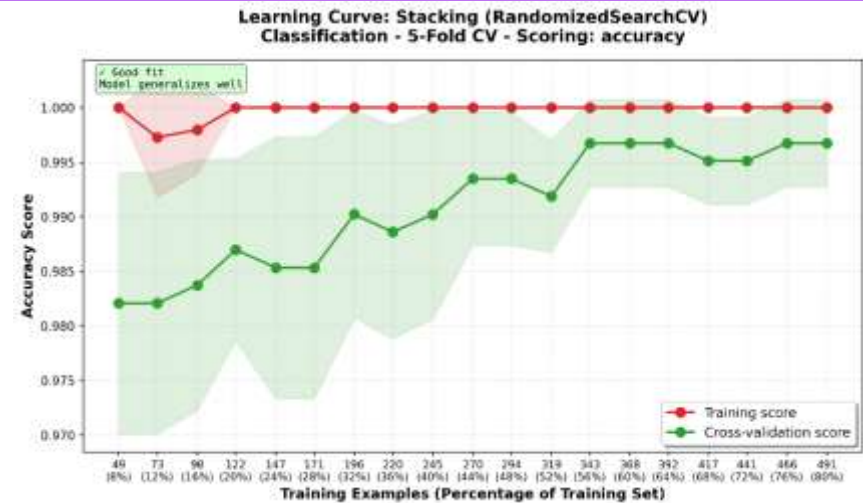
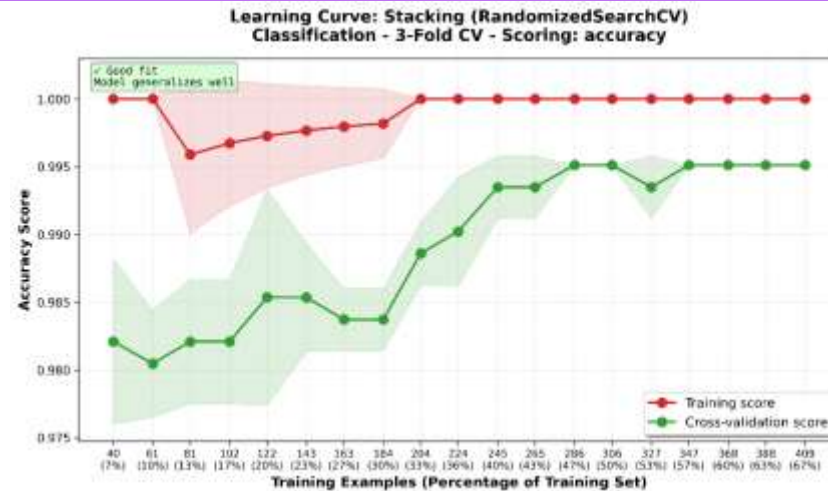
Peu importe la valeur de CV, avec peu de données ce modèle probabiliste converge déjà vers la cible du training Accuracy score (.98), car ces paramètres ne sont pas sensibles à la quantité des données.



Bagging



Stacking

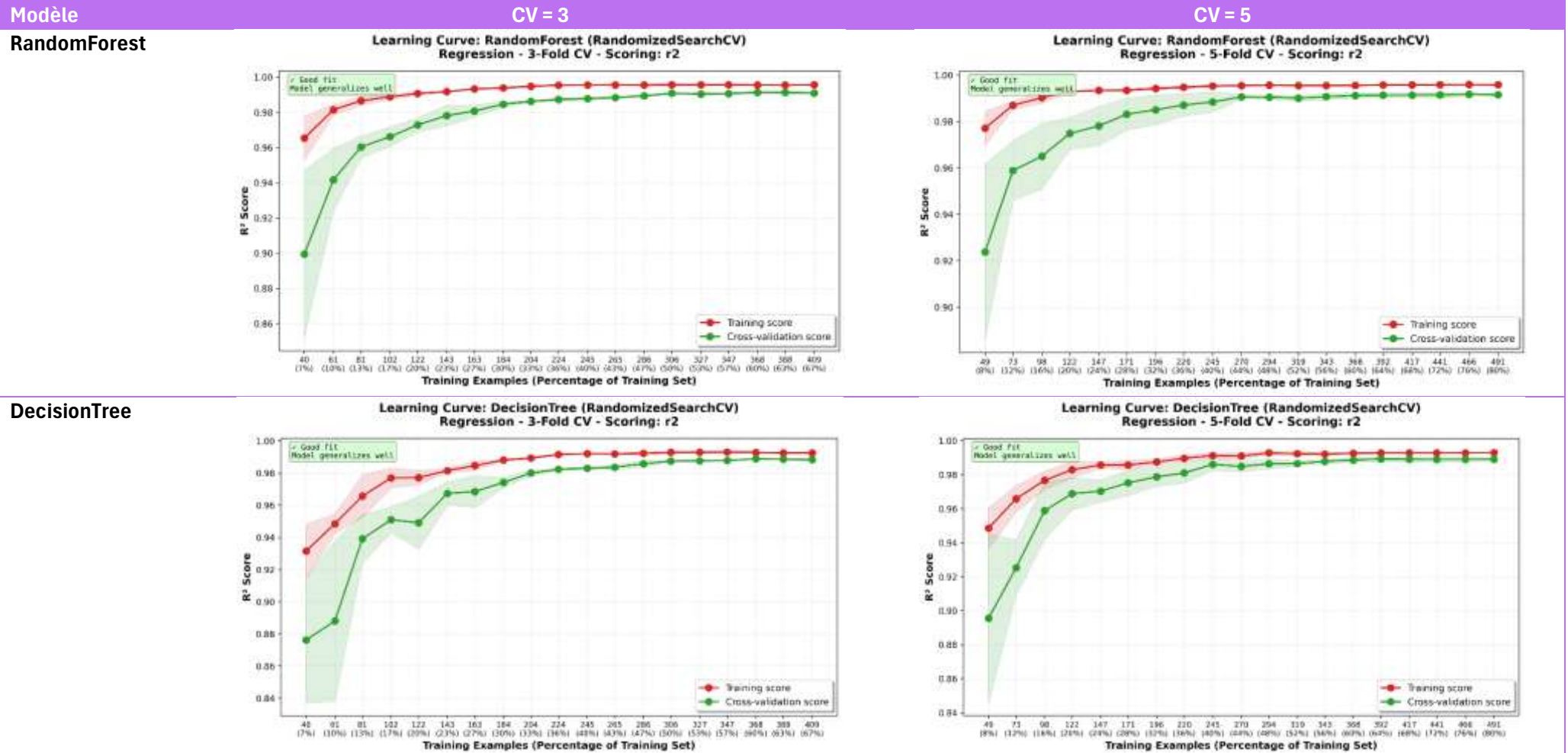


Pour les modèles de Régression

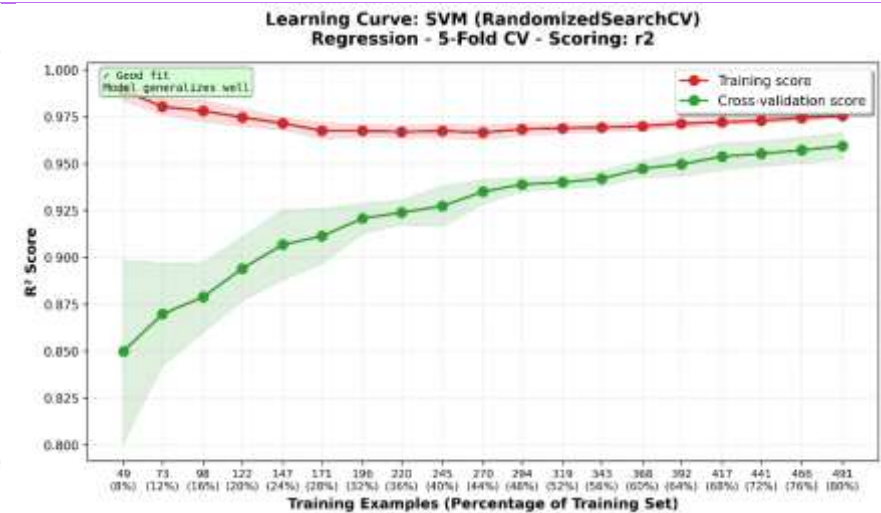
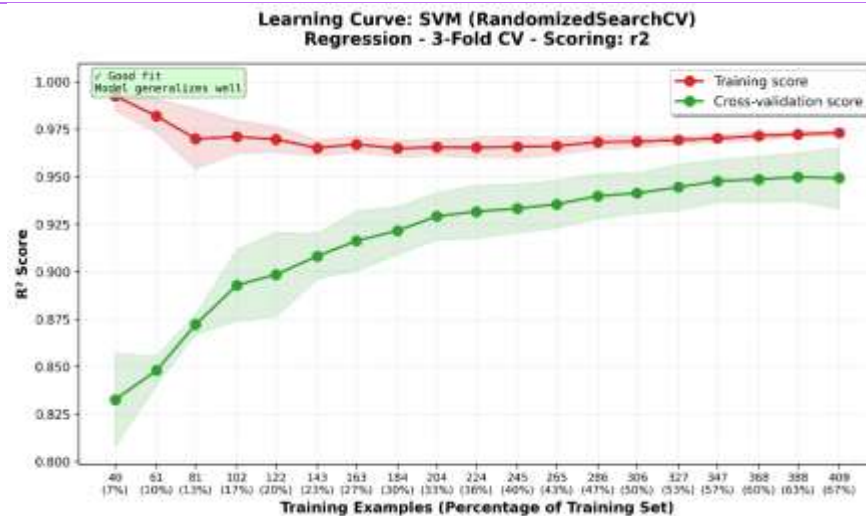
Le score utilisé est R^2 .

Modèle	CV	Train_Score	std (Standard Deviation)	CV_Score	std (Standard Deviation)	Gap	Diagnostic
RandomForest	3	0.9956	(± 0.0004)	0.9910	(± 0.0007)	0.0046	Good fit
	5	0.9959	(± 0.0003)	0.9916	(± 0.0006)	0.0043	Good fit
DecisionTree	3	0.9926	(± 0.0003)	0.9884	(± 0.0007)	0.0042	Good fit
	5	0.9929	(± 0.0003)	0.9891	(± 0.0011)	0.0038	Good fit
SVM	3	0.9731	(± 0.0015)	0.9493	(± 0.0163)	0.0238	Good fit
	5	0.9755	(± 0.0011)	0.9594	(± 0.0071)	0.0161	Good fit
KNN	3	1.0000	(± 0.0000)	0.9754	(± 0.0094)	0.0246	Good fit
	5	1.0000	(± 0.0000)	0.9810	(± 0.0042)	0.0190	Good fit
Ridge	3	0.9112	(± 0.0040)	0.9095	(± 0.0086)	0.0017	Good fit
	5	0.9113	(± 0.0016)	0.9088	(± 0.0069)	0.0026	Good fit
Lasso	3	0.9112	(± 0.0040)	0.9095	(± 0.0086)	0.0017	Good fit
	5	0.9113	(± 0.0016)	0.9088	(± 0.0070)	0.0026	Good fit
ElasticNet	3	0.9112	(± 0.0040)	0.9095	(± 0.0086)	0.0017	Good fit
	5	0.9113	(± 0.0016)	0.9088	(± 0.0069)	0.0026	Good fit
GradientBoosting	3	0.9997	(± 0.0000)	0.9964	(± 0.0005)	0.0033	Good fit
	5	1.0000	(± 0.0000)	0.9975	(± 0.0006)	0.0025	Good fit
XGBoost	3	0.9998	(± 0.0000)	0.9966	(± 0.0002)	0.0032	Good fit
	5	0.9997	(± 0.0000)	0.9975	(± 0.0003)	0.0023	Good fit
LightGBM	3	0.9990	(± 0.0001)	0.9959	(± 0.0002)	0.0031	Good fit
	5	0.9990	(± 0.0001)	0.9968	(± 0.0005)	0.0023	Good fit
Bagging	3	0.9981	(± 0.0001)	0.9914	(± 0.0008)	0.0067	Good fit
	5	0.9983	(± 0.0001)	0.9917	(± 0.0008)	0.0066	Good fit
Stacking	3	0.9985	(± 0.0005)	0.9913	(± 0.0006)	0.0072	Good fit
	5	0.9986	(± 0.0005)	0.9916	(± 0.0005)	0.0070	Good fit

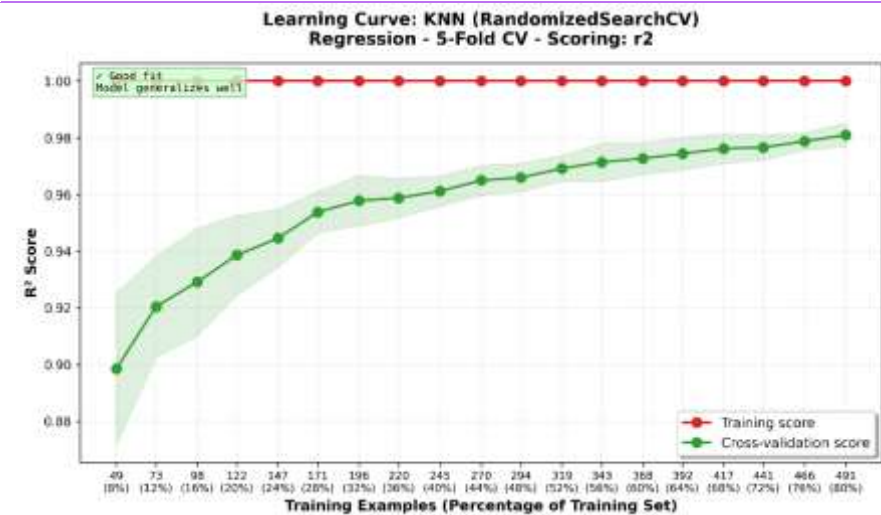
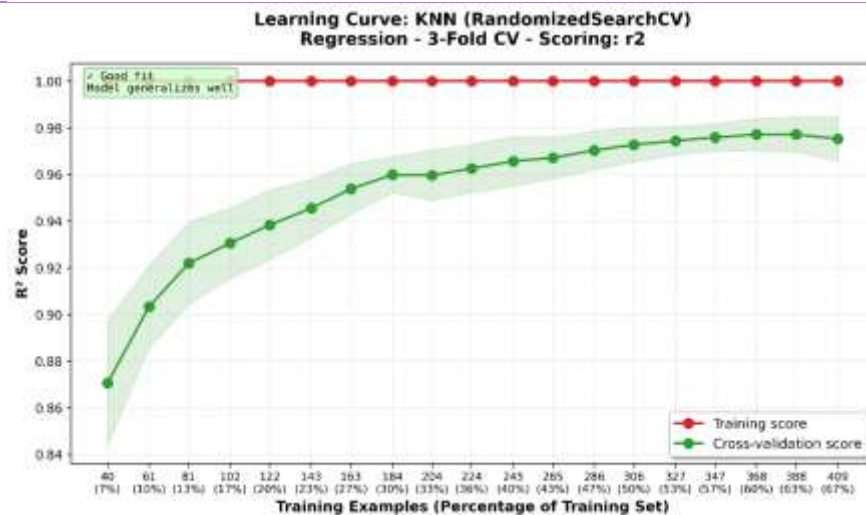
Au vu des valeurs faibles de la déviation standard (std) ne dépassant jamais ± 0.05 , cela prouve que tous les modèles sont stables pour l'ensemble des FOLDS. Cela se manifeste clairement dans les courbes d'apprentissage aussi bien pour les ensembles de training que du test aussi bien pour CV = 3 que CV = 5.



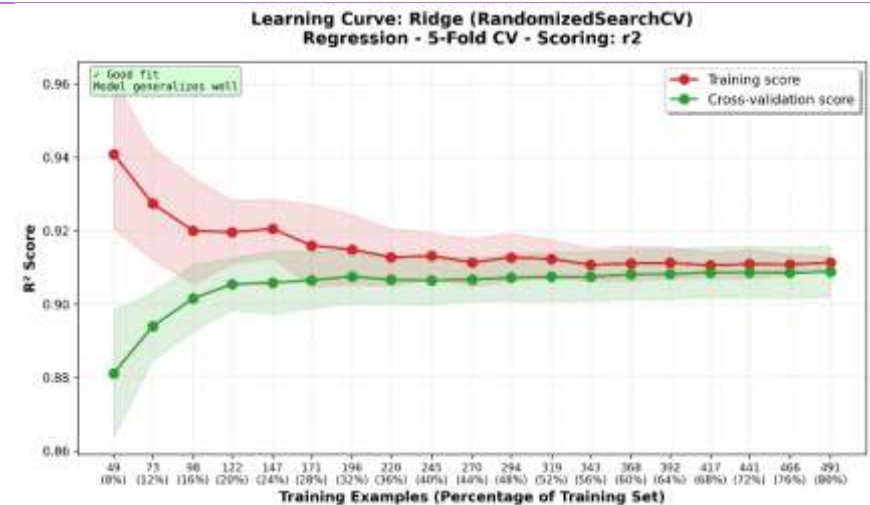
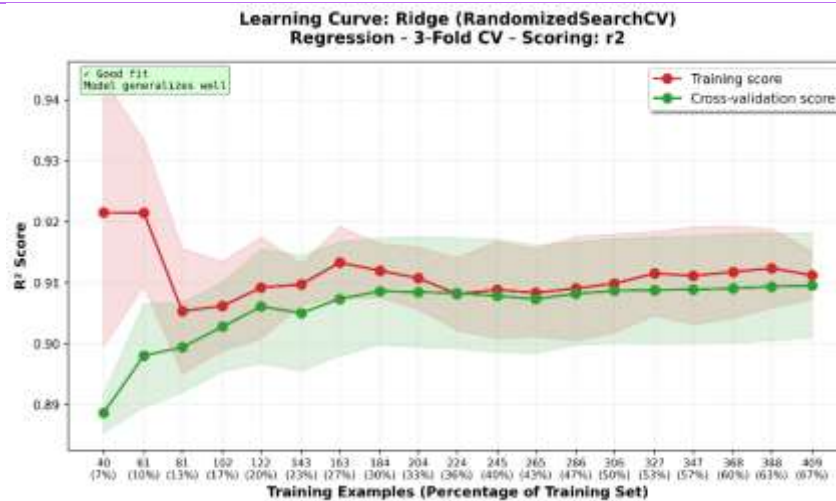
SVM



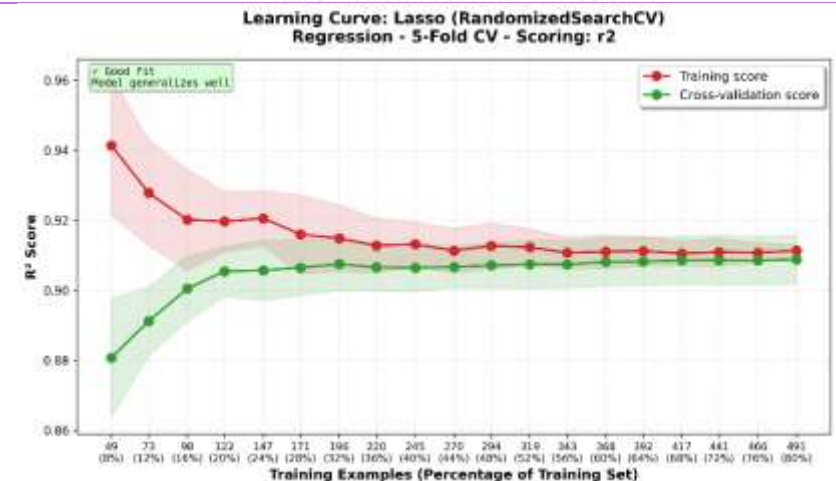
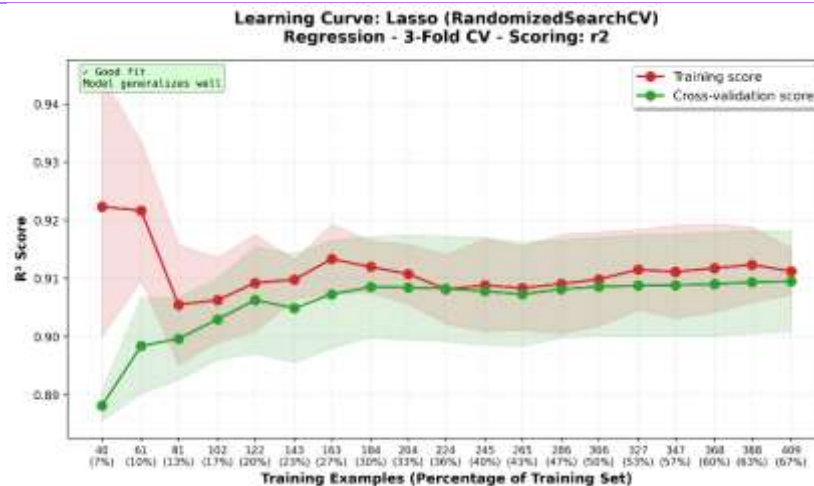
KNN



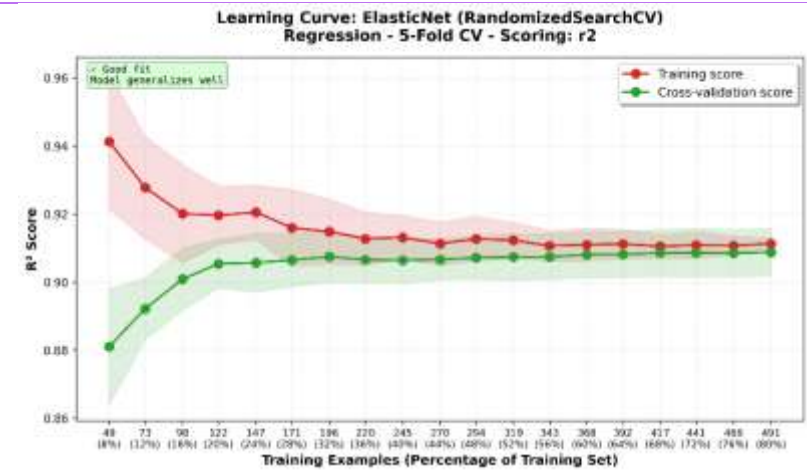
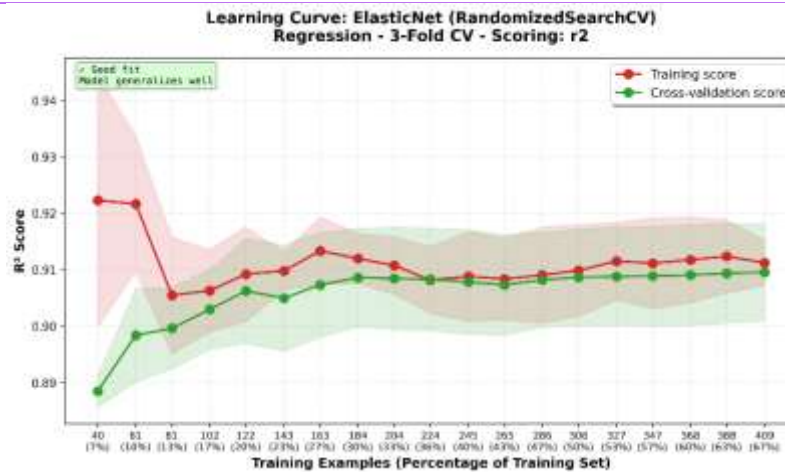
Ridge



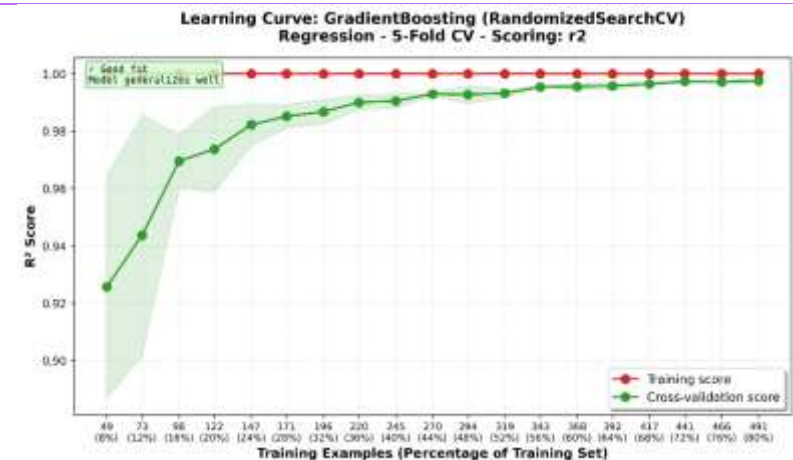
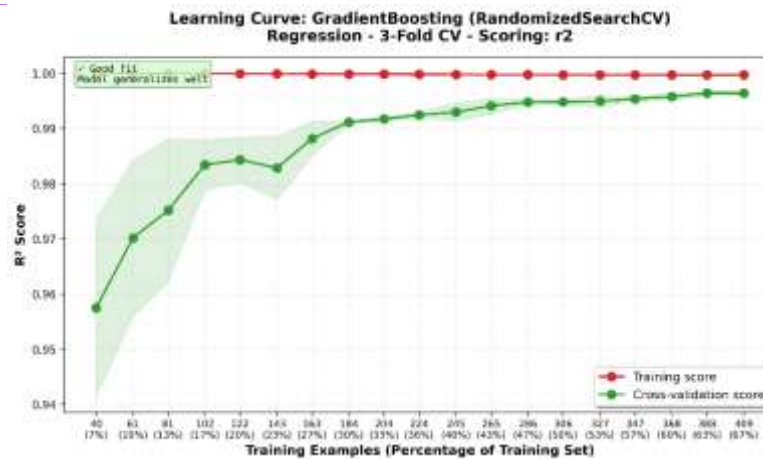
Lasso



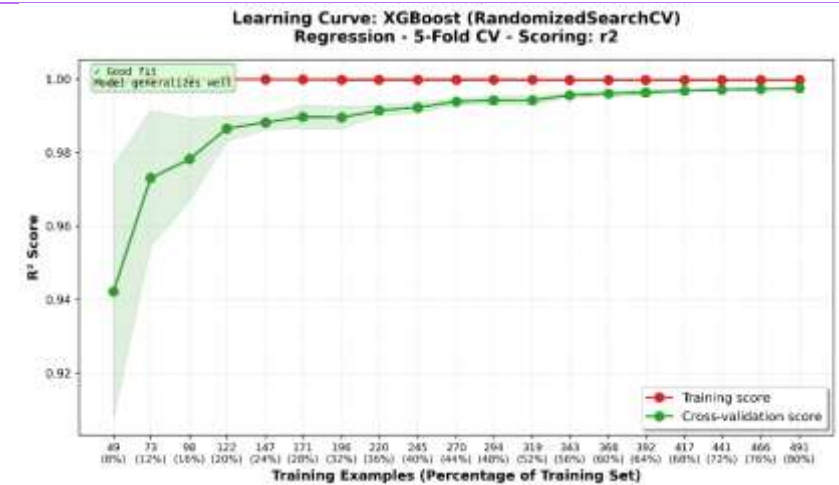
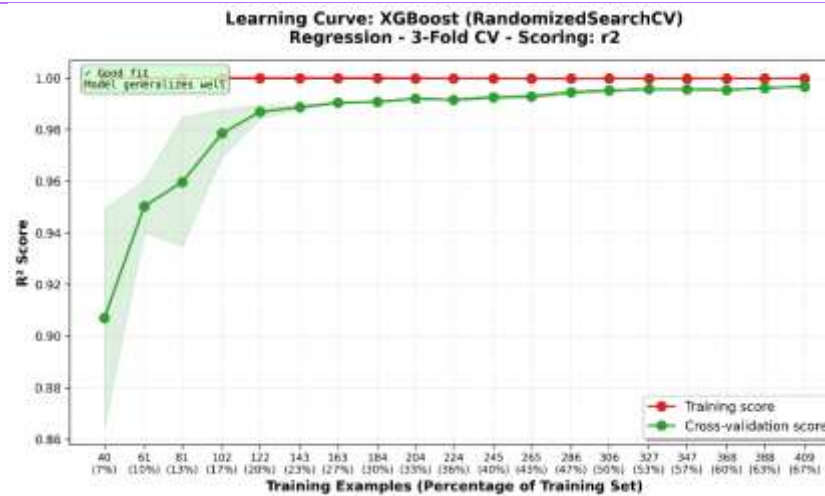
ElasticNet



GradientBoosting

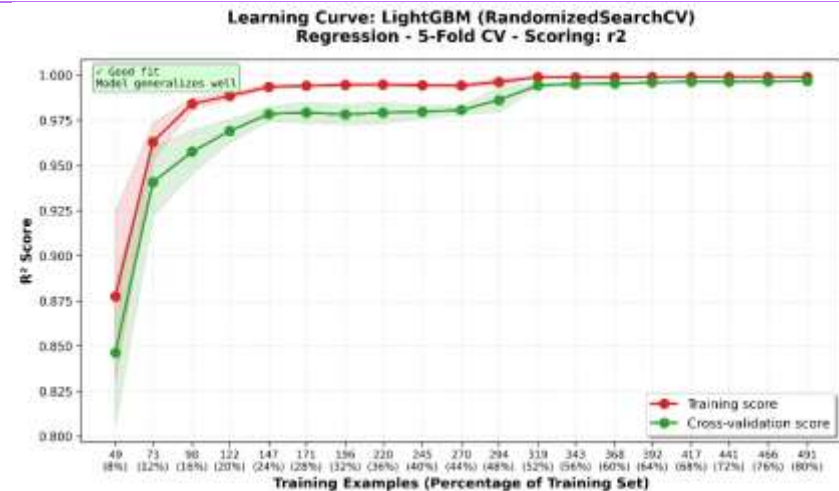
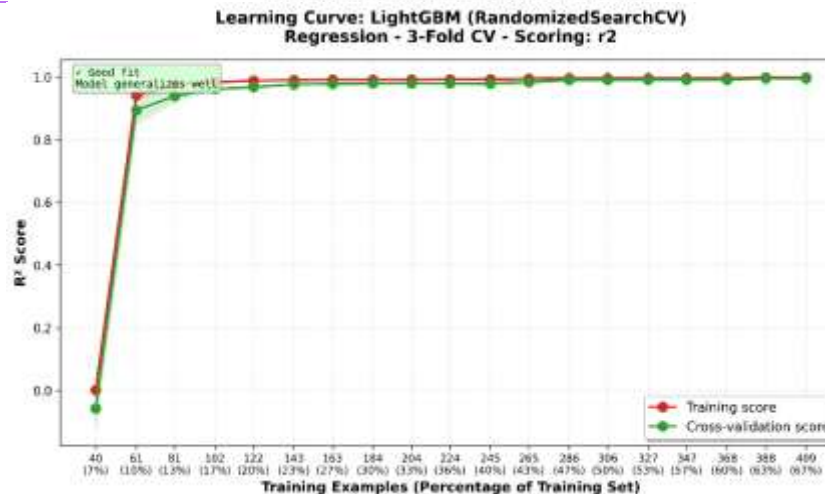


XGBoost

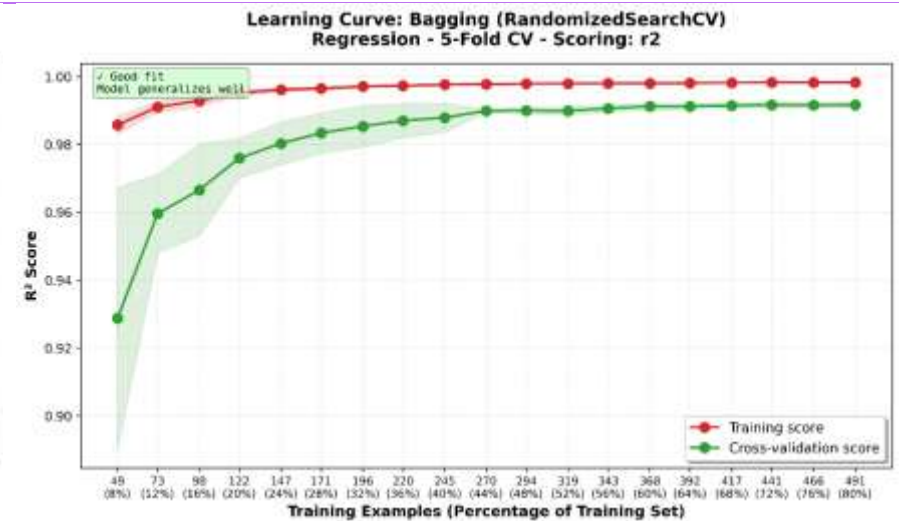
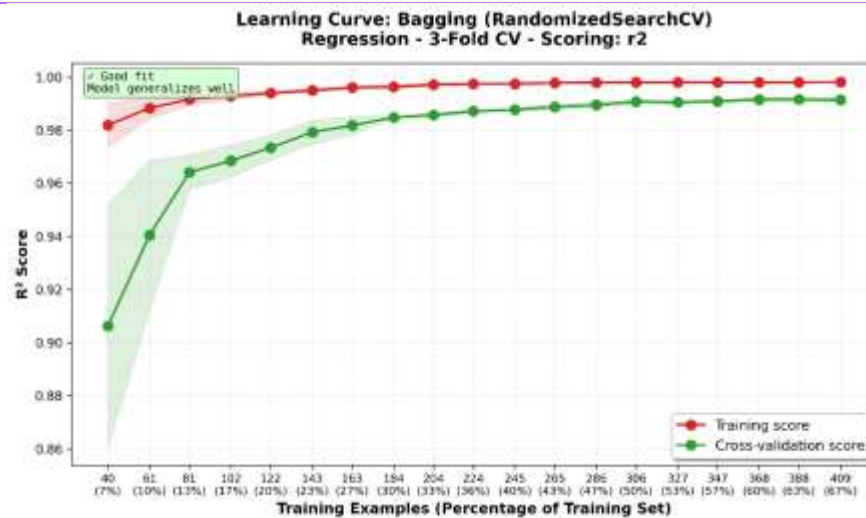


LightGBM

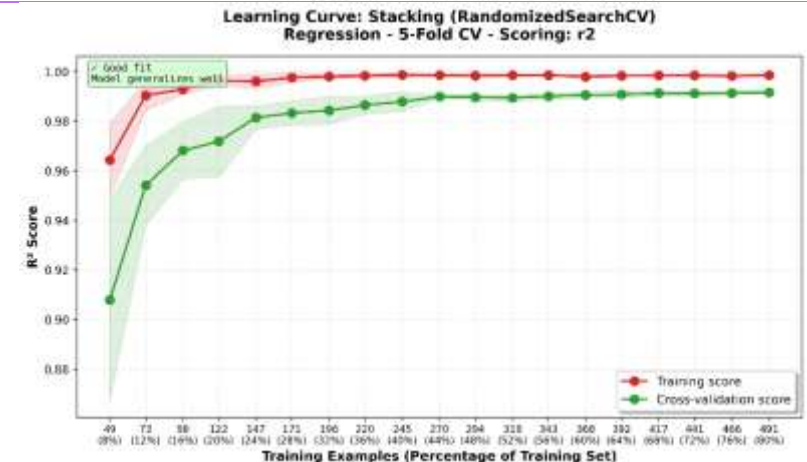
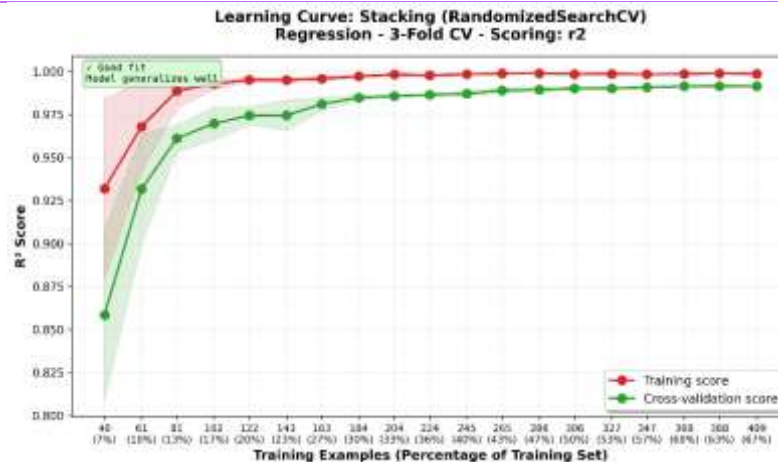
La convergence se fait très rapidement même avec un CV = 3 avec un std presque nul !



Bagging



Stacking



B4. Comparaison des durées de compilation

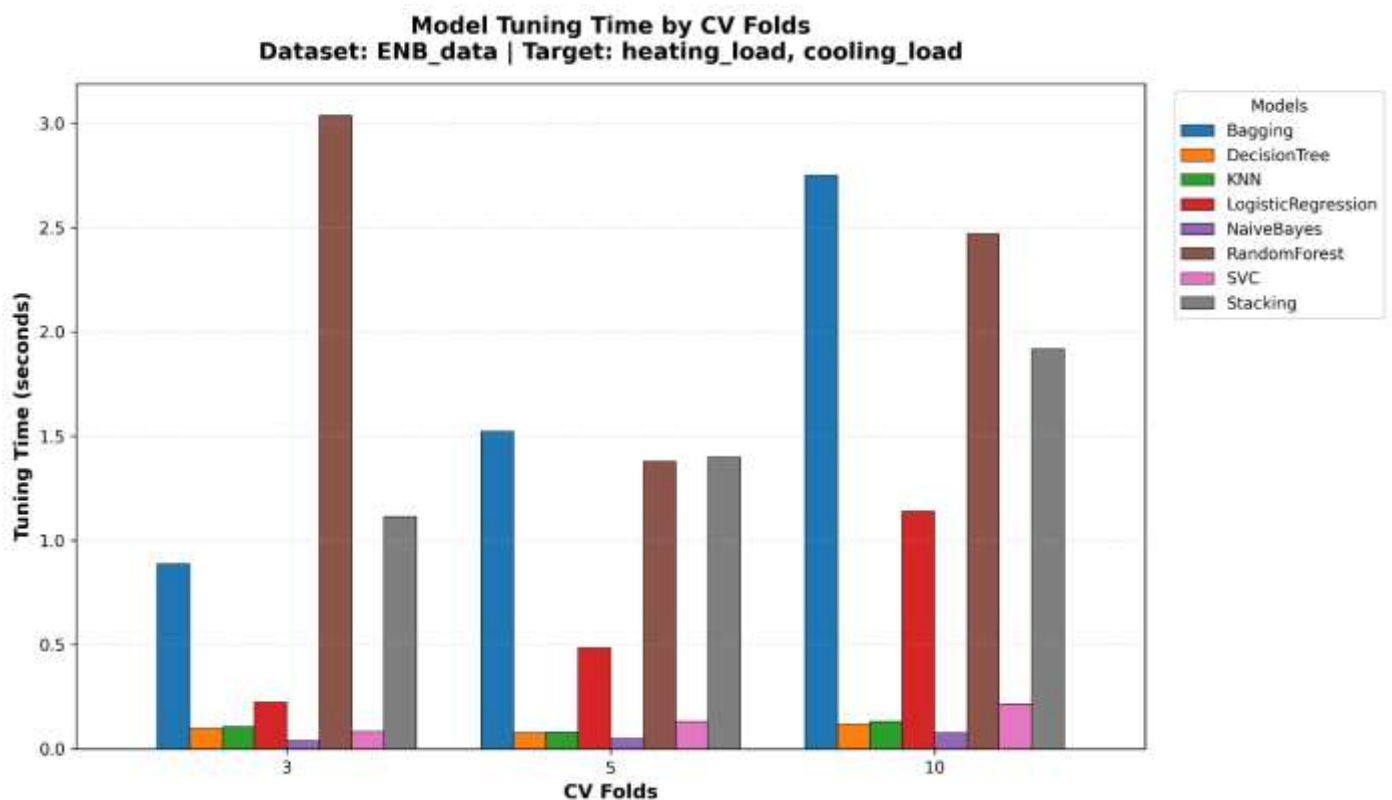
Pour choisir le modèle qui répond le mieux aux exigences de performances, il faut combiner plusieurs facteurs pour trouver le bon compromis (performance vs durée d'exécution) :

4. **Critère de performance** : La métrique de performance prise en compte dans la comparaison lors de la recherche par CV (grid ou random) :
 - **Pour la classification** : **Accuracy** ou **F1_Score** comme dans notre cas
 - **Pour la régression** : **R²**
5. **Durée de compilation (donnée relative)** : Ce critère est relatif, car dépend de la machine (le host) sur laquelle on exécute la recherche CV. Si les écarts des performances entre CV = 3 et CV = 5 ne sont pas importants, il vaut mieux opter pour le plus petit d'au moins en phase d'expérimentation et ajustement des hyperparamètres du modèle, car un modèle n'est jamais figé ; il doit évoluer avec les nouvelles données issues du terrain et les nouvelles règles exigées. Un autre paramètre non négligeable à prendre en compte, est la dimension de l'espace du dataset :

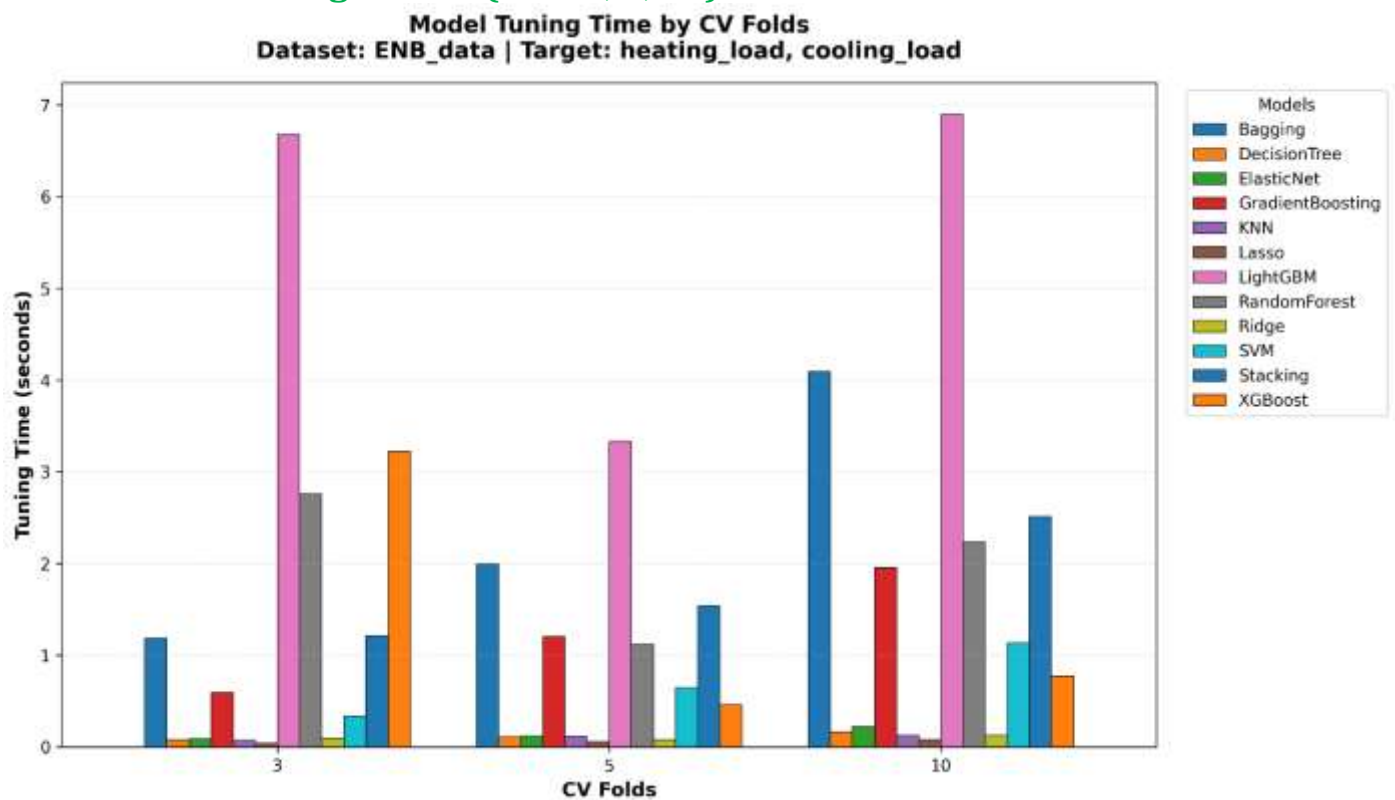
nombre de données × nombre des variables descriptives (features).

Cas : modèles de classification (CV = 3, 5, 10)

Ce graphe illustre les durées de compilation (en secondes) des phases d'entraînement et de test de chaque modèle :



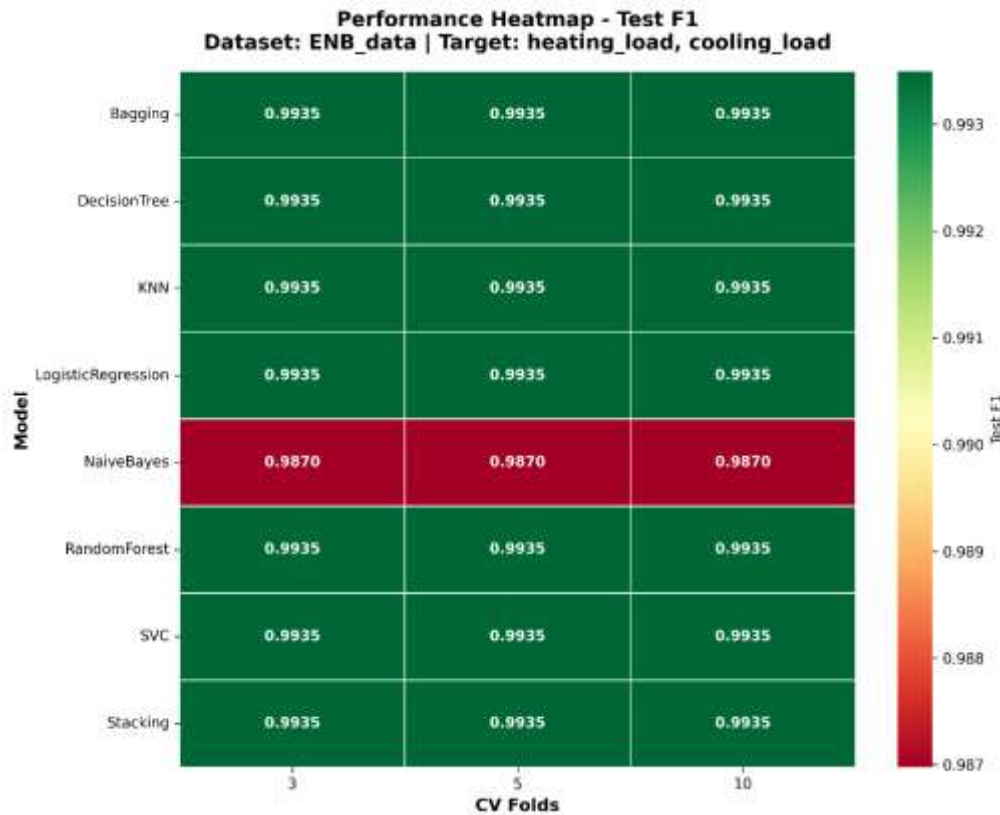
Cas : modèles de régression (CV = 3, 5, 10)



On remarque que globalement, les durées pour la compilation des métriques de performance des modèles de régression sont plus longues que celles des modèles de classification à cause des calculs plus intenses !

B5. Les heatmaps des scores

Cas : modèles de classification (CV = 3, 5, 10)



Tous les modèles possèdent le même **F1_Score** à l'exception de NaiveBayes dès CV = 3.

Cas : modèles de régression (CV = 3, 5, 10)



C. 2^{ème} cas : dataset bike_sharing_hour.csv

- **Dataset CSV** : bike_sharing_hour.csv
- **URL 1**: <https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset> (consultez ce lien pour plus de détails sur le dataset)
- **URL Espace GITHUB de ce guide** : https://github.com/ayeza/ML_CV/blob/main/data/bike_sharing_hour.csv
- **Dimension du dataset** 🗡 : 17 379 × 15 ➔ 260 685 valeurs

C1. Les variables descriptives et les cibles

Variable	Type	Description	Descriptive / cible (Target)
dteday	Date	date	Descriptive
season	Categorical	1:winter, 2:spring, 3:summer, 4:fall	Descriptive
yr	Categorical	year (0: 2011, 1: 2012)	Descriptive
mnth	Categorical	month (1 to 12)	Descriptive
hr	Categorical	hour (0 to 23)	Descriptive
holiday	Binary	weather day is holiday or not (extracted from http://dchr.dc.gov/page/holiday-schedule)	Descriptive
weekday	Categorical	day of the week	Descriptive
workingday	Binary	if day is neither weekend nor holiday is 1, otherwise is 0	Descriptive
weathersit	Continuous	Normalized temperature in Celsius. The values are derived via $(t-t_{min})/(t_{max}-t_{min})$, $t_{min}=-8$, $t_{max}=+39$ (only in hourly scale)	Descriptive
temp	Continuous	Normalized feeling temperature in Celsius. The values are derived via $(t-t_{min})/(t_{max}-t_{min})$, $t_{min}=-16$, $t_{max}=+50$ (only in hourly scale)	Descriptive
atemp	Continuous	Normalized humidity. The values are divided to 100 (max)	Descriptive
hum	Continuous	Normalized wind speed. The values are divided to 67 (max)	Descriptive
windspeed	Integer	count of casual users	Descriptive
casual	Integer	count of registered users	Target
registered	Integer	count of total rental bikes including both casual and registered	Target

Dans le dataset initial, une variable supplémentaire **cn** représentant la cible, est présente ; elle est égale à la somme **casual + registered**. Elle n'a pas été intégrée dans notre dataset, car elle sera prise en tant qu'agrégation en utilisant l'addition (`AGGREGATION_NAME = 'sum'`) comme nous l'avons vu dans le 1^{er} dataset.

C2. Options de configuration

Paramètres de configuration (dans config.py):

```
DATA_PATH = DATA_DIR / 'bike_sharing_hour.csv'
CSV_DELIMITER = 'auto'

REMOVE_OUTLIERS = True
USE_POLYNOMIAL_FEATURES = False
POLYNOMIAL_DEGREE = 2

AGGREGATION_COLS = ['casual', 'registered']
EXCLUDE_COLS = ['instant', 'dteday', 'casual', 'registered']

AGGREGATION_NAME = 'sum'

# Model settings
RANDOM_STATE = 42
TEST_SIZE = 0.2
CV_FOLDS = 10
N_JOBS = -1

SEARCH_TYPE = 'random'

N_CLASSES = 'auto'

CATEGORICAL_ENCODING_THRESHOLD = 100
CATEGORICAL_ENCODING_METHOD = 'label'

CLUSTERING_METHOD = 'kmeans'
LEARNING_CURVE_TRAIN_SIZES = [0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, .45, 0.5, .55, 0.6, .65, 0.7, .75, 0.8, .85, 0.9, .95, 1.0]

CLF_LEARNING_CURVES_SCORE = 'both'
REG_LEARNING_CURVES_SCORE = 'both'

GENERATE_PREDS = False
```

La recherche CV se fait en mode (`SEARCH_TYPE = 'random'`) ce qui devrait accélérer le temps de compilation généralement, car ce dernier dépend aussi du total des combinaisons possibles des hyperparamètres liées à chaque modèle. L'autre alternative consiste à variabiliser le nombre d'itérations à réaliser pour chaque modèle, modifiables dans **hyperparameters.py**. Pour plus de détails, référez-vous à la section : [A2.2 Recherche aléatoire \(Randomized Search\)](#)...

Pour des expérimentations en fonction de la dimension du dataset et des hyperparamètres par défaut, on peut opter par exemple pour :

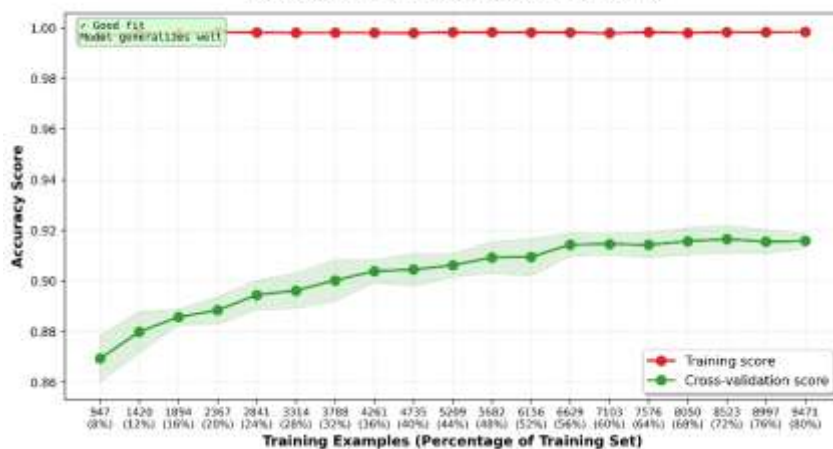
Classification	n_iter	Regression	n_iter
RandomForest	30	RandomForest	30
DecisionTree	25	DecisionTree	25
SVC	20	SVM	20
KNN	20	KNN	20
LogisticRegression	25	LinearRegression	2
NaiveBayes	8	Ridge	15
Bagging	15	Lasso	10
Stacking	2	ElasticNet	15
		GradientBoosting	30
		XGBoost	25
		LightGBM	25
		Bagging	15
		Stacking	2

C3. Principaux résultats

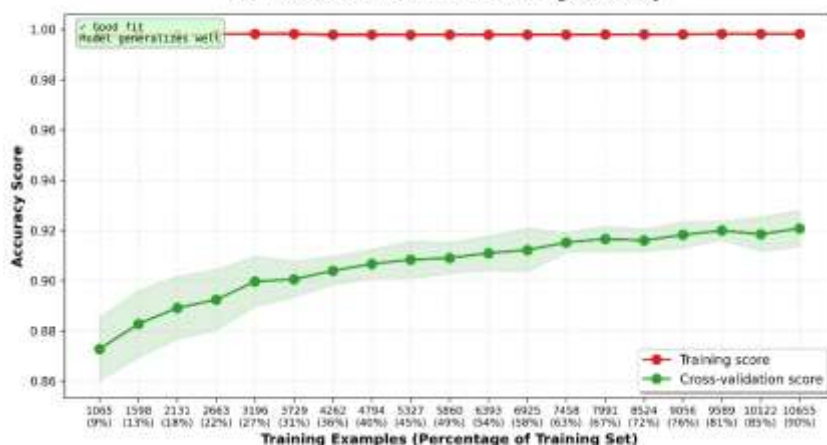
Résultats de la classification

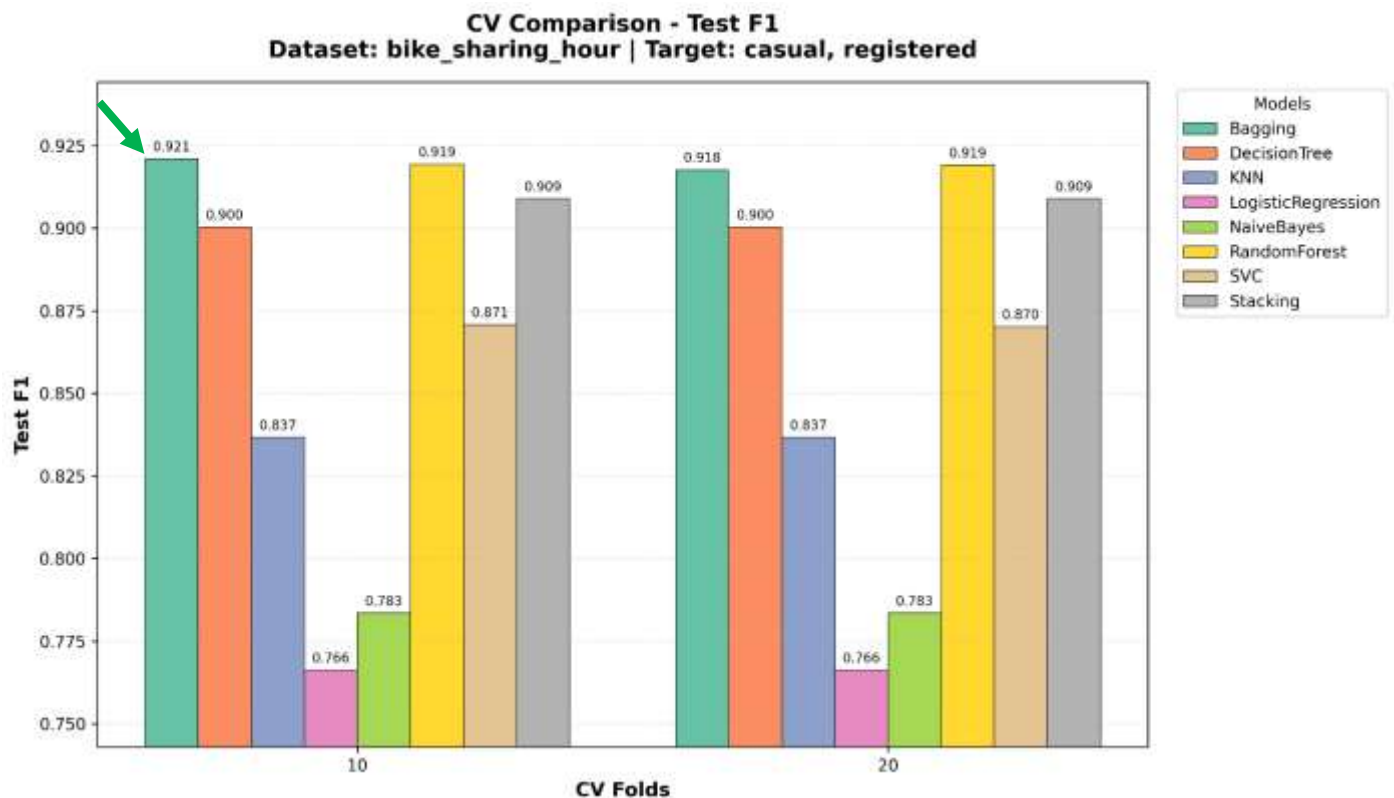
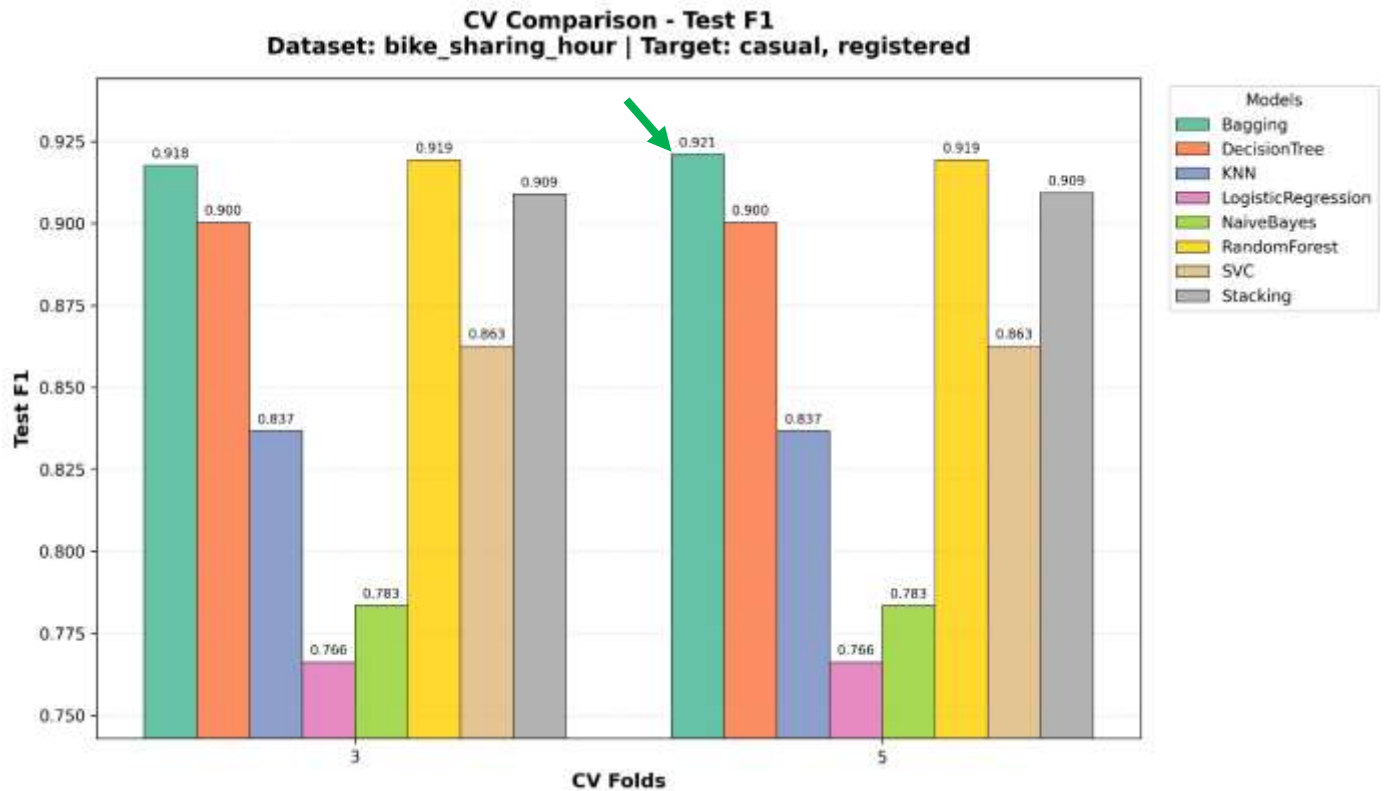
Model	CV_Folds	Tuning_Time_sec	Best_CV_Score	Test_Accuracy	Test_F1	GAP Acc vs F1
RandomForest	3	9.982699	0.905522	0.926689	0.919312	0.74%
	5	12.023531	0.909534	0.926689	0.919312	0.74%
DecisionTree	3	0.383474	0.880059	0.909797	0.900313	0.95%
	5	0.436999	0.885241	0.909797	0.900313	0.95%
SVC	3	219.965265	0.865157	0.876014	0.862531	1.35%
	5	443.862072	0.868473	0.876014	0.862531	1.35%
KNN	3	1.841691	0.831945	0.854054	0.836657	1.74%
	5	2.194611	0.839544	0.854054	0.836657	1.74%
LogisticRegression	3	0.558906	0.765646	0.780068	0.766196	1.39%
	5	0.86204	0.766693	0.780068	0.766196	1.39%
NaiveBayes	3	0.204597	0.785655	0.805405	0.783476	2.19%
	5	0.302205	0.786299	0.805405	0.783476	2.19%
Bagging	3	4.973691	0.903361	0.925	0.917656	0.73%
	5	9.816764	0.911265	0.928041	0.921046	0.70%
Stacking	3	67.956861	0.896289	0.91723	0.908901	0.83%
	5	75.574098	0.900205	0.917568	0.909293	0.83%

Learning Curve: Bagging (RandomizedSearchCV)
Classification - 5-Fold CV - Scoring: accuracy



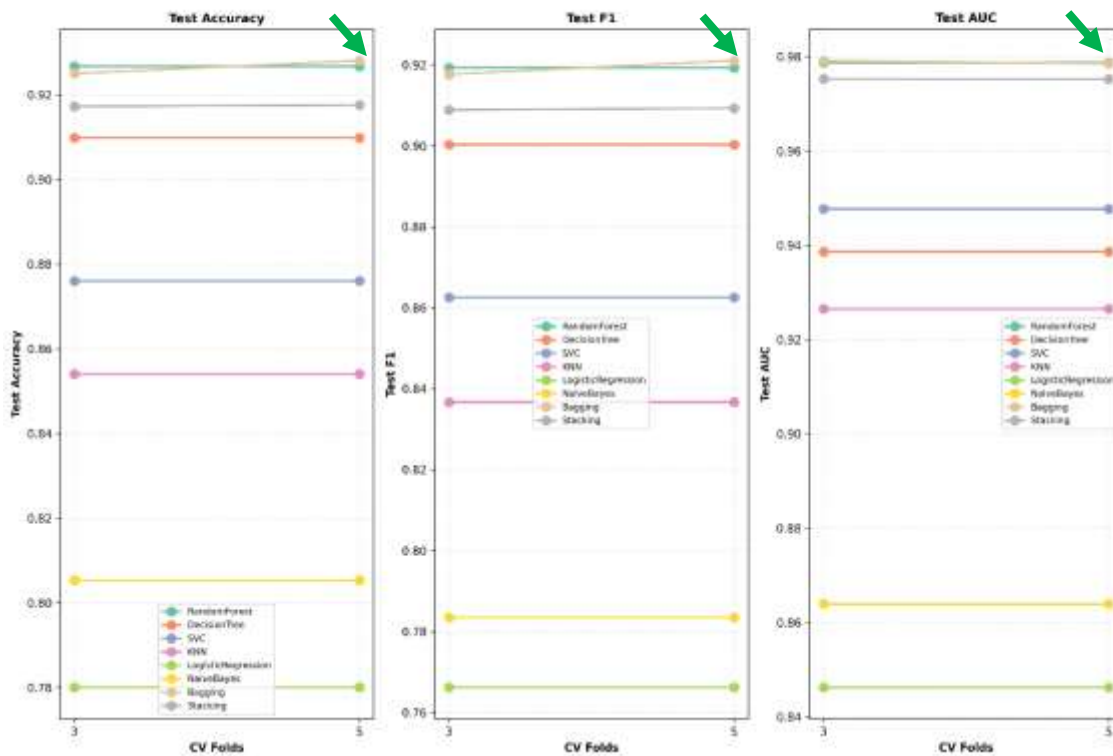
Learning Curve: Bagging (RandomizedSearchCV)
Classification - 10-Fold CV - Scoring: accuracy



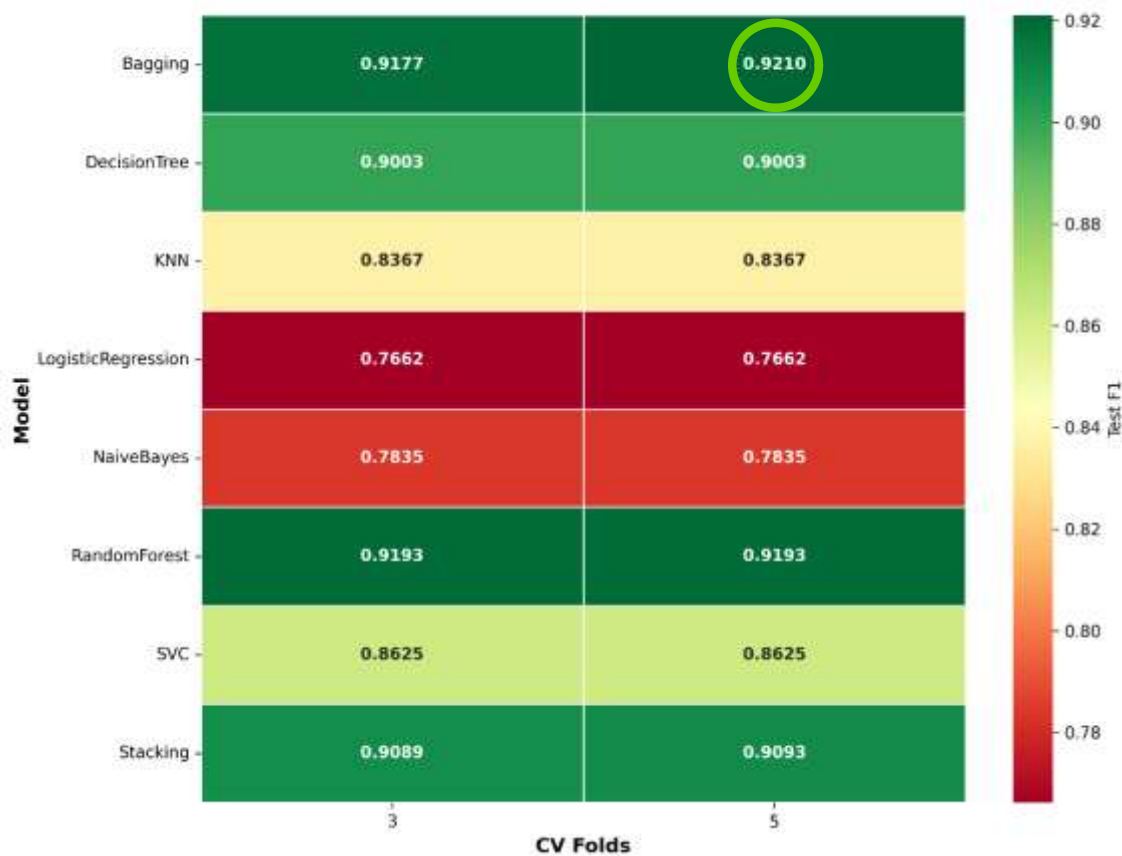


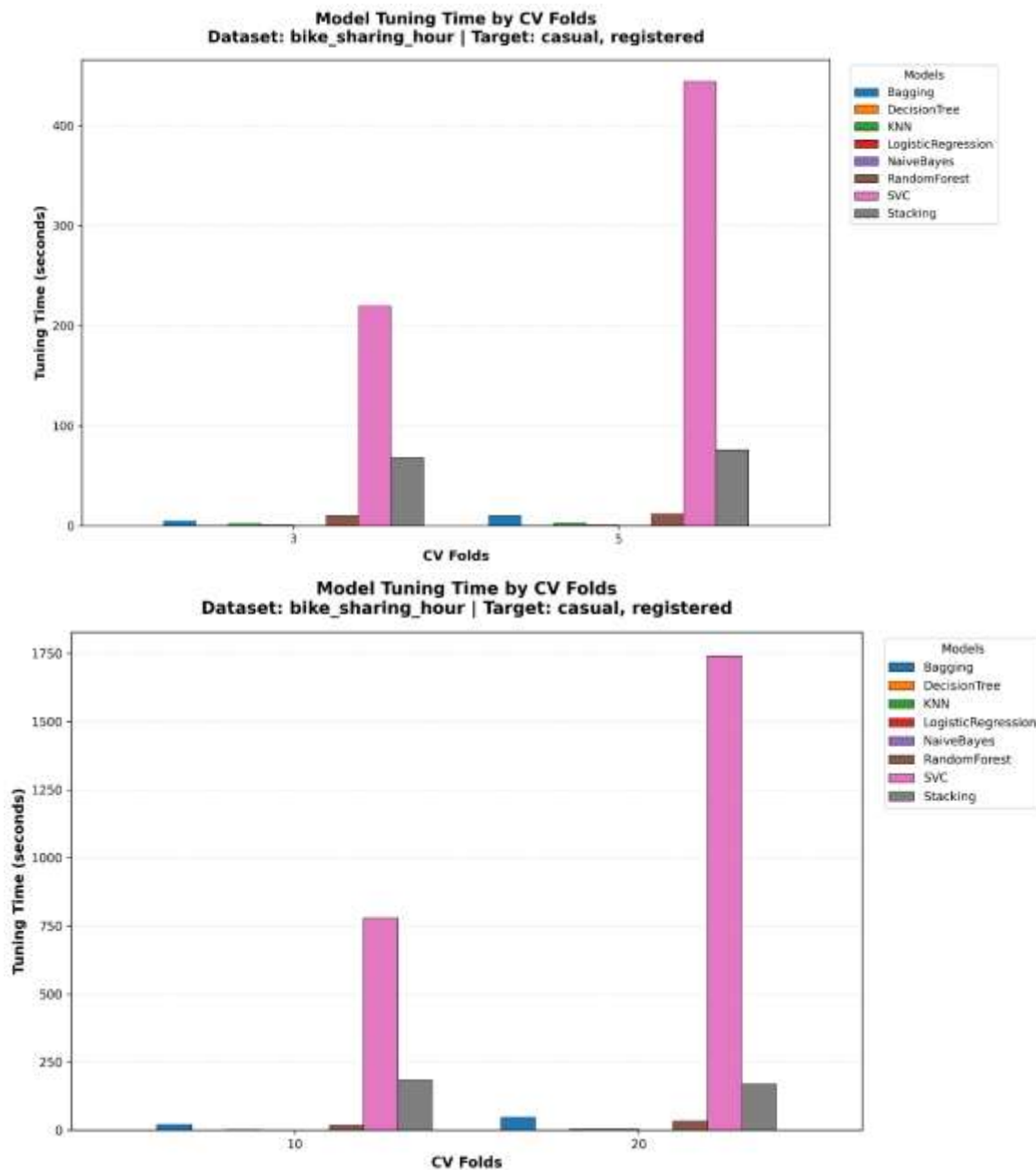
On constate que même en augmentant la valeur de CV, les meilleurs modèles restent capés relativement au score F1.

Multi-Metric Comparison (Classification)
Dataset: bike_sharing_hour | Target: casual, registered



Performance Heatmap - Test F1
Dataset: bike_sharing_hour | Target: casual, registered



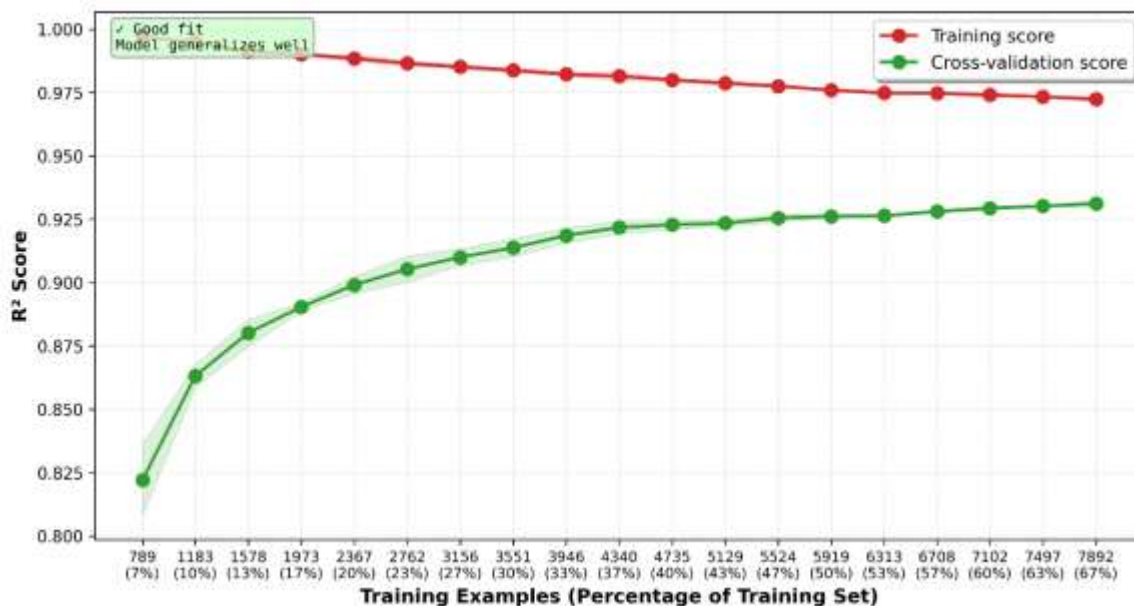


On constate pour le modèle le plus performant (**Bagging**) que la durée augmente très peu et reste raisonnable en passant de CV = 3 à CV = 20 avec une un F1 score qui ne s'améliore plus à partir de CV = 5.

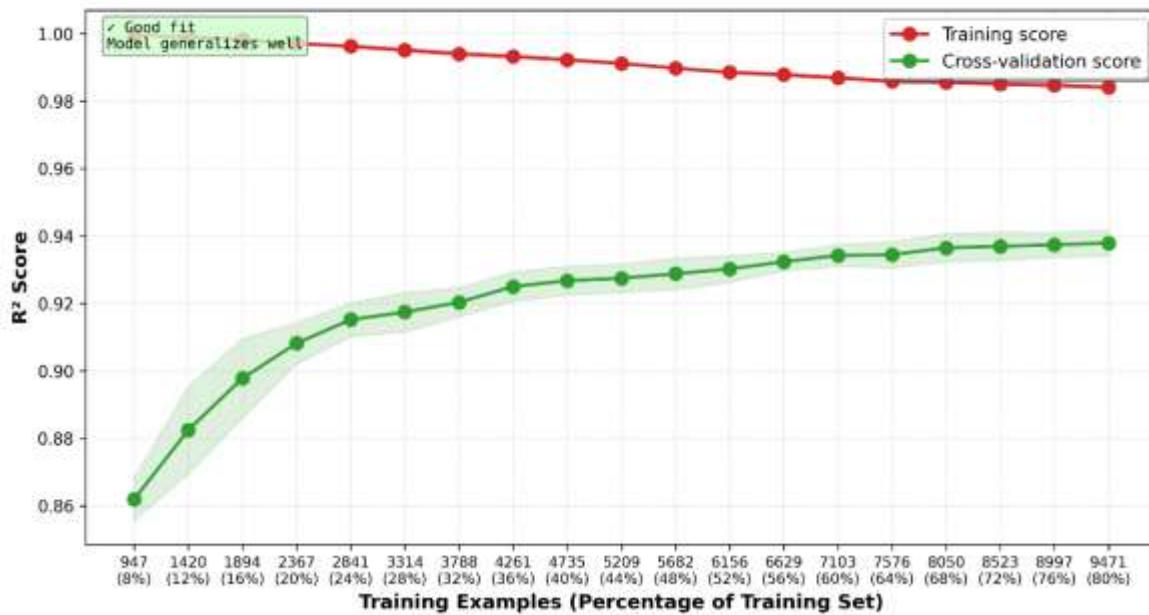
Résultats de la régression

Model	CV_Folds	Tuning_Time_sec	Best_CV_Score	Test_R2	Test_RMSE
RandomForest	3	21.709614	0.918607	0.924444	36.660025
	5	29.028594	0.924253	0.924444	36.660025
DecisionTree	3	9.256246	0.735027	0.745666	67.260688
	5	8.134015	0.739987	0.745666	67.260688
SVM	3	27.478825	0.573337	0.572752	87.176352
	5	40.886768	0.581953	0.572752	87.176352
KNN	3	1.127803	0.593787	0.610371	83.249984
	5	1.313919	0.610831	0.610371	83.249984
Ridge	3	0.249969	0.387860	0.378553	105.138244
	5	0.286026	0.388143	0.378553	105.138264
Lasso	3	0.566113	0.387899	0.378683	105.127270
	5	0.566577	0.388175	0.378683	105.127270
ElasticNet	3	0.651661	0.387962	0.378312	105.158649
	5	0.750695	0.388282	0.378312	105.158649
GradientBoosting	3	7.096844	0.931062	0.933501	34.392822
	5	11.877198	0.935182	0.933501	34.392822
XGBoost	3	3.144980	0.933811	0.935413	33.894623
	5	1.506882	0.938034	0.936279	33.666683
LightGBM	3	6.088810	0.932916	0.936959	33.486564
	5	6.088917	0.936978	0.936599	33.581938
Bagging	3	8.094173	0.923671	0.928334	35.703757
	5	14.813935	0.929767	0.928449	35.675238
Stacking	3	24.795259	0.923198	0.928802	35.587125
	5	34.348507	0.929710	0.928802	35.587125

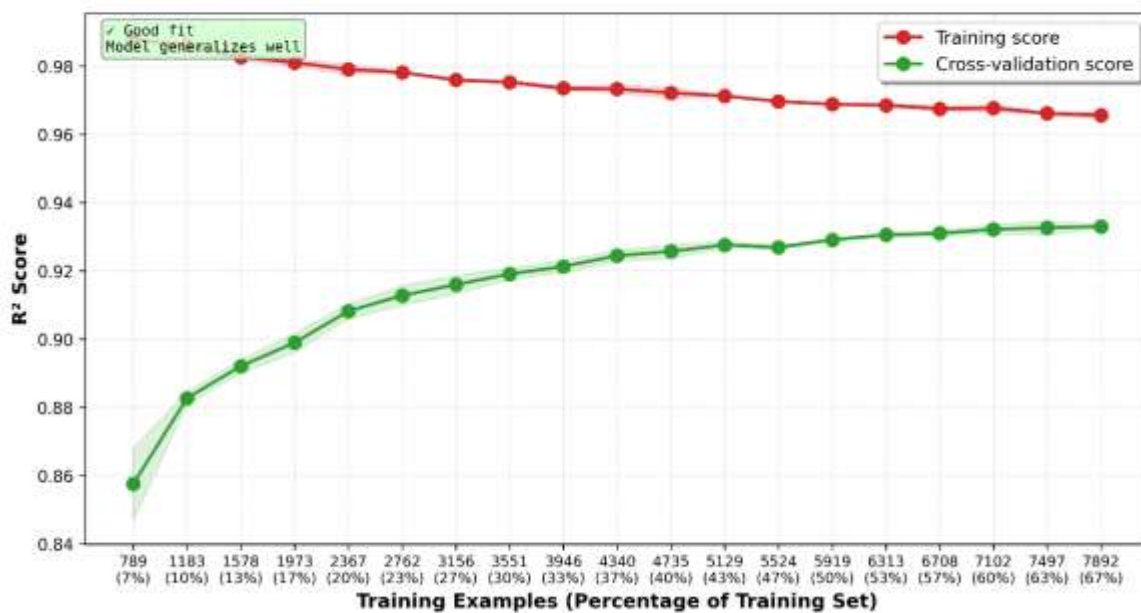
Learning Curve: GradientBoosting (RandomizedSearchCV)
Regression - 3-Fold CV - Scoring: r2

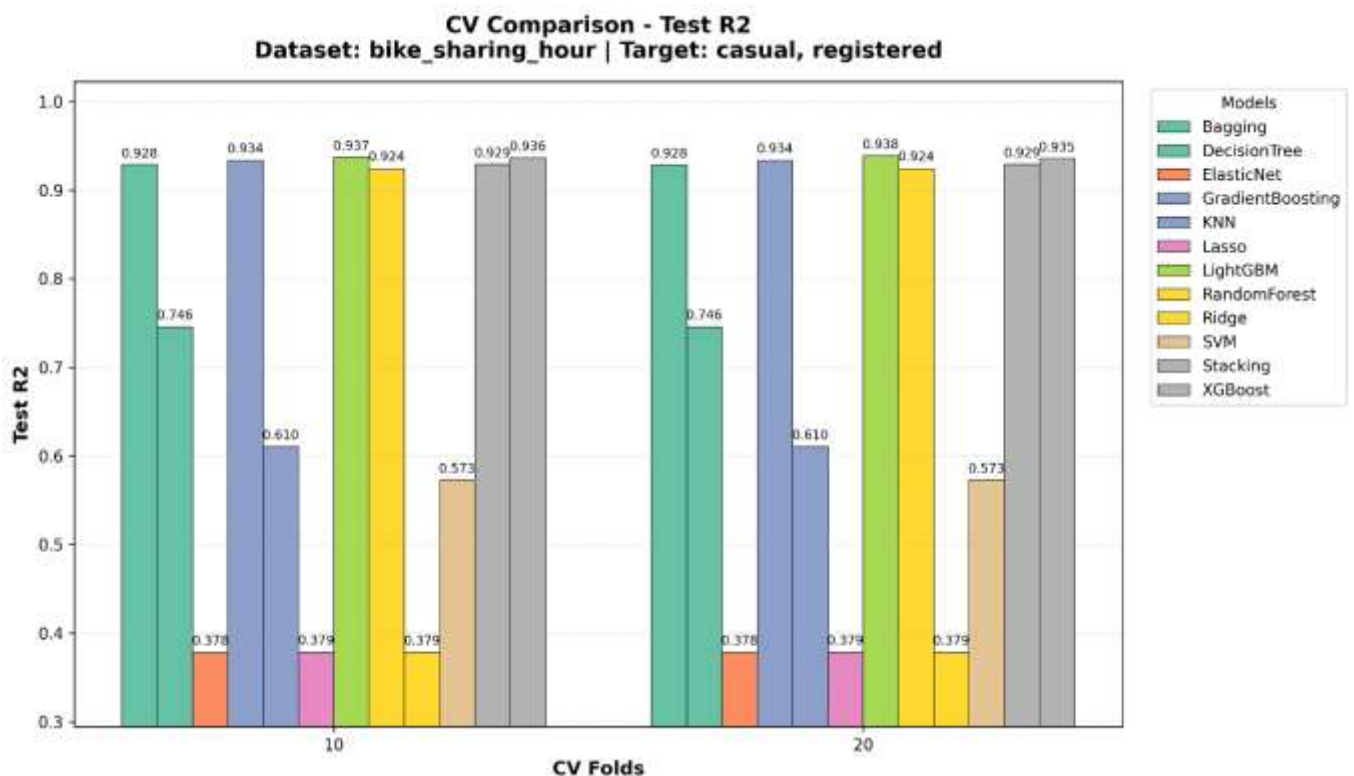
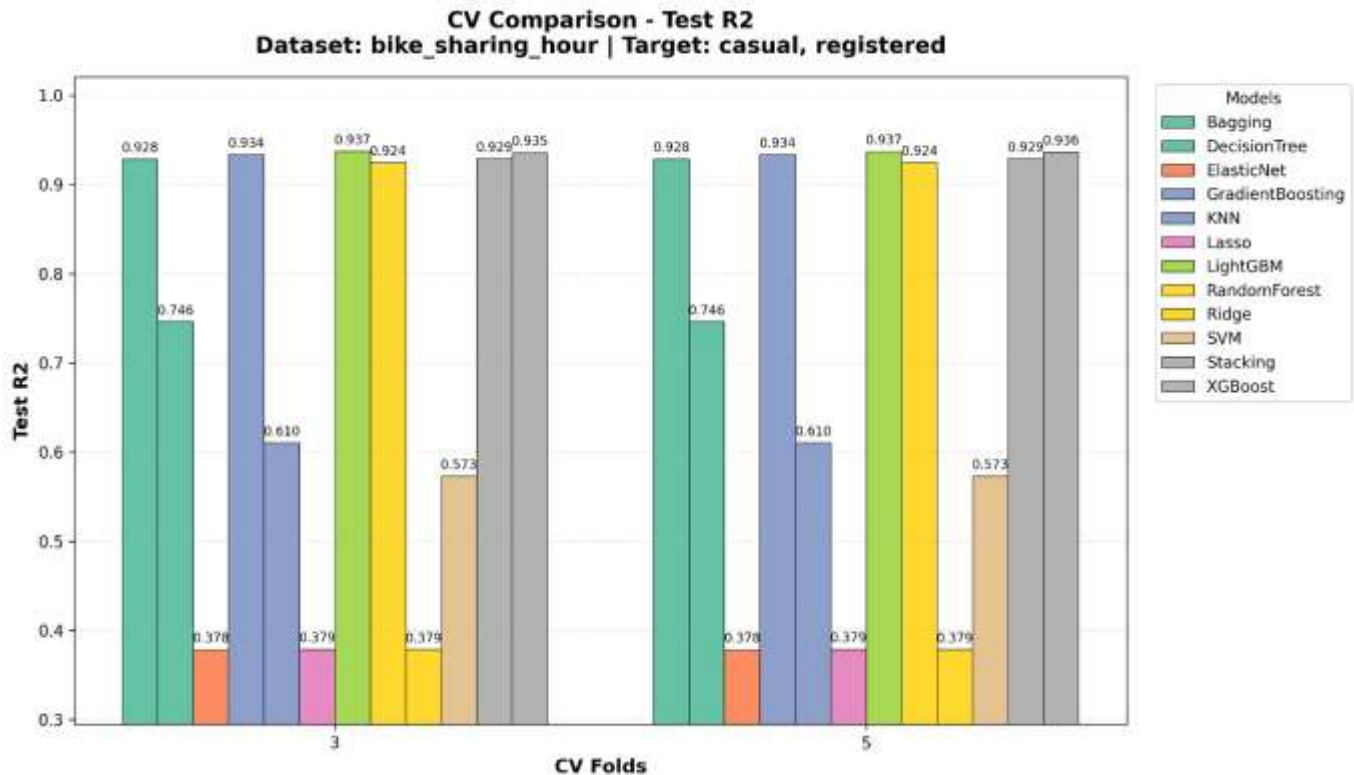


Learning Curve: XGBoost (RandomizedSearchCV)
Regression - 5-Fold CV - Scoring: r2



Learning Curve: LightGBM (RandomizedSearchCV)
Regression - 3-Fold CV - Scoring: r2





On constate que même en augmentant la valeur de CV, les performances demeurent stables ; dans ce cas CV = 3 peut être suffisant.

D. 3^{ème} cas : dataset weather_data_cleaned.csv

- **Dataset CSV** : weather_data_cleaned.csv
- **URL** : <https://www.kaggle.com/datasets/muthuj7/weather-dataset>
- **Dimension du dataset** 📏 : 95 936 × 12 ➔ 1 151 232 entrées (avant cleaning)

D1. Les variables descriptives et les cibles

En comptant le nombre d'occurrences des valeurs manquantes, on obtient :

```
Number of missing values in each column:
Formatted Date          0
Summary                 0
Precip Type             517
Temperature (C)         0
Apparent Temperature (C) 0
Humidity                0
Wind Speed (km/h)       0
Wind Bearing (degrees)  0
Visibility (km)         0
Loud Cover              0
Pressure (millibars)    0
Daily Summary           0
```

Par conséquent, on doit hôter du dataset les 517 lignes où **Precip Type** est manquant par le biais du code ci-dessous par exemple :

```
import pandas as pd

if __name__ == "__main__":
    df = pd.read_csv("weather_data.csv")
    print(df.head())
    print(f"\nNumber of missing values in each column:\n{df.isnull().sum()}")
    df = df[df['Precip Type'].notna()]
    df.to_csv("weather_data_cleaned.csv", index=False)
```

D2. Options de configuration

Paramètres de configuration (dans config.py):

```
DATA_PATH = DATA_DIR / 'weather_data_cleaned.csv'
CSV_DELIMITER = 'auto'

REMOVE_OUTLIERS = True
USE_POLYNOMIAL_FEATURES = False
POLYNOMIAL_DEGREE = 2

AGGREGATION_COLS = ['Temperature (C)']
EXCLUDE_COLS = ['Formatted Date', 'Summary', 'Daily Summary', 'Temperature (C)']

AGGREGATION_NAME = 'sum'

# Model settings
RANDOM_STATE = 42
TEST_SIZE = 0.2
CV_FOLDS = 10
N_JOBS = -1

SEARCH_TYPE = 'random'

N_CLASSES = 'auto'

CATEGORICAL_ENCODING_THRESHOLD = 10
CATEGORICAL_ENCODING_METHOD = 'label'

CLUSTERING_METHOD = 'kmeans'
LEARNING_CURVE_TRAIN_SIZES = [0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, .45, 0.5, .55, 0.6, .65, 0.7, .75, 0.8, .85, 0.9, .95, 1.0]

CLF_LEARNING_CURVES_SCORE = 'both'
REG_LEARNING_CURVES_SCORE = 'both'

GENERATE_PREDS = False
```

L'écartement des lignes contenant des valeurs aberrantes sur la base du critère IQR, a été réalisé afin que le dataset soit équilibrés ; ce qui représente 7,47% de réduction d'enregistrements avec les colonnes concernées y compris la cible Temperature (C) :

```
=====
                        OUTLIER REMOVAL (IQR Method)
=====
IQR multiplieur: 1.5

Rows before: 95936
Rows after: 88767
Rows removed: 7169 (7.47%)

Columns with outliers detected:
Column                Outliers  Lower Bound  Upper Bound
-----
Temperature (C)        39        -16.7563     40.2049
Apparent Temperature (C) 22        -22.5757     43.6965
Humidity                46         0.1650       1.3250
Wind Speed (km/h)       2987       -6.7137      26.6455
Pressure (millibars)     4423       998.1500     1034.7900
=====
```

La détermination du nombre de classes par les méthodes Silhouette, Calinski-Harabasz et Davies-Bouldin et en utilisant l'algorithme KMEANS comme méthode pour déterminer les clusters, donne :

```
=====
                        FINDING OPTIMAL NUMBER OF CLASSES
Testing k values from 2 to 10...
Method: SILHOUETTE

k      Silhouette    Calinski-H    Davies-B    Inertia
-----
2      0.5862         196702.85    0.5550     27601.81
3      0.5494         236826.78    0.5471     14009.86
4      0.5395         286487.55    0.5405     8309.49
5      0.5296         327833.57    0.5393     5627.70
6      0.5233         364080.06    0.5450     4127.02
7      0.5420         426304.07    0.5235     2977.07
8      0.5416         454321.71    0.5208     2410.23
9      0.5384         512612.70    0.5140     1880.74
10     0.5447         573308.43    0.5142     1501.33

Optimal k=2 (highest silhouette score: 0.5862)
=====

Using automatically determined n_classes=2

Single target column: 'Temperature (C)' (no aggregation needed)

Clustering method: KMEANS
Using KMeans clustering to find 2 natural clusters...
Created 2 classes using KMeans clustering
Cluster quality metrics:
  Silhouette Score:      0.5862 (higher is better, -1 to 1)
  Calinski-Harabasz:    196702.85 (higher is better)
  Davies-Bouldin:       0.5550 (lower is better)

Cluster centers ['Temperature (C)']:
  Cluster 0: (20.30)
  Cluster 1: (4.57)

Class 0: 44117 samples
  Range: [12.43, 38.98]

Class 1: 44650 samples
  Range: [-16.71, 12.43]
```

D3. Principaux résultats

Bien que le but consiste à déterminer le meilleur modèle pour prédire la température sur la base des variables descriptives, donc il s'agit bien d'un **problème de régression** et **non de classification**, nous allons toutefois traiter aussi les modèles de classification dans un but d'apprendre plus sur le comportement des modèles de classification.

Résultats de la classification

Model	CV_Folds	Tuning_Time_sec	Best_CV_Score	Test_Accuracy	Test_F1
RandomForest	3	21.031347	1.000000	1.000000	1.000000
RandomForest	5	35.057680	1.000000	1.000000	1.000000
DecisionTree	3	0.536072	1.000000	1.000000	1.000000
DecisionTree	5	0.680059	1.000000	1.000000	1.000000
SVC	3	158.300684	0.999662	0.999944	0.999944
SVC	5	335.970412	0.999718	0.999944	0.999944
KNN	3	11.762129	0.972470	0.974485	0.974483
KNN	5	13.781223	0.973484	0.974034	0.974032
LogisticRegression	3	40.588934	0.999944	0.999887	0.999887
LogisticRegression	5	88.342087	0.999916	0.999887	0.999887
NaiveBayes	3	0.333053	0.798622	0.806016	0.798430
NaiveBayes	5	0.386596	0.798622	0.806016	0.798430
Bagging	3	11.072825	1.000000	1.000000	1.000000
Bagging	5	21.254516	1.000000	1.000000	1.000000
Stacking	3	248.394476	1.000000	1.000000	1.000000
Stacking	5	461.536367	1.000000	1.000000	1.000000

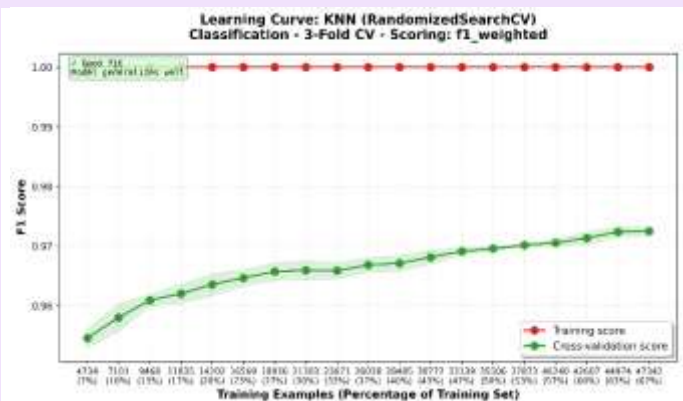
On constate que l'écart entre l'Acc et F1 est presque nul pour tous les modèles, preuve que les 2 classes sont presque parfaitement équilibrées et séparées comme cela a été constaté dans la section précédente. Le modèle **DecisionTree** présente le meilleur résultat avec la durée la plus courte. Quant au modèle **Stacking**, bien qu'il soit performant, il est trop gourmand en temps de traitement !

Exemples des courbes d'apprentissage et matrices de confusion

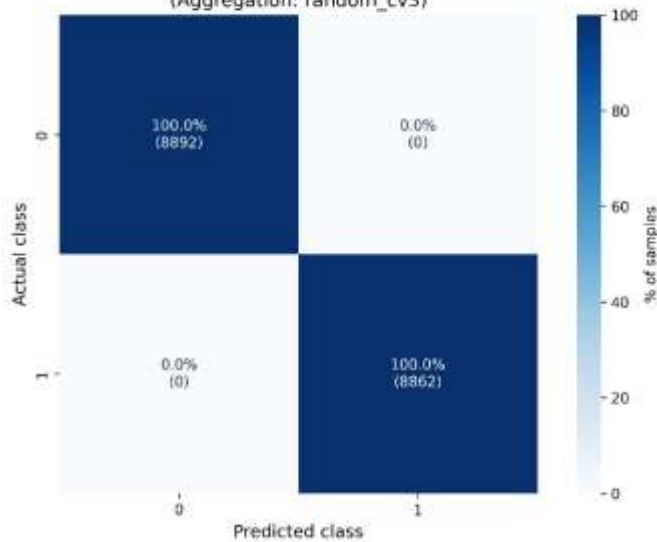
Learning curve Acc du modèle Stacking



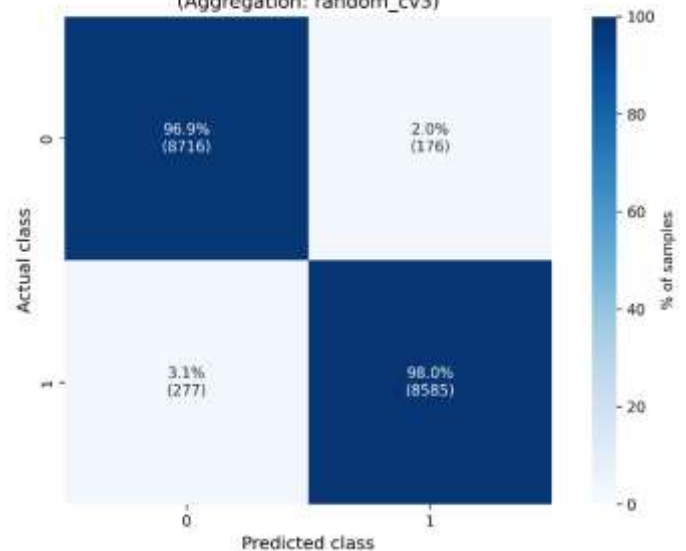
Learning curve Acc du modèle KNN



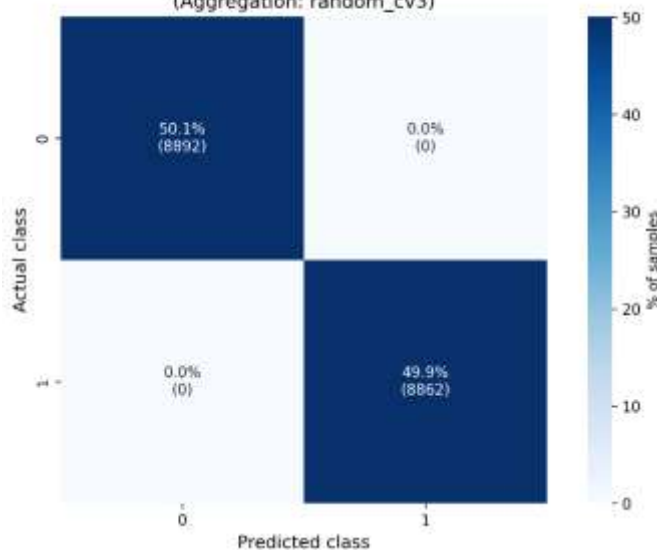
Percentage Confusion Matrix - Stacking
Column-normalized (Precision per class)
(Aggregation: random_cv3)



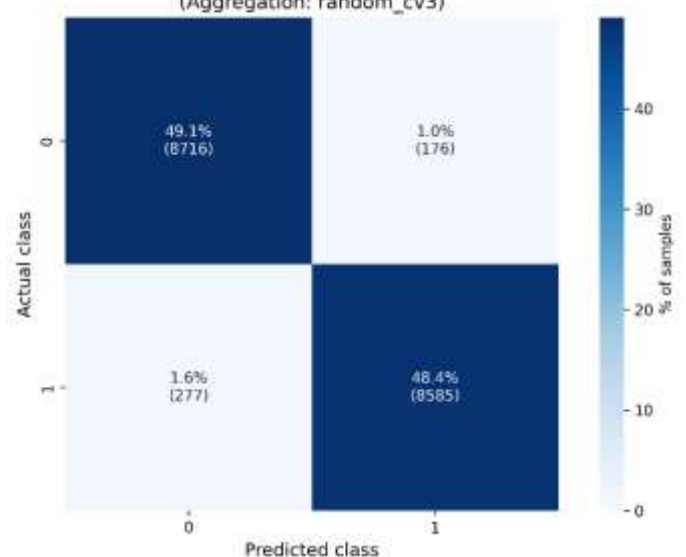
Percentage Confusion Matrix - KNN
Column-normalized (Precision per class)
(Aggregation: random_cv3)

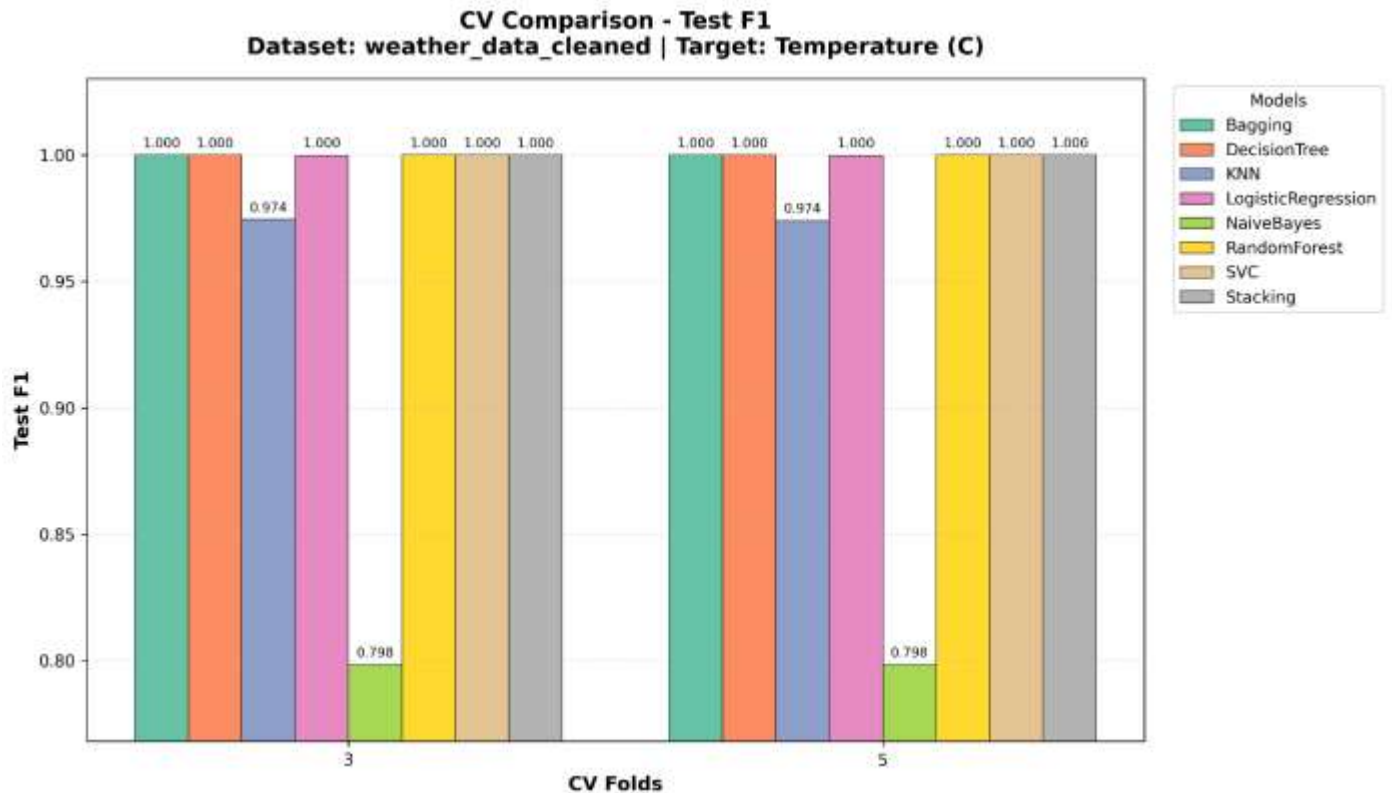


Percentage Confusion Matrix - Stacking
% of total (N=17754)
(Aggregation: random_cv3)

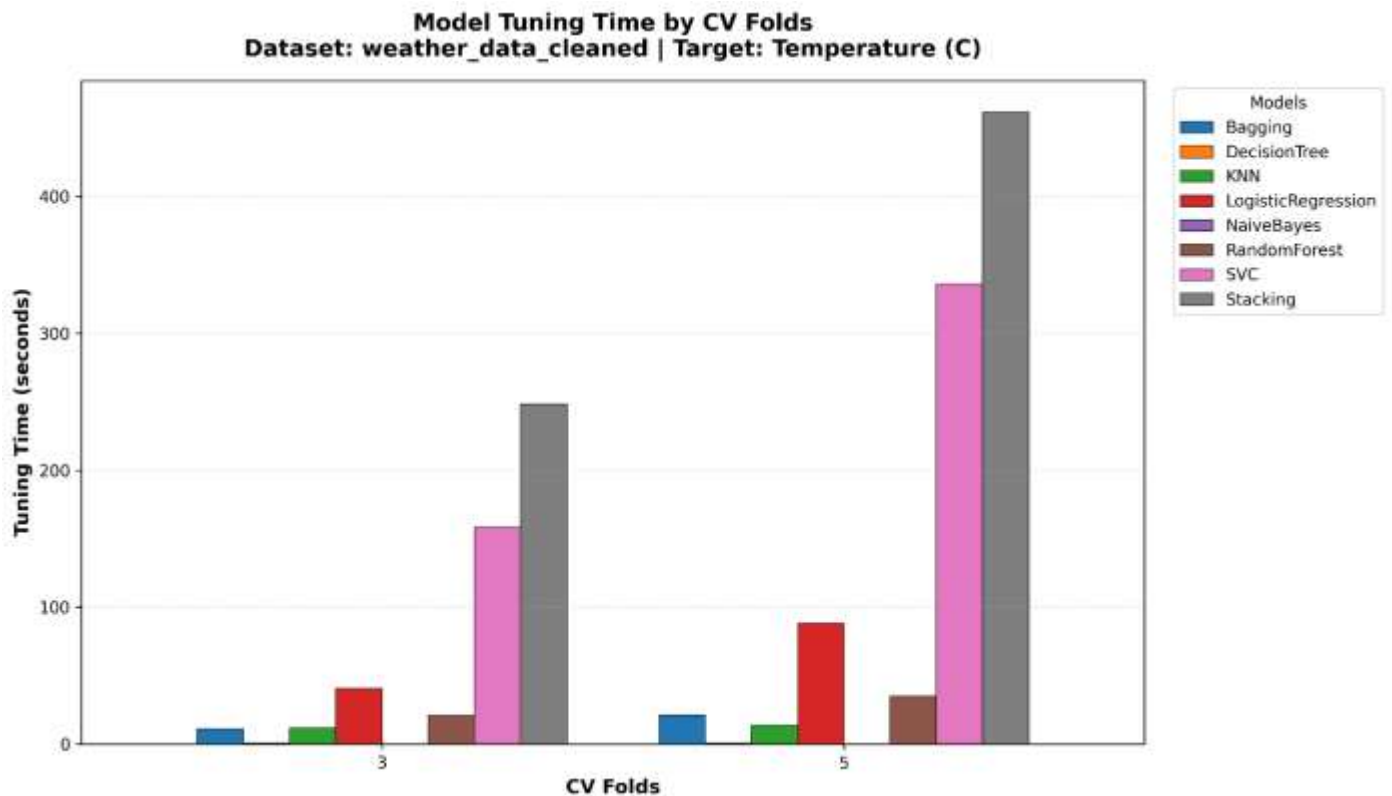


Percentage Confusion Matrix - KNN
% of total (N=17754)
(Aggregation: random_cv3)





On constate que même en augmentant la valeur de CV, les meilleurs modèles restent capés au même score F1 = 1.



Résultats de la régression

Model	CV_Folds	Tuning_Time_sec	Best_CV_Score	Test_R2	Test_RMSE
RandomForest	3	142.457354	0.999968	0.999981	0.041851
RandomForest	3	142,457354	0,999968	0,999981	0,041851
	5	209,778282	0,999973	0,999981	0,041851
DecisionTree	3	89,27391	0,998125	0,997431	0,480692
	5	155,49932	0,998055	0,99799	0,425191
KNN	3	11,815529	0,989567	0,991345	0,8822
	5	15,450207	0,990488	0,991345	0,8822
ElasticNet	3	0,371824	0,991313	0,991188	0,890167
	5	0,482168	0,991312	0,991188	0,890167
GradientBoosting	3	108,472791	0,99996	0,999975	0,047286
	5	178,905958	0,999966	0,999975	0,047286
XGBoost	3	6,135041	0,999824	0,999821	0,126898
	5	7,205111	0,999825	0,999827	0,124819
LightGBM	3	11,675877	0,999824	0,999797	0,13509
	5	14,306422	0,99982	0,999797	0,13509
Bagging	3	37,306115	0,999969	0,999981	0,041297
	5	69,111799	0,999974	0,999981	0,041297
MIN ou MAX:		0,371824	0,999974	0,999981	0,041297

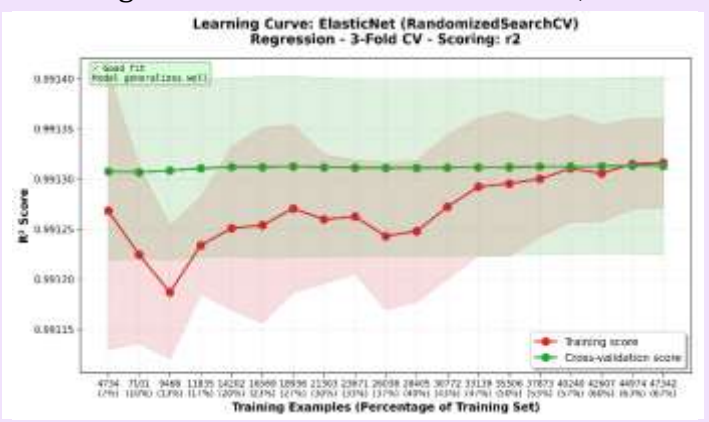
En combinant les performance et le coût en termes de durée d'exécution, le modèle **ElasticNet** paraît à ce stade être un meilleur compromis performance/temps de compilation. On note que le modèle **ElasticNet** converge quasiment dès le départ vers le score Acc cible et reste stable tout au long des itérations, contrairement au set d'entraînement qui converge plus tardivement !

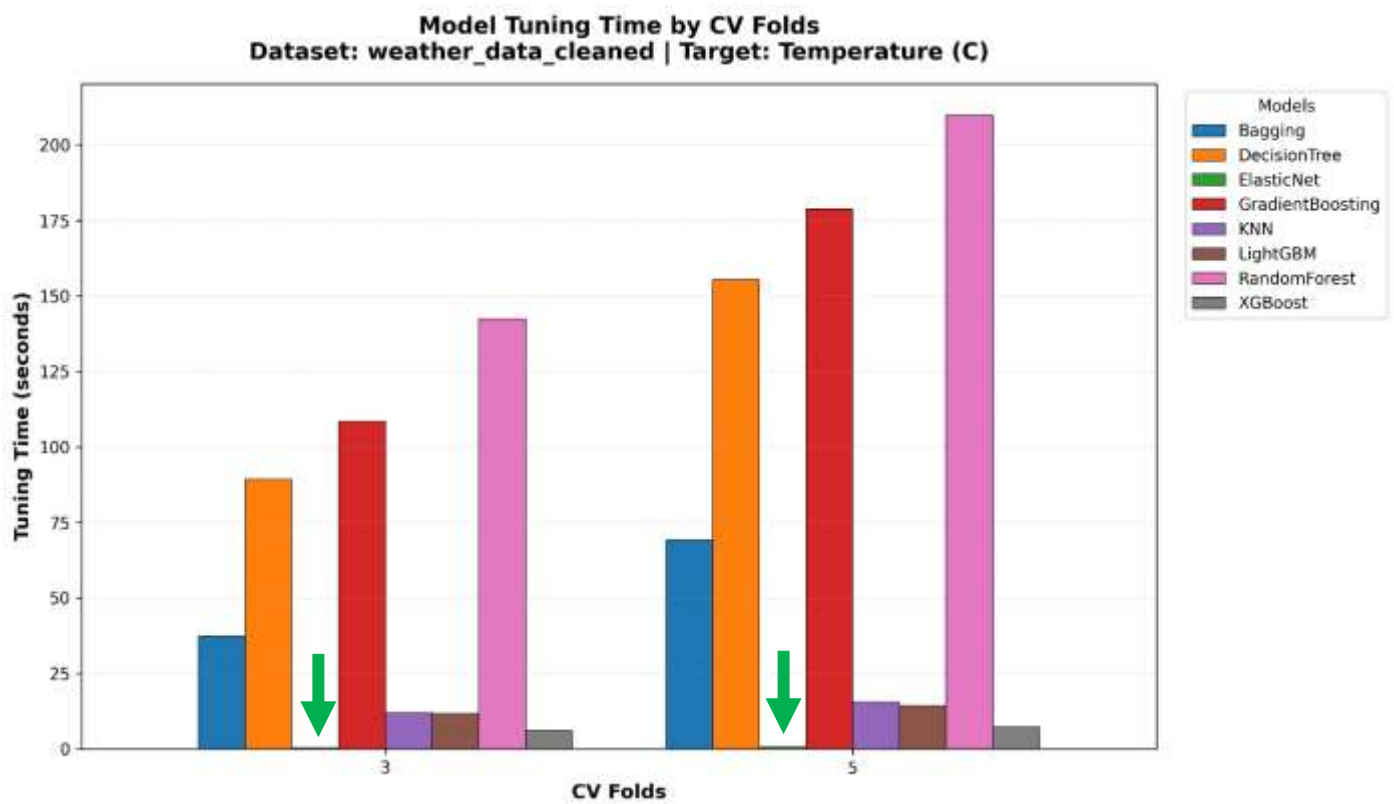
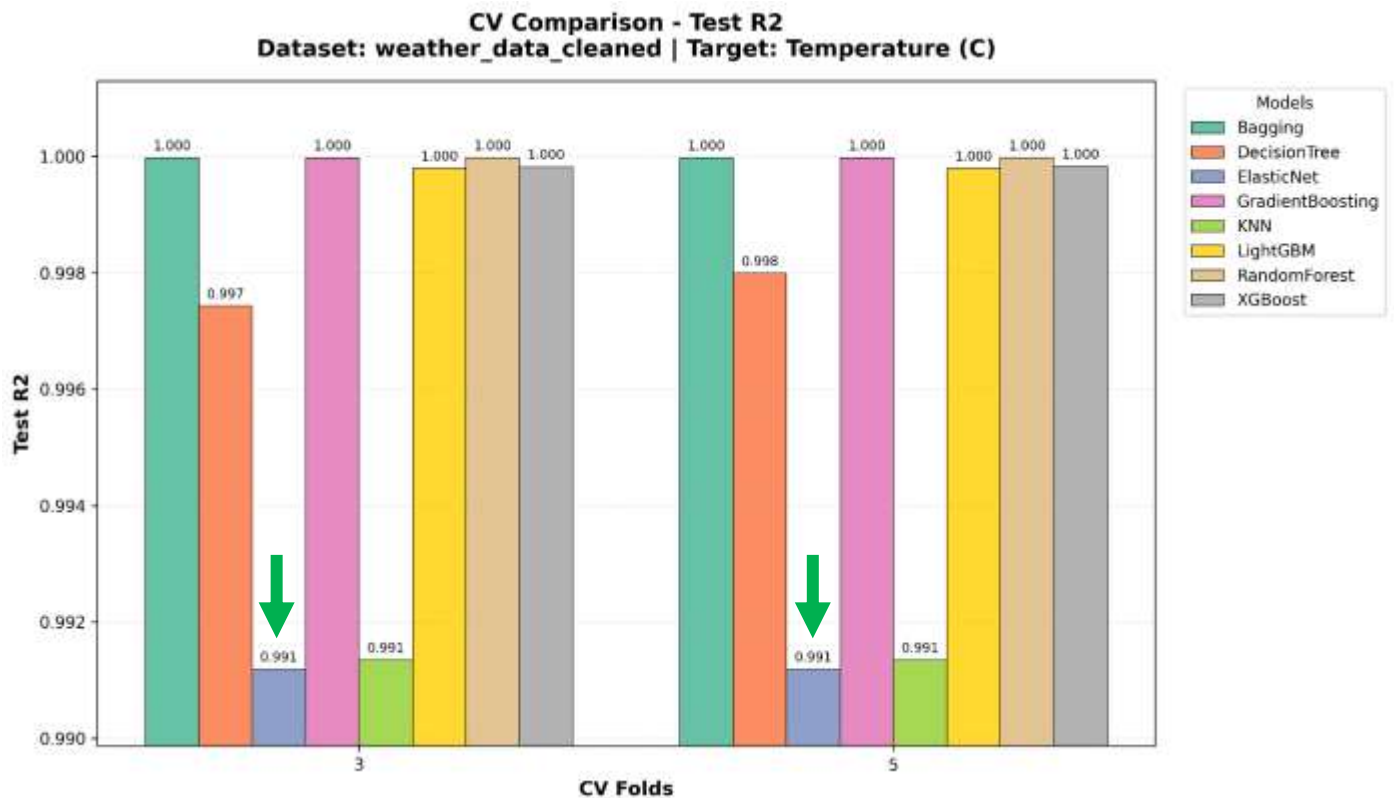
Exemples des courbes d'apprentissage et matrices de confusion

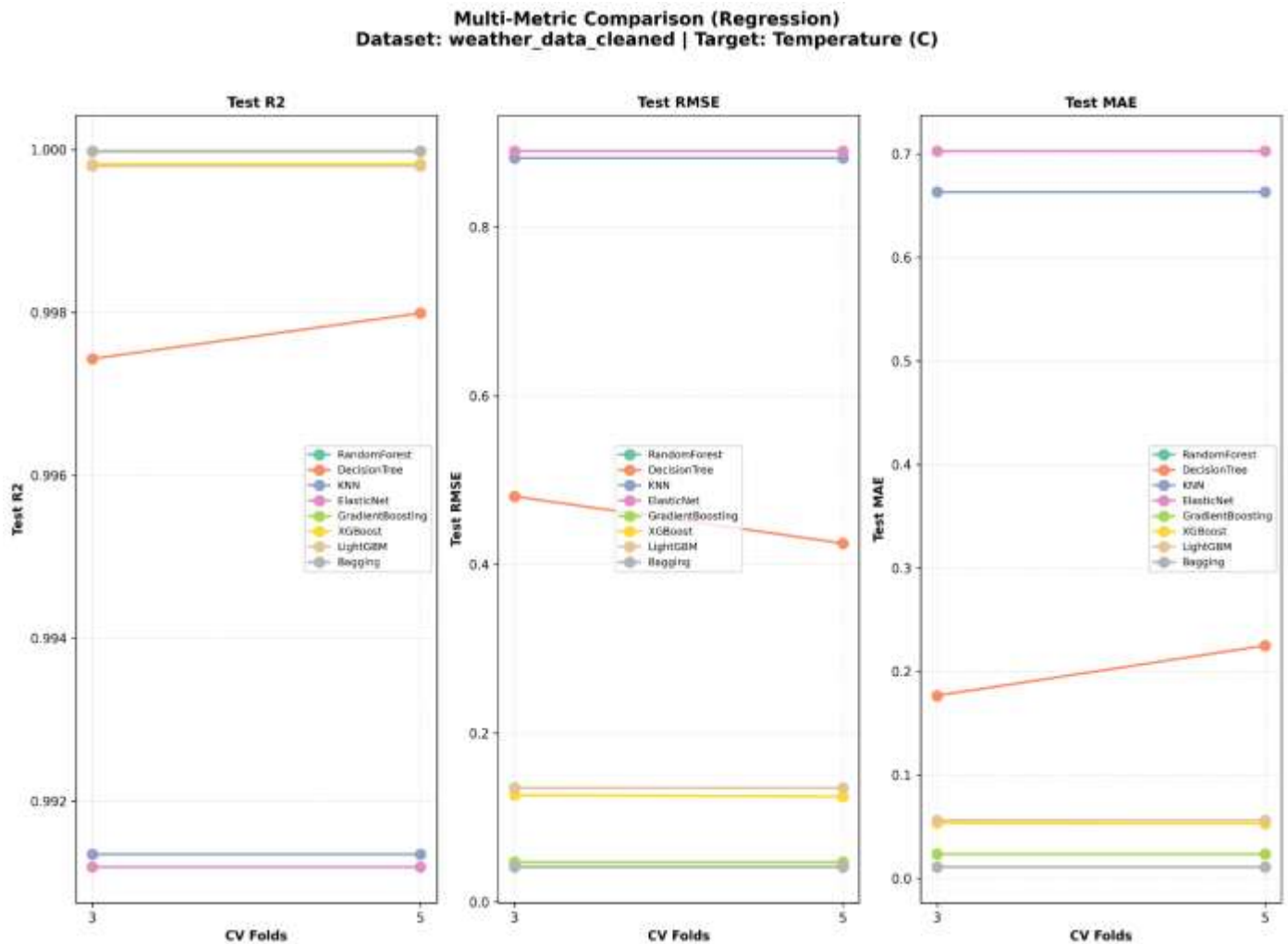
Learning curve Acc du modèle **Bagging**, CV=3



Learning curve Acc du modèle **ElasticNet**, CV = 3







On constate que même en augmentant la valeur de CV, les performances demeurent stables ; dans ce cas CV = 3 peut être suffisant.

Avec cette section, on clôt les sujets autour de la recherche par validation croisée (CV) et les combinaisons possibles des hyperparamètres pour mieux ajuster les modèles et les rendre plus optimisés en fonction du dataset qui doit les alimenter.

Le chapitre suivant traitera le sujet de l'écart entre l'Acc et le F1 score de point de vue mathématique pour tenter d'expliquer les raisons derrière cet écart quand il n'est pas négligeable.

E. Comment quantifier et expliquer l'écart Accuracy vs F1 Score pour une classification binaire

E1. Analyse

Définissons les cardinalités des ensembles des classes + et - comme étant N_+ et N_- . Alors :

$$N = N_+ + N_- \text{ avec } N = \text{cardinalité de tout le dataset}$$

$$N_+ = \text{Cardinalité de l'ensemble des données classées en tant +}$$

$$N_- = \text{Cardinalité de l'ensemble des données classées en tant -}$$

Soit : $\rho = \text{proportion de la classe +}$

$$\text{Donc : } N_+ = \rho N, N_- = (1 - \rho)N$$

Soit :

$$TPR (\text{Proportion des True Positive}) = \alpha$$

$$TNR (\text{Proportion des True Negative}) = \beta$$

Par conséquent, on peut écrire

$$TP = \alpha N_+ = \alpha \rho N$$

$$FN = (1 - \alpha)N_+ = (1 - \alpha)\rho N$$

$$TN = \beta N_- = \beta(1 - \rho)N$$

$$FP = (1 - \beta)N_- = (1 - \beta)(1 - \rho)N$$

Sur la base de la répartition, la matrice de confusion binaire se présente comme suit :

		Classes prédites	
		Positive	Negative
Classes réelles	Positive	$\alpha \rho N$	$(1 - \alpha)\rho N$
	Negative	$(1 - \beta)(1 - \rho)N$	$\beta(1 - \rho)N$

En mettant N en facteur, on obtient la matrice de confusion par proportions :

$$Conf_Matrix = N \times Conf_Rate_matrix = N \begin{pmatrix} \alpha \rho & (1 - \alpha)\rho \\ (1 - \beta)(1 - \rho) & \beta(1 - \rho) \end{pmatrix}$$

On a alors, après remplacement :

$$Acc = Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \alpha \rho + \beta(1 - \rho)$$

$$Precision = \frac{TP}{TP + FP} = \frac{\alpha \rho}{\rho(\alpha - \beta - 1) + (1 - \beta)} = \frac{\alpha \rho}{\alpha \rho + (1 - \beta)(1 - \rho)}$$

$$Recall = \frac{TP}{TP + FN} = \frac{\alpha \rho N}{\alpha \rho N + (1 - \alpha)\rho N} = \alpha$$

$$F1_{score} = \frac{2TP}{2TP + FN + FP} = \frac{2\alpha \rho N}{2\alpha \rho N + (1 - \alpha)\rho N + (1 - \beta)(1 - \rho)N} = \frac{2\alpha \rho}{\alpha \rho + (1 - \beta)(1 - \rho)}$$

Calculons maintenant l'écart $Acc - F1_{score}$:

$$\begin{aligned}\Delta = Acc - F1_{score} &= \alpha\rho + \beta(1 - \rho) - \frac{2\alpha\rho}{\alpha\rho + (1 - \beta)(1 - \rho)} \\ &= \frac{(\alpha\rho + \beta(1 - \rho))(\alpha\rho + (1 - \beta)(1 - \rho)) - 2\alpha\rho}{\alpha\rho + (1 - \beta)(1 - \rho)} \\ &= \frac{2\alpha^2\rho^2 + \alpha\rho(1 - \beta)(1 - \rho) + 2\alpha\beta\rho(1 - \rho) + \beta(1 - \beta)(1 - \rho)^2 - 2\alpha\rho}{\alpha\rho + (1 - \beta)(1 - \rho)} \\ &= \frac{\beta(1 - \rho)(2\alpha\rho + (1 - \beta)(1 - \rho))}{2\alpha\rho + (1 - \beta)(1 - \rho)} = \beta(1 - \rho)\end{aligned}$$

- A. En considérant β (**TNR**) en tant que constante et Δ en tant que fonction de ρ , elle est décroissante, autrement dit, **plus la proportion de la classe + est importante, l'écart diminue linéairement et proportionnellement à β .**
- B. En considérant ρ (*proportion de la classe +*) en tant que constante et Δ en tant que fonction de β (**TNR**), elle est **linéairement croissante ($1 - \rho > 0$)**, notamment pour des valeurs de ρ **trop petites (cas de la classe positive minoritaire/classe négative majoritaire)**.

On conclut ainsi que que l'écart :

$$\Delta = Acc - F1_{score} = \beta(1 - \rho) = TNR \times Proportion\ de\ la\ classe\ - :$$

1. dépend uniquement d'une manière multiplicative :
 - Du bon classement du modèle de la classe négative (**TNR**)
 - De la proportion de la classe négative
2. Est indépendant de la proportion des $TPR = \alpha$
3. Est petit, plus le $\rho = proportion\ de\ la\ classe\ +$, est importante :
 - $\lim_{\rho \rightarrow 1} \Delta = 0$, autrement dit, Acc et F1_Score sont presque identiques
4. Est relativement grand, plus le $\rho = proportion\ de\ la\ classe\ +$, est petit :
 - $\lim_{\rho \rightarrow 0} \Delta = \beta$, autrement dit, l'écart entre Acc et F1_Score est trop important en s'approchant de la proportion **TNR**
5. Atteint sa valeur maximale pour $\beta = 1$, autrement dit, une classification parfaite de la classe négative, càd : $Conf_Rate_matrix = \begin{pmatrix} \alpha\rho & (1 - \alpha)\rho \\ \mathbf{0} & (1 - \rho) \end{pmatrix}$

6. A noter par ailleurs quand le modèle classe parfaitement toutes les classes positives :

$$Conf_Rate_matrix = \begin{pmatrix} \rho & 0 \\ (1 - \beta)(1 - \rho) & \beta(1 - \rho) \end{pmatrix}$$

7. Et en cas de parfaite classification du modèle des classes positives et négatives ($\alpha = 1$ et $\beta = 1$) :

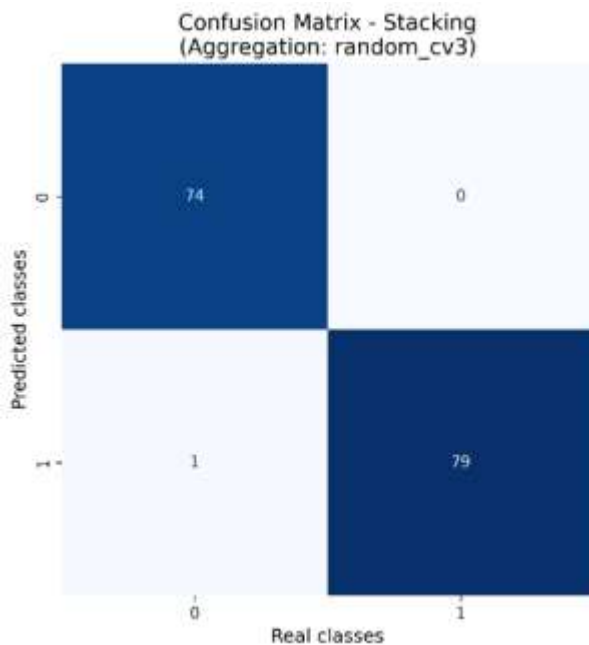
$$Conf_Rate_matrix = \begin{pmatrix} \rho & 0 \\ 0 & 1 - \rho \end{pmatrix}, Conf_matrix = \begin{pmatrix} \rho N & 0 \\ 0 & (1 - \rho)N \end{pmatrix}$$

Dans les matrices de confusion exemples ci-dessous, on indiquera des fois en plus des % les entrées de ces deux matrices.

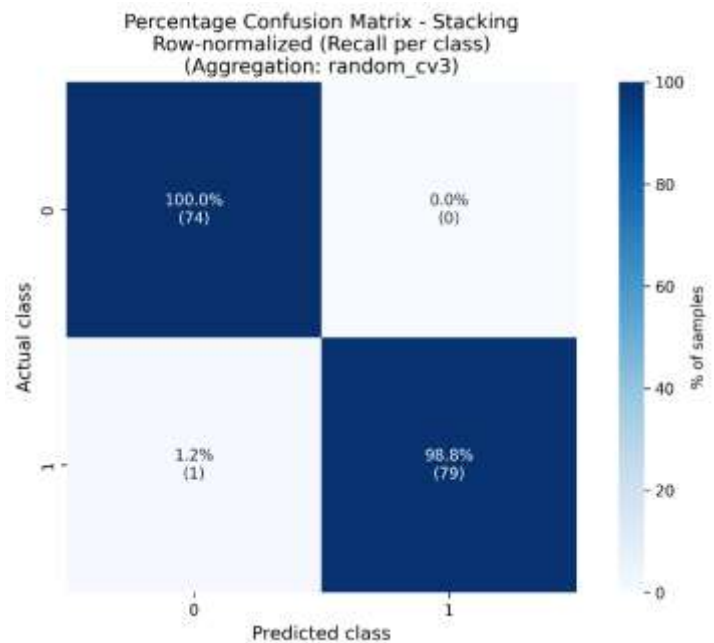
E2. Exemple numérique :

1. $\rho = 75\%$ (.75), $\beta = TNR = 85\%$ (.85) $\rightarrow \Delta = 0.21$ trop important
2. $\rho = 75\%$ (.75), $\beta = TNR = 50\%$ (.5) $\rightarrow \Delta = .12$ moyennement acceptable
3. $\rho = 75\%$ (.75), $\beta = TNR = 25\%$ (.25) $\rightarrow \Delta = .06$ acceptable
4. $\rho = 50\%$ (.5), $\beta = TNR = 10\%$ (.1) $\rightarrow \Delta = .05$ très acceptable

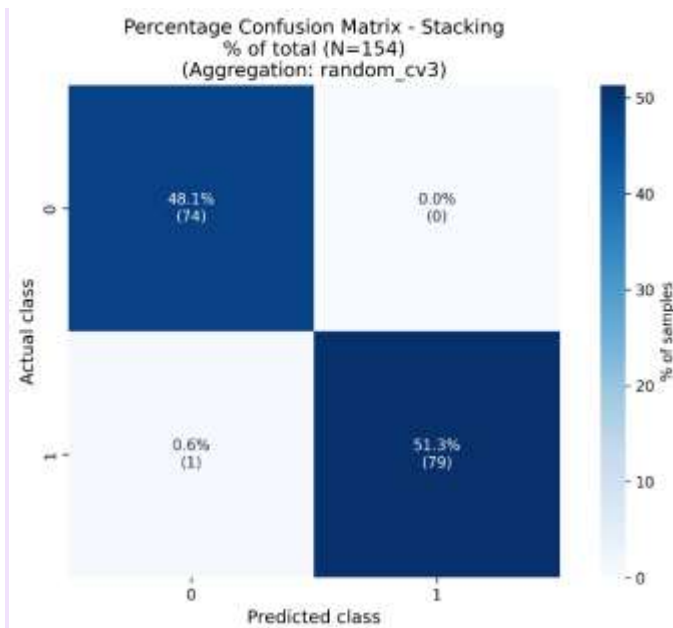
Voici en graphes les matrices de confusion issues de l'exemple B. 1er cas concret de l'application de CV avec CV = 3 et mode = random :



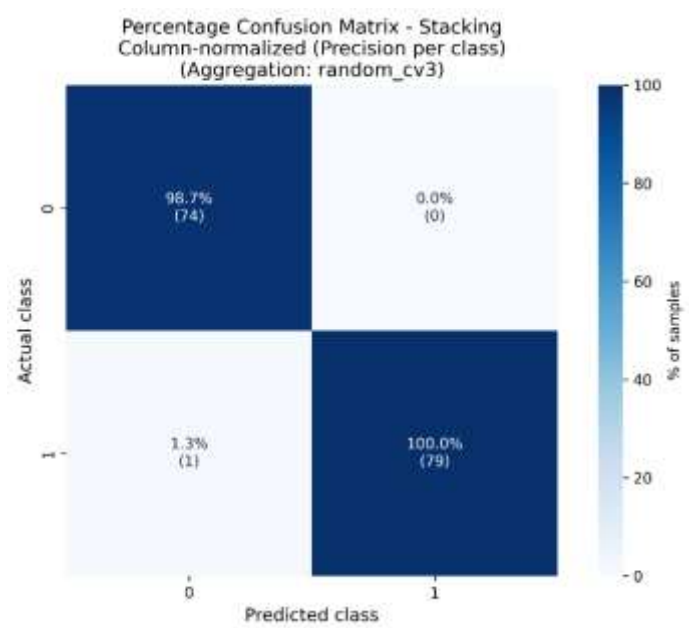
Matrice de confusion en nombre uniquement



Matrice de confusion en % par ligne (nombre)

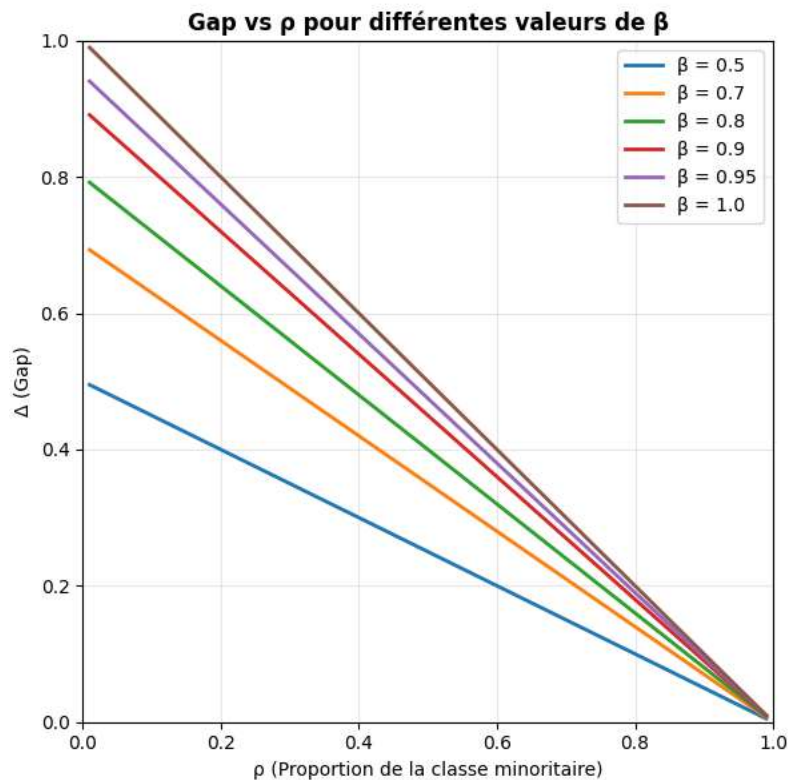


Matrice de confusion en % sur le total (nombre)

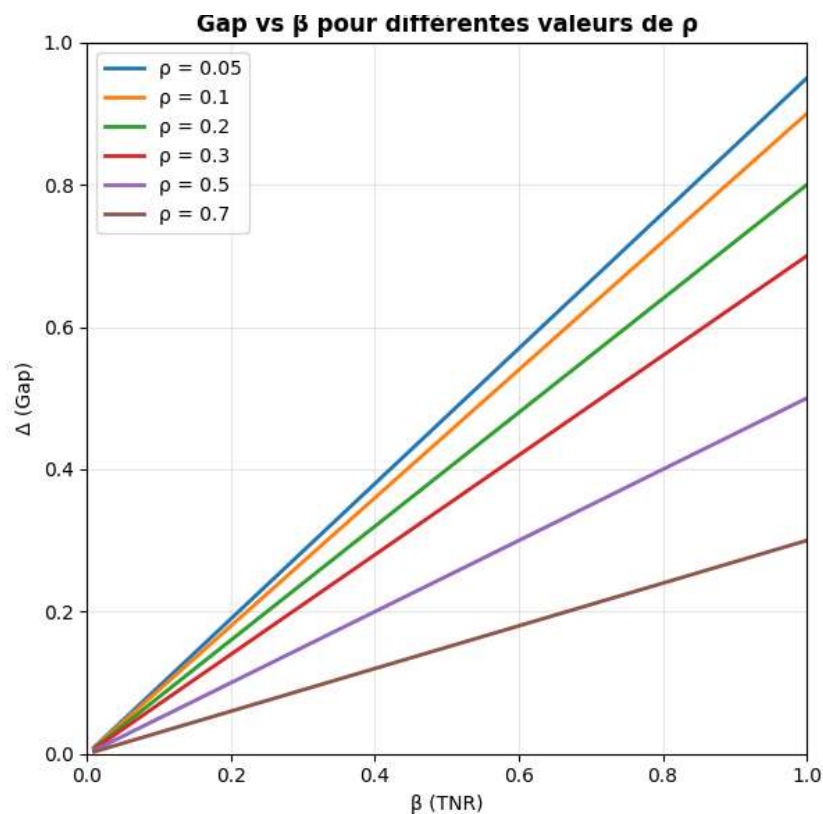


Matrice de confusion en % par colonne, i.e. par classe (nombre)

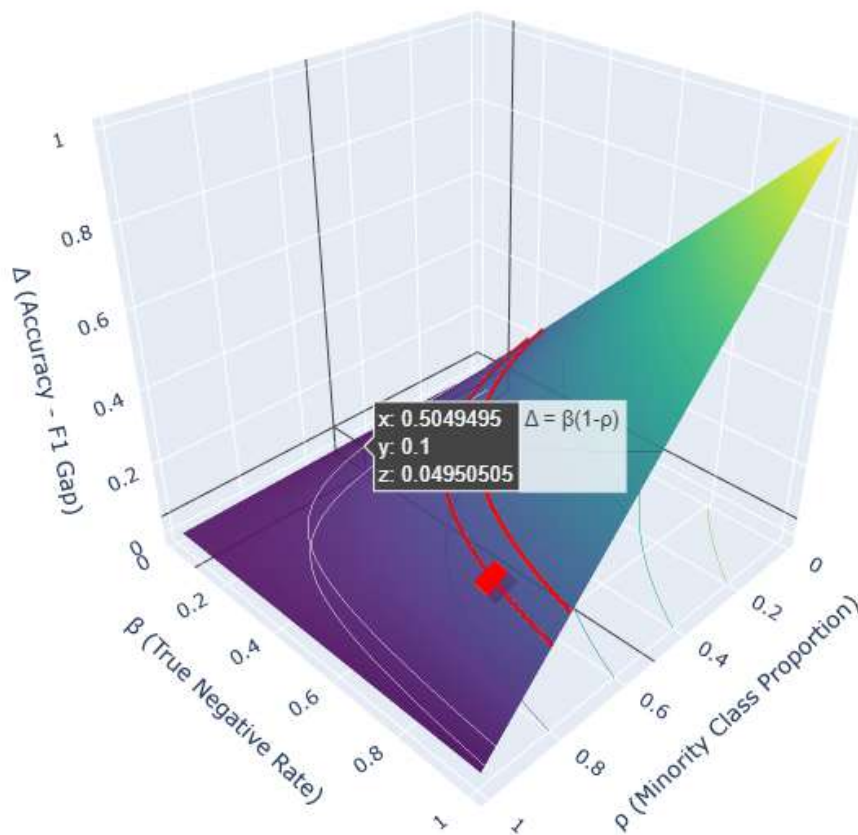
Si on varie β pour différentes valeurs, on obtient les lignes indiquant la variation de Δ en fonction de ρ :



Si on varie ρ pour différentes valeurs, on obtient les lignes indiquant la variation de Δ en fonction de β :



Si on trace la surface en 3D de la fonction à 2 variables $(\beta, \rho) : \Delta = Acc - F1_{score} = \beta(1 - \rho)$ en utilisant **Plotly**, cela donne le graphe suivant avec une mise en évidence de l'exemple 4. ci-dessus :



Annexes

Annexe 1 : Critères de sélection des modèles

Pour la classification

Les métriques ci-dessous sont valables pour une seule classe. Dans le cas d'un modèle multi-classes les métriques utilisées sont exprimées autrement (voir plus bas).

Métriques classe unique (Classification binaire)

Matrice de confusion : Performance par classe de dimension

Nombre de classes actuelles × Nombre de classes prédites

		Classes prédites	
		Négative	Positive
Classes réelles (actuelles)	Negative	True Negative (TN)	False Positive (FP)
	Positive	False Negative (FN)	True Positive (TP)

Abréviations:

TP = True Positive
TN = True Negative
FP = False Positive
FN = False Negative

Les 4 métriques ci-dessous sont basées sur les entrées issues de la matrice de confusion :

Précision (plus **FP** est faible, la précision est meilleure) :

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Positive} = \frac{TP}{TP + FP}$$

Rappel (mesure la sensibilité, plus **FN** est faible, le Rappel est meilleur) :

$$Recall = \frac{TP}{TP + FN}$$

Accuracy (mesure le taux (ratio) du total des classes correctement prédites relativement au total des données à classer toute classe comprise) :

$$Accuracy = \frac{\# \text{ des prédictions correctes}}{\text{Total de toutes les prédictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

F1 Score (quantifié par la moyenne harmonique entre le **Rappel** et la **Précision** de manière à prendre en compte à la fois les **FP** et les **FN**) :

$$F1_{Score} = 2 \cdot \frac{Précision \times Rappel}{Précision + Rappel} = \frac{2TP}{2TP + FN + FP} \quad (\text{ne dépend pas des } TN)$$

AUC-ROC : Aire sous la courbe **ROC** (Receiver Operating Characteristic curve) traçant les valeurs des **TPR** (**True-Positive Rate**) en fonction **FPR** (**False-Positive Rate**) en fixant au préalable un seuil acceptable à atteindre pour le choix d'un modèle, ex. 90%.

Lignes directrices de sélection :

- **Séparation linéaire** : Régression logistique, SVC (linéaire)
- **Non-linéaire** : SVC (RBF), Forêt aléatoire, Gradient Boosting
- **Interprétabilité** : Arbre de décision, Régression logistique
- **Haute précision** : XGBoost, LightGBM, Stacking

- **Petits jeux de données** : Naive Bayes, KNN
- **Grands jeux de données** : LightGBM, modèles linéaires

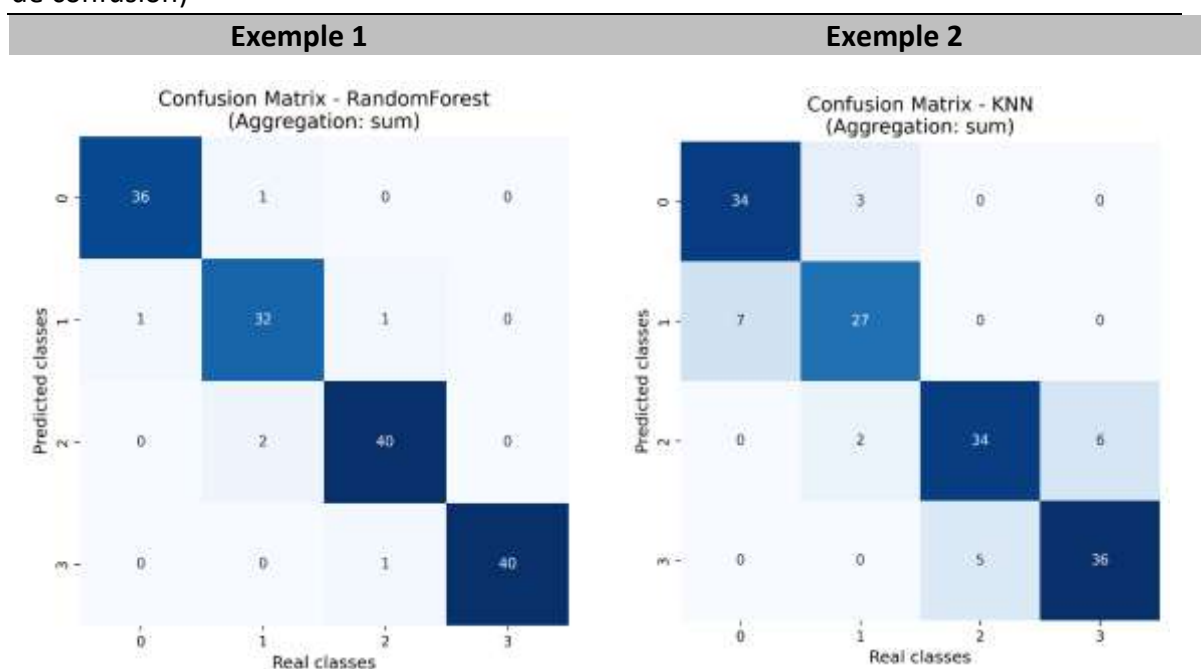
Métriques multi-classes

Les exemples fournis ci-dessous ont été réalisés manuellement pour la bonne compréhension des métriques d'évaluation de base des modèles. Rassurez-vous, des fonctions spécialisées existent pour réaliser ces calculs manuels ! Voir la [section CODE](#) en annexe.

Matrice de confusion : Matrice de performance par classe, de dimension n^2 pour n classes distinctes cibles à prédire

		Classes prédites				
		classe 1	Classe 2	Classe 3	...	Classe n
Classe réelles	Classe 1	$N11$	$N12$	$N13$	...	$N1n$
	Classe 2	$N21$	$N22$	$N23$	...	$N2n$
	Classe 3	$N31$	$N32$	$N33$	...	$N3n$
	...	...	...	...	...	...
	Classe n	$Nn1$	$Nn2$	$Nn3$	...	Nnn

Exemples: 4 classes cibles à prédire notées par 0, 1, 2 et 3 (sur les deux axes de la matrice de confusion)



NOTE : Les exemples de quelques métriques ci-dessous sont basés sur les entrées issues des 2 matrices de confusion.

Précision : 3 types de précision utilisés usuellement faisant appel à des fonctions de moyenne distinctes (**MACRO**, **MICRO** et **WEIGHTED AVERAGE** basé sur la fréquence de chaque classe)

Precision_macro en tant que moyenne simple des Précisions de chaque classe :

$$Precision_macro = \left(\frac{1}{K}\right) \sum_k Precision_k$$

Precision_weighted en tant que moyenne pondérée des Précisions de chaque classe :

$$Precision_weighted = \sum_k W_k Precision_k$$

Où:

W_k = Nombre des échantillons de la classe k correctement prédits

et

$$Precision_k = \frac{TP_{classe\ k}}{TP_{classe\ k} + FP_{classe\ k}}$$

Precision_micro est exprimé de manière à prendre en compte à la fois des **TP** et des **FP** de l'ensemble des classes :

$$Precision_micro = \frac{\sum_k TP_{classe\ k}}{\sum_k TP_{classe\ k} + \sum_k FP_{classe\ k}}$$

Exemple : Reprenons les données issues des [matrices de confusion ci-dessus](#)

Exemple 1 (Random Forest)	Exemple 2 (KNN)
$Precision_0 = \frac{36}{[(36) + (1 + 0 + 0)]} = 0,972972973$	$Precision_0 = \frac{34}{[(34) + (3 + 0 + 0 + 0)]} = 0,918918919$
$Precision_1 = \frac{32}{[(32) + (1 + 1 + 0)]} = 0,941176471$	$Precision_1 = \frac{27}{[(27) + (7 + 0 + 0)]} = 0,794117647$
$Precision_2 = \frac{40}{[(40) + (0 + 2 + 0)]} = 0,952380952$	$Precision_2 = \frac{34}{[(34) + (0 + 2 + 6)]} = 0,80952381$
$Precision_3 = \frac{40}{[(40) + (0 + 0 + 1)]} = 0,975609756$	$Precision_3 = \frac{36}{[(36) + (0 + 0 + 5)]} = 0,87804878$
$Precision_{macro} = \left(\frac{1}{4}\right)(0,972972973 + 0,941176471 + 0,952380952 + 0,975609756) = 0,960535038$	$Precision_{macro} = \left(\frac{1}{4}\right)(0,918918919 + 0,794117647 + 0,80952381 + 0,87804878) = 0,850152289$
$Precision_{micro} = \frac{36 + 32 + 40 + 40}{[(36 + 32 + 40 + 40) + (1 + 2 + 2 + 1)]} = 0,961038961$	$Precision_{micro} = \frac{37 + 0 + 26 + 35}{[(37 + 0 + 26 + 35) + (0 + 34 + 16 + 6)]} = 0,850152289$

Rappel : Les mêmes fonctions d'agrégation sont utilisées pour les différents types de cette métrique

Rappel_macro en tant que moyenne simple des **Rapports** de chaque classe :

$$Recall_macro = \left(\frac{1}{K}\right) \sum_k Recall_k$$

Rappel_weighted en tant que moyenne pondérée des **Rapports** de chaque classe :

$$Recall_weighted = \sum_k W_k Recall_k$$

Où:

W_k = Nombre des échantillons de la classe k correctement prédits

et

$$Recall_k = \frac{TP_{classe\ k}}{TP_{classe\ k} + FN_{classe\ k}}$$

Rappel_micro est exprimé de manière à prendre en compte à la fois des **TP** et des **FN** de l'ensemble des classes :

$$Recall_micro = \frac{\sum_k TP_{classe\ k}}{\sum_k TP_{classe\ k} + \sum_k FN_{classe\ k}}$$

Reprenons les données issues des [matrices de confusion ci-dessus](#).

Exemple 1 (Random Forest):

$$Recall_micro = \frac{36 + 32 + 40 + 40}{(36 + 32 + 40 + 40) + (1 + 0 + 0) + (1 + 1 + 0) + (0 + 2 + 0) + (0 + 0 + 1)} = 0,961038961$$

Exemple 2 (KNN):

$$Recall_micro = \frac{34 + 27 + 34 + 36}{(34 + 27 + 34 + 36) + (3 + 0 + 0) + (7 + 0 + 0) + (0 + 2 + 6) + (0 + 0 + 5)} = 0,850649351$$

Accuracy : mesure le taux (ratio) du total des classes correctement prédites relativement au total des données à classer toute classe comprise

$$Accuracy = \frac{\# \text{ des prédictions correctes}}{\text{Total de toutes les prédictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Reprenons les données issues des [matrices de confusion ci-dessus](#).

Exemple 1 (Random Forest):

$$Accuracy = \frac{36 + 32 + 40 + 40}{(36 + 1 + 0 + 0) + (1 + 32 + 1 + 0) + (0 + 2 + 40 + 0) + (0 + 0 + 1 + 40)} = 0,961038961$$

Exemple 2 (KNN):

$$Accuracy = \frac{34 + 27 + 34 + 36}{(34 + 3 + 0 + 0) + (27 + 7 + 0 + 0) + (34 + 0 + 2 + 6) + (36 + 0 + 0 + 5)} = 0,850649351$$

F1 Score : 3 types

F1_macro en tant que moyenne simple des F1 de chaque classe :

$$F1_{macro} = \left(\frac{1}{K}\right) \sum_k F1_k$$

F1_weighted en tant que moyenne pondérée des F1 de chaque classe :

$$F1_{weighted} = \sum_k W_k F1_k$$

Où:

W_k = Nombre des échantillons de la classe k correctement prédits

et

$$F1_k = 2 \cdot \frac{Précision_k \times Rappel_k}{Précision_k + Rappel_k}$$

Le **F1_micro** quantifié par la moyenne harmonique entre le **Recall_micro** et la **Precision_micro** de manière à prendre en compte à la fois les **TP**, **FP** et **FN** :

$$F1_{micro} = 2 \cdot \frac{Précision_{micro} \times Rappel_{micro}}{Précision_{micro} + Rappel_{micro}}$$

NOTE : Dans le cas d'un modèle multi-classes, on a :

$$F1_{micro} = Precision_{micro} = Recall_{micro} = Accuracy$$

AUC-ROC : Les aires de l'ensemble des classes à prédire sous leurs courbes **ROC** respectives y compris la courbe représentant la métrique moyenne **Micro-Average** permettant de mesurer la performance de l'ensemble des classes.

AUC_macro en tant que moyenne simple des AUC de chaque classe :

$$AUC_{macro} = \left(\frac{1}{K}\right) \sum_1^K AUC_k$$

AUC_weighted en tant que moyenne pondérée des AUC de chaque classe AUC_k :

$$AUC_{weighted} = \sum_k \frac{n_k}{n} AUC_k$$

Où:

n_k = Nombre des échantillons de la classe k

AUC_micro quantifié par la moyenne harmonique entre le **Recall_micro** et la **Precision_micro** de manière à prendre en compte à la fois les **TP**, **FP** et **FN** :

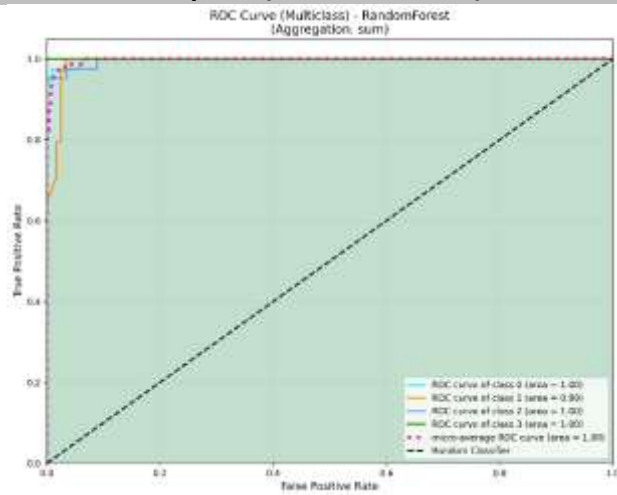
$$TPR_{micro} = \frac{\sum_k TP_{classe\ k}}{(\sum_k TP_{classe\ k} + \sum_k FP_{classe\ k})}$$

$$FPR_{micro} = \frac{\sum_k FP_{classe\ k}}{(\sum_k TF + \sum_k TN_{classe\ k})}$$

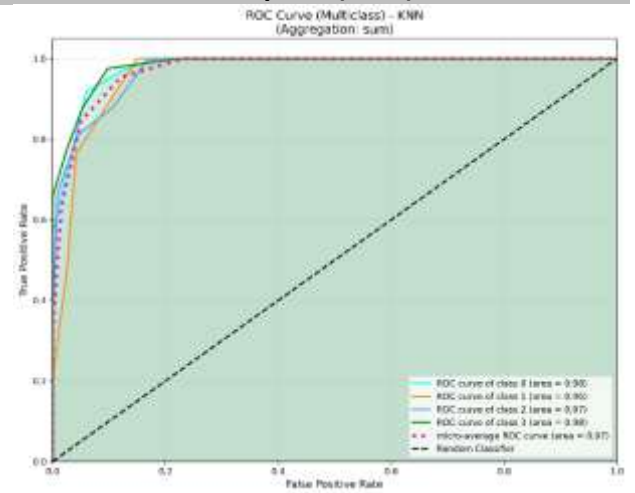
AUC_{micro} = l'aire comprise sous la courbe reliant TPR_{micro} et FPR_{micro}

Exemples: 4 classes cibles à prédire notées par 0, 1, 2 et 3

Exemple 1 (Random Forest)



Exemple 2 (KNN)



Pour la régression

Métriques

Score de performance

Score R^2 : sert à expliquer la variance relativement au baseline et à l'interpréter

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

$$= 1 - \frac{MSE}{VAR(y)}$$

où :

- Somme des résidus au carré:

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = n \times MSE$$

avec: $\hat{y}_i$ = Valeur prédite pour l'échantillon i

- Somme des carrés des distances Euclésiennes entre les valeurs actuelles et leur moyenne globale :

$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2 = n \times Var(y)$$

- Moyenne globale des valeurs actuelles :

$$\bar{y} = \left(\frac{1}{n}\right) \sum_{i=1}^n y_i$$

- La variance de la variable cible y (Moyenne de la somme des carrés des distances Euclésiennes entre les valeurs actuelles et leur moyenne globale) :

$$Var(y) = \frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n}$$

Erreurs quantitatives**MSE :**

$$MSE = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

où :

 n = Total des échantillons y_i = Valeur actuelle pour l'échantillon i $\hat{y}_i$ = Valeur prédite pour l'échantillon i

Plus la valeur de la **MSE** est petite (relativement aux autres modèles comparatifs), meilleur il est le modèle. Elle pénalise fortement les erreurs issues des variables ayant des grandes valeurs par rapport au reste, comme les variables ayant des valeurs aberrantes (outliers).

RMSE :

$$RMSE = \sqrt{MSE}$$

La métrique **RMSE** est exprimée dans la même unité de mesure que celle des échantillons, ce qui est plus significatif avec une estimation de l'erreur plus juste des variables cibles. Plus sa valeur est petite (relativement aux autres modèles comparatifs), meilleur il est le modèle.

MAE :

$$MAE = \frac{1}{n} \sum_i |y_i - \hat{y}_i|$$

Plus la valeur de la **MAE** est petite (relativement aux autres modèles comparatifs), meilleur il est le modèle. Contrairement aux métriques **MSE** et **RMSE**, cette métrique est moins sensible aux variables ayant des valeurs aberrantes (outliers) du fait de l'absence du carré, et elle est exprimée dans la même unité que les variables cibles.

MAPE (Mean Absolute Percentage Error) :

$$MAPE = \frac{100}{n} \sum_{i=1} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Elle exprime tout simplement en % la moyenne des écarts relatifs absolues des valeurs actuelles par rapport aux valeurs prédites. Cette métrique sert à matérialiser numériquement l'interprétabilité de l'exactitude (Accuracy) du modèle :

Valeur de la MAPE	Interprétabilité
< 10%	Une Accuracy très performante
Entre 10% et 20%	Une bonne Accuracy
Entre 20% et 50%	Une Accuracy acceptable
> 50%	Une Accuracy plutôt pauvre

Il est recommandé de l'utiliser uniquement dans les situations :

2. Les valeurs cibles sont > 0
3. L'erreur relative fournit plus d'information d'interprétabilité que l'erreur absolue (MAE)
4. On veut comparer plusieurs modèles avec des échelles différents comme par exemple dans le domaine de commerce, les prévisions des ventes futures pour différents produits

Lignes directrices de sélection :

- **Relation linéaire** : Linéaire, Ridge, Lasso
- **Sélection de variables** : Lasso, ElasticNet
- **Non-linéaire** : Forêt aléatoire, XGBoost, LightGBM
- **Interprétabilité** : Régression linéaire, Arbre de décision
- **Robustesse** : Forêt aléatoire, Gradient Boosting
- **Vitesse** : Modèles linéaires, KNN

